

HAMON — Documentation

Harmony Model and Notation — a universal, lossless harmony-label interlingua

OMR-PRAIG-UA-ES

2026-09-13

Contents

HAMON — Harmony Encoding Formats	3
Tutorial: encoding harmony with HAMON	10
Processing harmony, format by format (CLI & Python)	15
Using HAMON in your editor / environment	19
MusicXML — Harmony Encoding	22
MEI — Harmony Encoding	27
Humdrum **kern — Harmony Encoding	35
LilyPond — Harmony Encoding	43
ABC Notation — Harmony Encoding	48
MuseScore — Harmony Encoding	51
MNX — Harmony Encoding	55
DCML Annotation Format	57
Harte Notation	62
iReal Pro Format	65
Dezrann (‘.dez’) Integration	67
JAMS	69
music21 Integration	71
ms3 Integration	75
partitura Integration	77
FlexOHR	79
HAMON — Analytical Layer (v0.2.0)	81
Berklee-style jazz analysis & the MuRET-hamon pipeline	91
Time-aligned positions (measures & beats)	93

Systems, the '@' hint, and the analytical layer	98
Datasets & the harmony hub	100
Command-line interface ('hamon' / 'python -m hamonpy')	103
Regex-based extractors	106
HAMON Architecture (Mermaid UML)	108
HAMON Grammar Internals (Mermaid)	113
HAMON ↔ Verovio integration	120

HAMON — Harmony Encoding Formats

HAMON — **H**armony **M**odel and **N**otation — is a universal harmony-label standard: a stable exchange layer that sits between music-encoding ecosystems. Everything hinges on one **canonical JSON structure** (`grammar/hamon-schema.json`): it is the normative hub, and every external format maps to and from it. As long as the source format is expressive enough, round-trips lose nothing.

The **EBNF grammar** (`grammar/hamon.ebnf`) is a compact text serialization of that JSON — a human-readable surface string like `CΔ7`, `V7`, or `6-5`. Think of it as one encoding of the JSON, not the hub itself.

This documentation walks through how each supported format encodes harmony, what it can and cannot represent, and how HAMON maps to and from it.

New to HAMON? Start with the [tutorial](#) — encoding harmony in HAMON surface and HAMON JSON, from a single chord to a positioned, analysed progression, with worked examples.

Just want to process a file? See the [processing cookbook](#) — read / export / report / validate for each format, in both the hamon CLI and Python.

Working in an editor? [Using HAMON in your editor / environment](#) covers PyCharm/IntelliJ, VS Code, Jupyter and digital-musicology tools.

Offline / print? The whole of this documentation is also built as a single **PDF** (re-generated by `tools/scripts/build_all.sh`).

Supported formats

Notation file formats

Format	File type	hamonpy adapter	Fixtures
MusicXML	<code>.xml</code> / <code>.mxl</code>	☐	☐ 14
MEI	<code>.xml</code>	☐	☐ 26
Humdrum **kern	<code>.krn</code>	☐	☐ 29
LilyPond	<code>.ly</code>	☐ (regex)	☐ 26
ABC	<code>.abc</code>	☐ (regex)	☐ 26
MuseScore MSCX	<code>.mscx</code> / <code>.mscz</code>	☐ (regex)	☐ 26
MNX	<code>.mnx</code> (JSON)	☐ (planned)	☐

Third-party annotation and audio formats

Format	File type	hamonpy adapter	Fixtures
DCML	.tsv	□ (dcm1.py)	□ 4
DCML expanded (Hentschel time-aligned + cadences/phrases)	.tsv	□ (dcm1_expanded.py)	—
DiLeMMa pitch arrays (note-level GNN tables: DLC + AugmentedNet)	.tsv	□ (dilemma.py)	—
Harte	.txt / .lab	□ (harte.py)	□ 4
iReal Pro	irealb:// URI / .txt	□ (ireal.py)	□ 3
Dezrann	.dez (JSON)	□ (dezrann.py)	—
JAMS	.jams (JSON)	□ (jams.py)	—
music21	music21.stream.Stream · .hamon	□ (music21_adapter.py + native I/O format)	—
ms3 (MuseScore → DCML expanded)	.mscx / .mscz	□ (ms3_adapter.py, optional dep)	—
partitura	partitura Part	□ (partitura_adapter.py, optional dep)	—
FlexOHR (DCMLab harmony object model)	flexohr.OHR	□ (flexohr.py, optional dep)	—
Illescas	TBD	—	□ (pending)

music21 integration is more than a Stream↔HAMON adapter. Chord symbols map to native ChordSymbol figures (slash bass, suspensions, added tones, alterations), and roman numerals become RomanNumeral objects resolved against the key of the analytical @key: regions (secondary dominants, borrowed degrees, inversions). On top of that, .hamon is registered as a **native music21 format** — converter.parse(..., format="hamon") and stream.write("hamon") — via register_hamon_format(). See [music21.md](#).

Capability matrix — which format carries which system

The two axes are **orthogonal**: rows are the encodings and libraries, columns are the harmony **systems** (and the key analytical aspects). A ✓ means the format can carry that aspect **natively**. HAMON is the only one that carries them all — which is exactly why it works as the interlingua.

Format / library	Chord sym.	Roman	Fig. bass	Functional	Nashville	Applied	Key/region
HAMON (interlin- gua)	✓	✓	✓	✓	✓	✓	✓
MEI	✓	—	✓	—	—	—	—

Format / library	Chord sym.	Roman	Fig. bass	Functional	Nashville	Applied	Key/region
Humdrum	✓	✓	✓	✓	–	✓	✓
MusicXML	✓	✓	–	–	–	–	–
Dezrann	–	–	–	–	–	–	–
DCML	–	✓	–	–	–	✓	✓
RomanText	–	✓	–	–	–	✓	✓
Harte	✓	–	–	–	–	–	–
iReal Pro							
JAMS	✓	–	–	–	–	–	✓
LilyPond	✓	–	–	–	–	–	–
ABC							
Mus-eScore							
music21	✓	✓	–	–	–	–	–
partitura							
FlexOHR (library)							
ms3 (library)	–	✓	–	–	–	✓	✓

Nashville numbers, chord-scales and non-harmonic tones are HAMON-only — no source format encodes them natively, so HAMON is what lets them travel. DCML / RomanText express tensions and inversions *on the Roman numeral* rather than as a bare chord symbol, hence the empty *Chord sym.* cell. **Text does not count:** MEI's <harm> and Dezrann's tag accept any string, so a Roman numeral parked there is carried as our text, not encoded — that is why MEI is credited only with figured bass and chord symbols, and Dezrann with nothing structured. The format rows are rendered from the capability model in [./hamonpy/hamonpy/capability.py](#) (the same table behind the ICCCM'26 loss matrix and the site; `test_capability.py` fails if this file drifts from it) — **provisional, to be validated with Johannes / Fabian**. The library rows are hand-kept.

Harmonic analysis (v0.2.0)

Labeling individual chords is only part of the story. HAMON v0.2.0 adds an optional **analytical layer** for a full harmonic analysis: tonal regions and modulations, tonicizations, secondary dominants / dominant-of-the-dominant, layered functional analysis (key / function / degree / bass), note-level melodic analysis of harmonic and non-harmonic tones (passing, neighbor, suspension, appoggiatura, *nota cambiata*, ...), modal harmony (church modes), omitted fundamentals, chord arpeggiation/unfolding, and alternative whole-progression readings. Every analytical field is optional, and older 0.1.0 documents keep working unchanged.

See [Analytical layer \(analysis.md\)](#) for the full specification, and [Berklee-style jazz analysis & the MuRET-hamon pipeline \(berklee-jazz.md\)](#) for the jazz/Berklee encoding and the HAMON-as-interchange workflow (JAZZMUS → HAMON → MuRET-engraving / Dezrann).

Two focused notes cover concerns that cut across everything else:

- **Time-aligned positions (positions.md)** — how measures, beats and audio seconds are encoded (m:1, ts:3, t:5/4, s:12.34, ref:id) in the surface and JSON, why the clocks coexist instead of converting, and why HAMON anchors labels without modelling durations.

- **Systems, the @ hint, and the analytical layer (systems.md)** — why @cs is still correct for a label carrying [of:II]/[scale:...], and when to reach for @auto instead.
-

Datasets & use cases (the harmony hub)

HAMON converts real corpora into the canonical model, but it **does not bundle** third-party datasets. Instead it ships a registry of the original sources (authors, license, citation, URL) plus a small downloader (hamonpy.datasets). You fetch each corpus from its authors, then convert it with a format adapter.

- **Datasets (datasets.md)** — the registry, download policy, and API.
 - **use-cases/** — worked, runnable recipes.
 - **CLI (cli.md)** — hamon convert <file> (any format → canonical JSON) and hamon datasets
-

The HAMON JSON specification

grammar/hamon-schema.json is a **JSON Schema 2020-12** document, and it is the normative hub of the standard. Every other encoding is described as a mapping to and from it.

Structure at a glance

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://hamon.music/schema/hamon-sequence.json",
  "version": "0.1.0",
  "$ref": "#/$defs/HamonSequence",
  "$defs": {
    "HamonSequence": { ... },
    "HarmonyGroup": { ... },
    "HarmonyLabel": { ... },
    "HarmonySemantic": { "oneOf": [ ... ] },
    "ChordSymbolSemantic": { ... },
    "RomanSemantic": { ... },
    "NashvilleSemantic": { ... },
    "FiguredBassSemantic": { ... },
    "FunctionalSemantic": { ... },
    "NoChordSemantic": { ... },
    "TextSemantic": { ... },
    ...
  }
}
```

Key design decisions

Schema feature	Choice	Reason
Meta-schema	JSON Schema 2020-12	Latest stable draft; unevaluatedProperties support

Schema feature	Choice	Reason
Top-level type	HamonSequence via \$ref	Single root object; all definitions are \$defs
Semantic union	HarmonySemantic as oneOf discriminated by "kind"	Maps 1-to-1 to the typed AST union in hamonpy/ast.py
Surface string	Required field on HarmonyLabel	Round-trip fidelity — exact original glyphs always preserved
additionalProperties: false	On every object	Strict schema; implementations catch unknown fields early
Versioning	"version": "0.1.0" top-level field (semver)	MAJOR = breaking, MINOR = new optional fields, PATCH = descriptions

The discriminated union

HarmonySemantic is a oneOf with seven variants. A literal "kind" field identifies which one you have:

kind	Covers	Typical surface
chordSymbol	Chord symbols	CΔ7, Am7, Bb, G7/B
roman	Roman numerals	I, bVII, V7/IV
nashville	Nashville numbers	1, 4m, b7
figuredBass	Figured bass	6-5, 4-3
functional	Functional harmony	T, DD, PD->T
noChord	No-chord indicator	N.C., NC
text	Unrecognized / free text	anything else

Reading the schema

- All reusable types live in \$defs; follow \$ref to navigate between them.
- HarmonyLabel.semantic is the parsed output; HarmonyLabel.surface always holds the original glyph string.
- HarmonyGroup.position (schema 0.2.1, optional) carries a time-aligned position when the label comes from a positioned format — measure+beat (MEI @tstamp), an absolute time fraction, or a ref (MEI @startid).
- RenderingHints (qualityGlyph, seventhGlyph, accidentalGlyphs) record display variants. That is how CΔ7, CM7, and Cmaj7 all normalize to the same semantic yet round-trip back to their original surface.
- The full changelog lives in CHANGELOG.md at the repo root.

The HAMON grammar as the exchange hub

The grammar (grammar/hamon.ebnf) defines five harmony systems plus two structural types:

System	systemId	Examples
Chord symbols	cs	CΔ7, Cm7, Bb, G7(#11)/D
Roman numerals	rn	I, bVII, V7/IV, #iv°7

System	systemId	Examples
Nashville numbers	ns	1, 4m, b7, 5/7
Figured bass	fb	6-5, 4-3, 7-6
Functional	fun	T, S, D, DD
No chord	—	N.C., NC
Text (lossless fallback)	text	any unrecognized label

A HamonSequence can carry an optional @<systemId> hint to disambiguate tokens that could belong to more than one system. For example, 4 reads as Nashville under @ns, but without that context it could just as easily be a figured-bass numeral.

Cross-format mapping summary

The tables below show which HAMON concepts each format represents natively. “□” = a dedicated element or field; “text” = stored as a plain string (parseable but not structured); “—” = not representable.

Notation file formats

HAMON concept	MusicXML	MEI	Humdrum	LilyPond	ABC	MuseScore	MNX
Chord symbol root	<root-step>	@root	**mxhm/**jazz/**irb	pitch name	quoted string	<text>	root.step
Chord quality	<kind>	@quality	spine token suffix	modifier	string suffix	<text>	kind.value
Major 7th glyph variant	kind text=	@label	token as-is	majorSevenSymbol	string as-is	<text>	display
Slash bass	<bass>	@bass	/ in token	/+pitch	/bass	<text>	bass.step
Extension / alteration	<degree>	@extens	token suffix	modifier	string suffix	<text>	TBD
Roman numeral	text	<harm> text	**harm token	comment	comment	<text>	roman-numeral
Figured bass	text	<fb><f>	**fb / **harm	comment	comment	<text>	□
Nashville number	text	text	**harm token	comment	comment	<text>	□
Functional harmony	□	text	**functioncomment token	comment	comment	<text>	□
No chord	text	text	. (null)	—	—	<text>	TBD
Time position	<offset>	@tstamp	spine row order	note position	note position	element order	position

Third-party annotation formats

HAMON concept	DCML TSV	Harte	iReal Pro	music21
Chord symbol root	—	Root (e.g. C, Ab)	root note	ChordSymbol.root
Chord quality	form column	Shorthand (e.g. maj, min)	quality suffix	ChordSymbol.chordKind
Seventh	figbass column	shorthand (e.g. maj7, 7)	quality suffix	ChordSymbol.chordKind
Suspension	—	shorthand (sus4, sus2)	sus/sus4/sus2	ChordSymbol.chordKind
Roman numeral	numeral+form+figbass	/degree (numeric)	□	RomanNumeral
Slash bass	—	/degree (numeric)	/Bass	ChordSymbol.bass
Prefix accidentals	b/# prefix on numeral	—	—	frontAlterationString
Secondary target	relativeroot	□	□	secondaryRomanNumeral
No chord	@ rows (skipped)	N	n/N	NoChord
Time position	mc/onset columns	timestamp columns	bar structure	offset attribute

Reading order

Jump straight to the format you are working with.

Notation file formats: - **MusicXML** → [musicxml.md](#) - **MEI** → [mei.md](#) - **Humdrum **kern** → [humdrum.md](#) - **LilyPond** → [lilypond.md](#) - **ABC notation** → [abc.md](#) - **MuseScore MSCX** → [musescore.md](#) - **MNX (planned)** → [mnx.md](#)

Third-party annotation and audio formats: - **DCML** (Hentschel et al. Roman numeral TSV corpora; plain + time-aligned *expanded* tables with cadences/phrases) → [dcml.md](#) - **Harte** (MIREX / audio chord recognition) → [harte.md](#) - **iReal Pro** (chord charts) → [ireal.md](#) - **Dezrann** (Algomus web platform; .dez time-aligned label JSON) → [dezrann.md](#) - **JAMS** (JSON Annotated Music Specification; MIR annotation container) → [jams.md](#) - **music21** (Python toolkit integration; native Stream↔HAMON objects + .hamon I/O format) → [music21.md](#) - **ms3** (Johannes Hentschel's MuseScore parser → DCML expanded tables) → [ms3.md](#) - **DiLeMMa pitch arrays** (the note-level training tables behind AnalysisGNN — DLC + AugmentedNet in one shape) → [dcml.md](#) - **partitura** (CP-JKU symbolic music processing; Part↔HAMON harmony objects) → [partitura.md](#) - **DiMCAT** (DCML's corpus-analysis library — <https://dimcat.readthedocs.io>; consumes the DCML expanded tables that HAMON reads/writes via [dcml.md](#), so it is an interop *target* rather than a format adapter)

The HAMON grammar itself is documented in [grammar/hamon.ebnf](#) and [grammar/README.md](#).

For the regex-based extractors used for LilyPond, ABC, and MuseScore, see [regex.md](#).

Architecture and internals: - **Architecture / UML diagrams** → [architecture.md](#) - **Grammar internals (ANTLR/EBNF)** → [grammar-internals.md](#) - **Railroad diagrams** → [railroad/index.html](#)

Tutorial: encoding harmony with HAMON

HAMON gives you **two serializations of one model**:

- **HAMON surface** — a compact, human-writable text line (Cmaj7,m:1,ts:1,cs:Dm7,rn:ii7). This is the format you *type*.
- **HAMON JSON** — the canonical, typed structure (grammar/hamon-schema.json). This is the **hub** every other format maps to and from.

Parsing turns surface into JSON; serializing turns JSON back into surface. The two round-trip cleanly: `parse(serialize(x)) == x`. This tutorial starts with a single chord and works up to a positioned, analysed progression, showing both forms side by side.

A note on the engraved staves throughout: they are **illustrative only**. HAMON encodes harmony *labels*, not notes. The label is the source of truth; the score is just there to help you read it.

1. Your first label — a single chord

The smallest HAMON document is a single label. Here it is as surface:

```
@cs  
Cmaj7
```

@cs is an optional **system hint** — it says “this sequence is chord symbols” (see [systems.md](#)). The label Cmaj7 is a C major seventh chord.

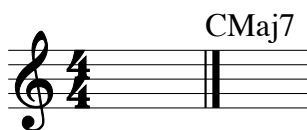


Figure 1: A single Cmaj7 chord symbol on a staff

Now the same label as **HAMON JSON**. The exact glyphs are kept in surface, and the parsed meaning goes in semantic:

```
{  
  "sequenceSystemHint": "cs",  
  "groups": [  
    { "primary": [ {
```

```

    "surface": "Cmaj7",
    "semantic": { "kind": "chordSymbol", "root": { "note": "C" }, "quality":
      ↪ "major", "seventh": "maj7" },
    "detectedSystem": "cs", "system": "cs"
  } ] }
]
}

```

Two ideas show up right away:

- **surface vs semantic.** surface is lossless ("Cmaj7"); semantic is the normalized reading. CM7, CΔ7, and Cmaj7 all share one semantic but each keeps its own surface.
- **Groups.** A sequence is a list of **groups**, one per time point. Each group holds one or more primary labels — more than one when several analytical readings coexist (see §4).

2. The surface at a glance

A few common chord-symbol glyphs (all optional spellings normalize to the same semantic):

Surface	Meaning
C	C major triad
Cm / Cmin / C-	C minor
C7	C dominant seventh
Cmaj7 / CM7 / CΔ7	C major seventh
Cm7b5 / Cø7	C half-diminished seventh
Cdim7 / C°7	C diminished seventh
C/E	C major with E in the bass
NC / N.C.	no chord

Other systems use their own spellings: Roman numerals (V7, ii°6), Nashville (1, 5-4), figured bass (6, 6-5), functional (T, D, S). See [systems.md](#).

3. A progression, in time — measures, beats, meter

Real music sits at a point in time. Since v0.4 a group can start with **position items** (written first, comma-separated), and the sequence can declare its **meter**:

```

@cs
@meter:4/4
m:1,ts:1,Dm7
m:2,ts:1,G7
m:3,ts:1,Cmaj7

```

- **@meter:4/4** — the time signature. Its numerator is the beat count, so it bounds the valid ts range to [1, 5) (see [positions.md](#)).
- **m:1,ts:1** — measure 1, beat 1. ts: follows MEI [data.BEAT](#) (1-based, decimals allowed).

In JSON, each group gets its own position and the meter lives in a top-level array:

```

{
  "sequenceSystemHint": "cs",
  "meters": [ { "numerator": 4, "denominator": 4, "fromGroup": 0 } ],

```

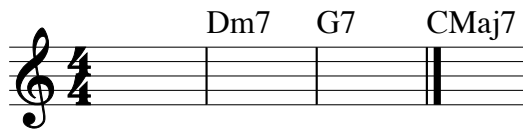


Figure 2: A ii-V-I chord chart: Dm7, G7, CMaj7 in 4/4

```
"groups": [
  { "primary": [ { "surface": "Dm7", "semantic": { "kind": "chordSymbol",
    ↪ "root": { "note": "D" }, "quality": "minor", "seventh": "min7" },
    ↪ "detectedSystem": "cs" } ],
    "position": { "measure": 1, "beat": 1.0 } },
  { "primary": [ { "surface": "G7", "semantic": { "kind": "chordSymbol",
    ↪ "root": { "note": "G" }, "quality": "major", "seventh": "dom7" },
    ↪ "detectedSystem": "cs" } ],
    "position": { "measure": 2, "beat": 1.0 } },
  { "primary": [ { "surface": "Cmaj7", "semantic": { "kind": "chordSymbol",
    ↪ "root": { "note": "C" }, "quality": "major", "seventh": "maj7" },
    ↪ "detectedSystem": "cs" } ],
    "position": { "measure": 3, "beat": 1.0 } }
]
```

A ts that falls outside its meter (say ts:5 in 4/4) is a **non-blocking warning**, never an error. HAMON has to round-trip imperfect sources, so it never rejects them.

A position can also say **when in seconds** (s:, since v0.5), which is what an audio annotation states, and it can say both at once:

```
m:1,ts:1,s:0,Dm7
m:2,ts:1,s:2.4,G7
```

The clocks sit side by side and neither is computed from the other — see [positions.md](#).

4. Layered analysis — a chord *and* its Roman numeral

One sonority can carry several **analytical layers** in a single group, joined by commas with short tags (cs: chord symbol, rn: Roman, fb: figured bass, fn: functional). A @key: directive opens a tonal region, which is what gives the Roman numerals something to mean:

```
@meter:4/4
@key:C
m:1,ts:1,cs:Dm7,rn:ii7
m:2,ts:1,cs:G7,rn:V7
m:3,ts:1,cs:Cmaj7,rn:I
```

Each group's primary now carries **two** labels — the chord symbol and the Roman numeral — and each one records its own layer:

```
{ "primary": [
  { "surface": "Dm7", "layer": "chord", "semantic": { "kind": "chordSymbol",
    ↪ "root": { "note": "D" }, "quality": "minor", "seventh": "min7" } },
```

```

    { "surface": "ii7", "layer": "degree", "semantic": { "kind": "roman",
      ↪ "degree": "ii", "tail": "7" } }
  ],
  "position": { "measure": 1, "beat": 1.0 } }

```

The @key:C region is recorded just once, at the top level, in regions (see [analysis.md](#) for tonicizations, modulations, chord-scales, and non-harmonic tones).

5. Figured bass

Figured bass is another system. @fb hints it at the sequence level; fb: tags it inside a layered group. Figures use - to stack intervals — 6-5 means a 6 moving to a 5:

```

@fb
@meter:4/4
m:1,ts:1,fb:6-5
m:1,ts:3,fb:6
{ "primary": [ { "surface": "6-5", "semantic": { "kind": "figuredBass",
  ↪ "number": 6, "tail": "-5" }, "detectedSystem": "fb" } ],
  "position": { "measure": 1, "beat": 1.0 } }

```

(A real figured-bass example — Mozart, K. 282/i — ships in publications/ICCCM26/examples/sat_mozart_fb

6. The JSON structure, in one view

```

HamonSequence
├ version?          "0.4.0"
├ sequenceSystemHint? "cs" | "rn" | "ns" | "fb" | "fun" | "auto"
├ meters?           [ { numerator, denominator, fromGroup } ] ← time
  ↪ signatures (§3)
├ regions?          [ { key, kind, fromGroup, toGroup?, scales? } ] ←
  ↪ @key: analysis (§4)
├ groups            [ HarmonyGroup ]
  └ HarmonyGroup
    ├── primary      [ HarmonyLabel ] ← one per layer (§4)
    ├── alternatives? [ [ HarmonyLabel ] ] ← competing whole readings
    └ position?      { measure?, beat?, time?, seconds?, ref? } ← where
      ↪ it starts (§3)
      └ HarmonyLabel
        ├── surface    "Dm7" ← exact glyphs (lossless)
        ├── layer?     "chord" | "degree" | "bass" | "function" | ...
        ├── semantic    { kind: "chordSymbol" | "roman" | "nashville" |
          ↪ "figuredBass" | "functional" | "noChord" | "text", ... }
        └ system        detected notation system

```

grammar/hamon-schema.json is the normative JSON Schema (2020-12). Everything above is optional except groups and each group's primary.

7. Round-tripping in code

Python (hamonpy):

```
from hamonpy.parse import parse_hamon_sequence
from hamonpy.serialize import sequence_to_json, sequence_to_hamon_text

seq = parse_hamon_sequence("@cs\n@meter:4/4\nm:1,ts:1,Dm7\nm:2,ts:1,G7\n┘
    ↪ m:3,ts:1,Cmaj7")
print(sequence_to_json(seq))      # → canonical JSON
print(sequence_to_hamon_text(seq)) # → back to surface (round-trips)
```

Where next

- [positions.md](#) — measures, beats, @meter, and absolute/reference anchors.
- [analysis.md](#) — the full analytical layer (regions, tonicizations, chord-scales, non-harmonic tones, alternative readings).
- [systems.md](#) — the @ system hint vs the per-label detected system vs the layer tag.
- [index.md](#) — every source/annotation format HAMON maps to and from.

Processing harmony, format by format (CLI & Python)

A practical cookbook: for each thing you'd want to do with a harmony file — read it, re-emit it in another format, measure what a conversion loses, or QA a corpus — here is the hamon command and the equivalent hamonpy Python.

Every snippet uses the ready-to-run inputs in [../examples/](#). Install first (`pip install ./hamonpy`, or `-e ./hamonpy[dev]` for development); that puts the hamon command on your path.

The format *semantics* — what each encoding can carry and where it loses information — live in the per-format pages under [index.md](#) (MEI, MusicXML, Humdrum, Harte, DCML, ...). This page is only about *running* the conversions.

What you can read and write

Read (import). The format is auto-detected from the file extension, with a content sniff as a fallback. Override it with `--format` (CLI) or the `fmt` argument (Python).

Extension	Default format
.hamon	hamon (the surface syntax)
.mei	mei
.xml / .mxl	musicxml (or mei by content)
.krn	humdrum (**text/KEY=>: → key_modulation)
.tsv	dcml (quarterbeats → dcml_expanded; note-level pitch arrays → dilemma)
.rntxt	romantext
.lab	harte
.abc	abc
.ly	lilypond
.mscx / .mscz	musescore
.json	treebank, kp, rock_corpus, bps_fh, dezzrann, jams (by content)

Full `--format` list: abc, bps_fh, choro, dcml, dcml_expanded, dezzrann, dilemma, hamon, harm, harte, humdrum, ireal, jams, key_modulation, kp, lilypond, mei, musescore, musicxml, rock_corpus, romantext, treebank.

Write (export). Any of these `--to` targets: abc, dcml, dezzrann, hamon, harte, humdrum, ireal, jams, lilypond, mei, musescore, musicxml, romantext. (Research corpora — treebank, kp, rock_corpus, bps_fh, dilemma — are import-only.)

1. Read any format → canonical HAMON JSON

CLI

```
hamon convert examples/song.mei           # auto-detect, JSON to stdout
hamon convert examples/changes.tsv -o out.json # write to a file
hamon convert chart.txt --format ireal      # force the input format
```

Python

```
from pathlib import Path
from hamonpy.cli import convert_file, convert_text
from hamonpy.serialize import sequence_to_json, sequence_to_hamon_text

seq = convert_file(Path("examples/song.mei"))           # auto-detect
seq = convert_file(Path("chart.txt"), "ireal")          # force the format
seq = convert_text("@cs\nCmaj7\nAm7\nDm7\nG7", "hamon") # from a string (fmt
↳ required)
```

```
print(sequence_to_json(seq))          # the canonical JSON
print(sequence_to_hamon_text(seq))    # back to the surface (round-trips)
```

convert_file/convert_text return a HamonSequence — the in-memory canonical model that every other operation below takes as input.

2. Export → another format (any → any)

export reads **any** supported input and writes **any** target format. There is no fixed source/target pairing: the input is parsed into the canonical HAMON model and the target is generated from it, so **X → HAMON → Y** works for every X you can read and every Y you can write. Harte → LilyPond, DCML → RomanText, MEI → Humdrum — all the same command.

CLI

```
hamon export examples/song.mei --to harte           # MEI <harm> → Harte labels
hamon export examples/changes.tsv --to romantext    # DCML TSV → RomanText
hamon export examples/song.mei --to lilypond -o out.ly
```

The HAMON JSON is always the intermediate. To keep it, and to see what the target couldn't carry, in the same run:

```
hamon export examples/song.mei --to harte --save-hamon mid.json --report
```

--save-hamon writes the intermediate canonical JSON; --report prints the semantic loss to stderr. If the target can't represent the source's harmony system at all (e.g. Roman numerals → Harte), the output is empty and a note explains why.

Python — transcode does the whole X → HAMON → Y in one call, always producing the intermediate JSON and measuring the loss:

```
from hamonpy.cli import transcode
```

```
tc = transcode("examples/song.mei", "harte") # any input path → any WRITERS
↳ target
print(tc.output)                             # the target-format text
print(tc.hamon_json)                         # the intermediate canonical HAMON JSON (always
↳ produced)
tc.source_format                             # 'mei' (what was detected)
semantic = [f for f in tc.report.findings if not f.notational] # what Harte
↳ dropped
```

Or compose the primitives yourself — `write_to(convert_file(path), target):`

```
from pathlib import Path
from hamonpy.cli import convert_file
from hamonpy.report import write_to, WRITERS

text = write_to(convert_file(Path("examples/song.mei")), "harte") # target ∈
↳ WRITERS
Path("out.ly").write_text(write_to(convert_file(Path("examples/changes.tsv")),
↳ "lilypond"),
                        encoding="utf-8")
```

Because no two formats carry the same information, a conversion can drop what the target can't represent — `transcode/- -report` (and `report, next`) show exactly what.

3. Report the loss of a conversion

Runs the export and diffs the result back against the source, splitting **semantic loss** (dropped/changed meaning) from **notational re-spelling** (glyphs that differ but mean the same).

CLI

```
hamon report examples/song.mei --to harte # human-readable
↳ summary
hamon report examples/song.mei --to harte --json # structured
hamon report examples/song.mei --to mei --write-output out.mei
```

Python

```
from hamonpy.report import lossy_report

rep = lossy_report(seq, "harte")
semantic = [f for f in rep.findings if not f.notational] # real loss
respelling = [f for f in rep.findings if f.notational] # same meaning,
↳ other glyphs
lossless = not semantic

print(f"{rep.target}: {len(semantic)} lost, {len(respelling)} respelled")
print(rep.output_text) # the exported text
for f in semantic:
    print(f" [{f.kind}] {f.path}: {f.summary}") # kind ∈ {dropped, changed,
↳ added}
```

For `examples/song.mei --to harte` this reports 7 semantic losses (the beat positions, and the b5 of Am7b5 that Harte flattens to A:min7) and 16 re-spellings.

4. Validate a corpus

Flags **opaque labels** (a surface the parser couldn't read as harmony, so its kind stayed text) and **position warnings** (a ts: beat outside the @meter). With `--strict` it exits non-zero when anything is flagged — a CI gate over a corpus.

CLI

```
hamon validate examples/corpus.hamon --strict
hamon validate corpus/*.hamon --json # many files at once
```

Python

```
from hamonpy.cli import convert_file
from hamonpy.validate import validate_positions

seq = convert_file(Path("examples/corpus.hamon"))
position_warnings = validate_positions(seq)
opaque = [lab.surface
           for g in seq.groups for lab in g.primary
           if lab.semantic.kind == "text"]
```

Format-specific behaviour

The command is the same for every format; what changes is *what survives the trip*. Each per-format page documents its mapping, its native capability, and its losses:

[MEI](#) · [MusicXML](#) · [Humdrum](#) · [LilyPond](#) · [ABC](#) · [MuseScore](#) · [Harte](#) · [DCML](#) · [RomanText](#) · [Dezrann](#) · [iReal Pro](#) · [JAMS](#)

Python API, at a glance

Function	Import from	Does
<code>convert_file(path, fmt=None)</code>	<code>hamonpy.cli</code>	file → HamonSequence (auto-detect unless fmt)
<code>convert_text(text, fmt)</code>	<code>hamonpy.cli</code>	string → HamonSequence (fmt required)
<code>transcode(path, to, input_format=None)</code>	<code>hamonpy.cli</code>	any → any → Transcode (.output, .hamon_json, .report)
<code>detect_file_format(path)</code>	<code>hamonpy.cli</code>	the format that auto-detection would pick
<code>sequence_to_json(seq)</code>	<code>hamonpy.serialize</code>	HamonSequence → canonical JSON
<code>sequence_to_hamon_text(seq)</code>	<code>hamonpy.serialize</code>	HamonSequence → surface string
<code>write_to(seq, target)</code>	<code>hamonpy.report</code>	HamonSequence → text in a WRITERS format
<code>lossy_report(seq, target)</code>	<code>hamonpy.report</code>	export + diff → LossyReport
<code>validate_positions(seq)</code>	<code>hamonpy.validate</code>	list of out-of-meter position warnings

See [cli.md](#) for the full command reference.

Using HAMON in your editor / environment

HAMON is a normal Python package (hamonpy) plus a hamon command. Nothing here is special to one editor — once hamonpy is installed in the interpreter you've selected, everything works. This page shows how to get it there and wire it into the common setups.

Get HAMON into your environment

The Python package is the hamonpy/ **subdirectory** of the repo (the repo root is not a package). Once it's published to PyPI (after ICCCM'26) using it is just `pip install hamonpy` into your project's environment. **Until then — and always if you want to edit HAMON — you install from a checkout**, as follows.

First get the code:

```
git clone https://github.com/OMR-PRAIG-UA-ES/hamon.git    # or download the
↳ ZIP from GitHub
```

Then install hamonpy from that checkout **into whichever environment you want** — a dedicated one, *or your own project's interpreter*. Because you point at the checkout by path, this works from any project, not just from inside the HAMON repo:

```
# in your project's env (conda / venv / poetry – whatever your project uses):
pip install -e /path/to/hamon/hamonpy                # editable install
```

```
# optional extras (comma-separated) – install only the ones you need:
# dev          pytest (run the test suite)
# demo         streamlit (the interactive demo)
# music21      music21 (score parsing / analysis / rendering)
# ms3          ms3 (DCML MuseScore corpora)
# partitura   partitura (score ingestion)
# flexohr     FlexOHR (OHR object model)
# bps         pandas + openpyxl (BPS-FH corpus)
# notebooks   jupyter + nbconvert + ipykernel (run the tours as notebooks)
# e.g.:
pip install -e "/path/to/hamon/hamonpy[music21,ms3,partitura,notebooks]"
```

That single step gives *your* project both the library (`import hamonpy...`) and the hamon CLI — you do **not** have to open the HAMON repo as your project. (Use `-e` if you might also edit HAMON; drop it for a plain install.) The [processing cookbook](#) has the read / export / report / validate recipes; the rest of this page is only about wiring that install into each tool.

PyCharm / IntelliJ IDEA

Work in **your own project** — no need to open the HAMON repo:

1. **Select your project's interpreter.** *Settings* → *Project* → *Python Interpreter* (an existing venv/conda env, or add one). IntelliJ users need the Python plugin first.
2. **Install hamonpy into it.** Open the built-in Terminal (it activates the selected interpreter) and `pip install -e /path/to/hamon/hamonpy`. Now `import hamonpy` resolves in your own code.
3. **Run the CLI** from that Terminal: `hamon convert your-score.mei`, or make a *Run Configuration* with module `hamonpy` and parameters `export your-score.mei --to harte --report`.
4. **Debug** by setting a breakpoint and running with the same interpreter; with the `-e` install you can step straight into `hamonpy`.

Hacking on HAMON *itself*? Then open the `hamon/` repo as the project instead and mark `hamonpy/` as a *Sources Root*; it ships an `.idea/` with `run/inspection` settings, and the CLI examples (`examples/song.mei`) resolve from the repo root.

Visual Studio Code

Open **your own project folder**:

1. Install the **Python** extension.
2. **Select the interpreter:** Command Palette → *Python: Select Interpreter* → your project's env.
3. In the integrated terminal, install HAMON into it once: `pip install -e /path/to/hamon/hamonpy`. `hamon` is then on the path and `import hamonpy` works: `hamon report your-score.mei --to harte`.
4. For notebooks, also install the **Jupyter** extension; it reuses the same interpreter, so `import hamonpy` works in `.ipynb` cells with no extra setup.
5. launch.json example to debug the CLI:

```
{ "type": "python", "request": "launch", "module": "hamonpy",  
  "args": ["export", "your-score.mei", "--to", "harte", "--report"] }
```

Jupyter notebooks

Install HAMON into the env you run Jupyter from, register it as a kernel once, then select it in the notebook:

```
pip install -e "/path/to/hamon/hamonpy" ipykernel  
python -m ipykernel install --user --name hamonpy --display-name "Python  
↪ (hamonpy)"
```

In a cell:

```
from pathlib import Path  
from hamonpy.cli import convert_file, transcode  
from hamonpy.serialize import sequence_to_json
```

```
seq = convert_file(Path("your-score.mei"))  
tc = transcode("your-score.mei", "harte")  
↪ report  
tc.report
```

```
# a file in your own project  
# any → any, with the loss  
  
# what Harte dropped
```

To pretty-print the canonical JSON inline:

```
import json
json.loads(sequence_to_json(seq))    # Jupyter renders the dict as a foldable
↳ tree
```

The guided tours under [use-cases/notebooks/](#) (in the HAMON checkout) are [jupyter](#) .py files — install the extra (`pip install -e "/path/to/hamon/hamonpy[notebooks]"`) and open them as notebooks (*right-click* → *Open as Notebook*, or `jupyter --to notebook <tour>.py`).

Digital-musicology environments

HAMON is the exchange layer *between* these tools — it reads their harmony and writes it back out. Install the matching extra (on your checkout path, e.g. `pip install -e "/path/to/hamon/hamonpy[music21]"`) and use `transcode / convert_file`:

Tool / ecosystem	How HAMON connects	Extra / docs
music21	import a music21-parsed score's harmony, or realise HAMON for display	[music21] · music21.md
DCML / ms3 (MuseScore corpora)	read harmonies/*.tsv, round-trip RomanText/DCML	[ms3] · dcml.md , ms3.md
partitura	ingest harmony from partitura-parsed scores	[partitura] · partitura.md
Humdrum / **kern	read/write **harm · **fb · **function spines	humdrum.md
MuseScore	read .mscx/.mscz chord symbols; export MSCX	musescore.md
MEI / Verovio	read <harm>/<fb>; export HA-MEI for engraving	mei.md , verovio.md
Harte / iReal / Dezzrann / JAMS	MIR annotation formats, import and export	harte.md , ireal.md , dezzrann.md , jams.md

Example — pull harmony out of a DCML corpus and hand it to a Humdrum toolchain:

```
hamon export harmonies.tsv --to humdrum -o harmonies.krn --report
```

Everything is one model underneath, so any of these can be converted to any other; the [processing cookbook](#) covers the full matrix.

MusicXML — Harmony Encoding

Overview

[MusicXML](#) is the XML interchange format for Western common-practice notation, maintained by the W3C Music Notation Community Group. It's how notation editors — Finale, Sibelius, MuseScore, Dorico — hand scores to one another, and it's a staple of music information retrieval.

Harmony rides on the `<harmony>` element, which sits as a sibling of notes inside a `<measure>`. MusicXML takes a **semantic** approach: the chord quality is a controlled-vocabulary keyword (major-seventh, say), and the display text goes separately in a `text` attribute. HAMON reads both, but prefers text when it's there, because that's the exact surface the composer or editor meant.

The `<harmony>` element

A `<harmony>` element holds child elements that together spell out one chord symbol:

```
<harmony print-frame="no">
  <root>
    <root-step>C</root-step>
    <root-alter>-1</root-alter>    <!-- optional: -1=flat, 1=sharp,
    ↪ 2=double-sharp -->
  </root>
  <kind text="m7">minor-seventh</kind>
  <bass>                                <!-- optional: slash bass -->
    <bass-step>E</bass-step>
    <bass-alter>-1</bass-alter>
  </bass>
</harmony>
```

`<root-step>` and `<root-alter>`

The root's pitch class: A-G. Accidentals are numeric semitone alterations — -1 = flat, 1 = sharp, -2 = double-flat, 2 = double-sharp. Natural is 0 or simply omitted.

`<kind>`

The chord quality. The inner content is the **semantic keyword**; the optional `text` attribute holds the **display string**.

Inner text (semantic)	Common text attribute	HAMON surface
major	(empty or "")	C
minor	m	Cm
augmented	+ or aug	C+
diminished	° or dim	C°
dominant	7	G7
major-seventh	maj7, M7, Δ7	Cmaj7 / CM7 / CΔ7
minor-seventh	m7	Am7
diminished-seventh	°7 or dim7	B°7
half-diminished-seventh	ø7 or m7b5	Bø7
suspended-second	sus2	Csus2
suspended-fourth	sus4	Csus4

When text is absent, HAMON reconstructs a surface string from the semantic keyword using the normalization table above.

<bass>

Optional. Encodes a slash bass note (C/E, for example). Its structure mirrors <root>: <bass-step> (A-G) plus an optional <bass-alter>.

<degree>

Optional. Encodes alterations or additions beyond the base quality (#11, b9). Each <degree> carries <degree-value>, <degree-alter>, and <degree-type> (add, alter, subtract). HAMON captures these as chordSymbol extension/alteration tokens when they also show up in the text attribute; a structural <degree> with no matching text attribute isn't mapped yet.

Mapping: JSON fields → MusicXML

All mappings are relative to the canonical JSON schema (grammar/hamon-schema.json).

Chord symbols (kind: "chordSymbol")

JSON field	HAMON → MusicXML	MusicXML → HAMON	Loss / note
root.note	<root-step>C</root-step>	read <root-step>	none
root.accidental: "sharp"	<root-alter>1</root-alter>	alter=1 → sharp	numeric; enharmonic preserved
root.accidental: "flat"	<root-alter>-1</root-alter>	alter=-1 → flat	none
root.accidental: "double-sharp"	<root-alter>2</root-alter>	alter=2 → double-sharp	none
root.accidental: "double-flat"	<root-alter>-2</root-alter>	alter=-2 → double-flat	none
quality: "major"	<kind>major</kind>	inner text major	none

JSON field	HAMON → MusicXML	MusicXML → HAMON	Loss / note
quality: "minor"	<kind text="m">minor</kind>	inner text minor	text attr preferred for surface
quality: "augmented"	<kind>augmented</kind>	inner text augmented	none
quality: "diminished"	<kind>diminished</kind>	inner text diminished	none
quality: "half-diminished"	<kind>half-diminished-seventh</kind>	inner text half-diminished...	none
seventh: "dom7"	<kind text="7">dominant</kind>	inner text dominant	none
seventh: "maj7"	<kind text="maj7">major-seventh</kind>	inner text major-seventh	display glyph (Δ7, M7, maj7) in text attr; preserved in rendering.seventhGlyph
seventh: "min7"	<kind text="m7">minor-seventh</kind>	inner text minor-seventh	none
seventh: "dim7"	<kind>diminished-seventh</kind>	inner text diminished-seventh	none
seventh: "hdim7"	<kind>half-diminished-seventh</kind>	inner text half-diminished...	none
suspensions: [4]	<kind text="sus4">suspended-fourth</kind>	inner text suspended-fourth	none
suspensions: [2]	<kind text="sus2">suspended-second</kind>	inner text suspended-second	none
bass.note	<bass><bass-step>E</bass-step></bass>	read <bass-step>	none
bass.accidental	<bass-alter>-1</bass-alter>	same as root alter	none
extensions[] / alterations[]	<degree> children	read <degree> children	only extracted when reflected in kind text; structured <degree> without text is currently ignored
rendering.seventhGlyph	<kind text="Δ7"> attribute	kind text attr → rendering.seventhGlyph	none
rendering.qualityGlyph	absorbed into kind text	kind text → rendering.qualityGlyph	none

Other notation systems

MusicXML's <harmony> only does chord symbols. There's no dedicated MusicXML element for Roman numerals, figured bass, Nashville numbers, or functional labels.

JSON kind	HAMON → MusicXML	MusicXML → HAMON
roman	<kind text="V7"/> (non-standard; surface in text)	text attr → parsed as surface
figuredBass	<kind text="6-5"/> (non-standard)	text attr → parsed as surface
nashville	<kind text="4m"/> (non-standard)	text attr → parsed as surface
functional	not representable	—
noChord	<kind text="N.C.">none</kind>	text or inner none → noChord
text	<kind text="..."/>	text attr → TextSemantic.text

Time-aligned positions (schema 0.2.1)

hamonpy.adapters.musicxml.musicxml_to_hamon(text) builds a positioned HamonSequence: each <harmony> becomes a HarmonyGroup that carries its position ({ measure, beat }). HAMON works the beat out from the MusicXML time model:

- <divisions> sets divisions-per-quarter, and it persists across measures.
- Elapsed time builds up within a measure from <note> durations (a <chord> note doesn't advance time), <forward>, and <backup>.
- A <harmony><offset> (in divisions) shifts the chord off the current position.
- beat = (elapsed + offset) / divisions + 1 (1-based).

The hamon convert CLI uses this positioned adapter for MusicXML; the flat formats ._extract_musicxml (surface strings only) is still there when you don't need positions.

Losses and ambiguities

- **text vs. semantic divergence:** When text="Δ7" but the inner content is major-seventh, HAMON goes with text and produces CΔ7. A tool that reads only the semantic keyword and ignores text will display something else.
- **<degree> elements:** Extension and alteration elements (#11, for instance) don't make it into HAMON's alteration tokens unless they also appear in the text attribute.
- **Display-only text:** Some exporters — older Finale versions among them — write arbitrary strings in text that don't parse cleanly. HAMON falls back to textLabel for those.
- **Time anchoring:** <harmony> elements have an <offset> child and are positioned relative to the enclosing <measure>. HAMON currently extracts a flat label list and keeps no beat position.

How HAMON reads MusicXML

hamonpy reads MusicXML with a pure-Python, text-level extractor (hamonpy/hamonpy/adapters/formats.py, _extract_musicxml):

1. Finds every <harmony>...</harmony> block by regex.

2. Reads <root-step> (required — a block with no root is skipped) and the optional numeric <root-alter>, converting the alteration to an ASCII accidental (1→#, -1→b, 2→x, -2→bb).
3. Reads the chord quality from the <kind> text attribute first, then the inner <kind> text, and normalizes it through a keyword map (major→"", minor→m, dominant-seventh→7, major-seventh→maj7, half-diminished-seventh→ø7, and so on).
4. Reads the optional <bass-step>/<bass-alter> and appends it as a slash bass (/E).
5. Assembles {root}{accidental}{quality}{bass} into one surface string per block, then deduplicates.

For position-aware reading — each <harmony> becomes a HarmonyGroup carrying its { measure, beat } — use the hamonpy.adapters.musicxml.musicxml_to_hamon(text) adapter described above; _extract_musicxml gives you the flat surface list.

Examples from the HAMON fixture corpus

Fixture	<kind> inner	<kind text>	HAMON surface	kind
C_major	major	—	C	chordSymbol
Bb_major	major	—	Bb	chordSymbol
C_minor	minor	m	Cm	chordSymbol
C_delta7	major-seventh	Δ7	CΔ7	chordSymbol
C_M7	major-seventh	M7	CM7	chordSymbol
C_maj7	major-seventh	maj7	Cmaj7	chordSymbol
G_dom7	dominant	7	G7	chordSymbol
B_hdim7	half-diminished-seventh	—	Bø7	chordSymbol
C_sus4	suspended-fourth	sus4	Csus4	chordSymbol

Full fixture files: fixtures/chords/.

MEI — Harmony Encoding

Overview

[MEI \(Music Encoding Initiative\)](#) is the XML format the scholarly music-encoding world reaches for. It's an established fixture in digital musicology, and it's the format HAMON's MEI customization and Verovio integration target first.

MEI puts harmony in the `MEI.harmony` module, which gives you:

- `<harm>` — the main element for any written harmony label (chord symbols, Roman numerals, figured bass, Nashville, functional)
- `<fb> + <f>` — structured figured bass notation
- `<chordDef> / <chordTable>` — chord definition library (less commonly used for exchange)

What MEI carries natively, and what it only holds as text

`<harm>` accepts arbitrary character data. That makes it a convenient home for a HAMON label — and it is why, until 2026-09-03, the loss matrix reported MEI as **lossless for chord-scales MEI cannot express**: the exporter wrote `<harm>Cmaj7[scale:ionian]``</harm>` and the importer read it straight back. The round-trip was measuring our own string.

HAMON now counts that as a **workaround**, not a capability:

Aspect	In MEI	Counted as
chord symbols	<code><harm> + <chordDef>/<chordMember></code>	native
figured bass	<code><harm> + <fb>/<f></code>	native
Roman numerals	<code><harm></code> character data	text — <i>loss</i>
applied / secondary	<code><harm></code> character data	text — <i>loss</i>
key / tonal region	<code><harm></code> character data	text — <i>loss</i>

The distinction is not pedantry: an analysis parked in `<harm>` text is unreadable to any MEI tool that is not HAMON, which is precisely what an interchange standard must not depend on. Shipping such a file is fine — carrying the surface out of band is a real, useful thing to do — but it is not MEI encoding the analysis.

This is the gap the planned MEI proposal closes (see `STATUS.md` → “New MEI harmony encoding”): give MEI a semantic vocabulary for Roman numerals, applied chords and tonal regions, rather than leaning on `<harm>` text forever. `hamonpy/capability.py` is what will score the proposal — each aspect that becomes structural moves from `TEXT_CAPABILITY` to `SEMANTIC_CAPABILITY`, and MEI's column in the loss matrix drops.

The <harm> element

<harm> is where harmony information lives. It sits inside <measure> (or <section>) as a sibling of the <staff> elements, anchored either to a time position or to a specific note.

Minimal form

At its simplest, a <harm> just holds a text label:

```
<harm>CΔ7</harm>
<harm>V7</harm>
<harm>6-5</harm>
```

This is the form the HAMON fixture snippets use, and HAMON's grammar parses the text content directly.

Time anchoring

You can anchor a <harm> two ways.

By timestamp (@tstamp) — beats counted from the start of the measure, 1-based:

```
<harm tstamp="1" place="above">CΔ7</harm>
<harm tstamp="3" place="above">G7</harm>
```

By note reference (@startid) — pointing at a note's @xml:id:

```
<note xml:id="n1" pname="c" oct="4" dur="4"/>
<harm startid="#n1" place="above">CΔ7</harm>
```

Reach for @startid when you want analytical precision; @tstamp travels better when note IDs aren't stable.

The type attribute (HAMON convention)

In the HAMON fixture corpus, chord symbols carry type="chordSymbol" so the notation system is stated outright:

```
<harm type="chordSymbol">CΔ7</harm>
<harm type="chordSymbol">Am7</harm>
```

Other labels — Roman numerals, Nashville, figured bass in text form — leave type off, and HAMON infers the system from the grammar.

The HAMON MEI customization

The HAMON ODD (mei-customization/hamon.odd) adds structured semantic attributes to the standard <harm> element — the same information MusicXML puts in <root>, <kind>, and <bass>:

```
<harm startid="#n1"
      root.pname="c"
      root.accid="s"
      bass.pname="g"
      chord.kind="major-seventh">
  C#maj7/G
</harm>
```

Attribute	Description	Values
root.pname	Root pitch class	a-g
root.accid	Root accidental	MEI accidental tokens: s=sharp, f=flat, n=natural, ss=double-sharp, ff=double-flat
bass.pname	Slash-bass pitch class	a-g
bass.accid	Slash-bass accidental	same as root.accid
chord.kind	Semantic chord quality	MusicXML kind vocabulary: major, minor, major-seventh, dominant, etc.

The text content always sits alongside the structured attributes and keeps the display surface intact, so the file works both for a human reader and for a parser that only reads text content.

There are two ODD variants:

File	Purpose
mei-customization/hamon.odd	Full HAMON MEI customization (all MEI modules)
mei-customization/mei-verovio-hamon.xml.odd	Verovio-compatible subset — uses only modules Verovio supports

For validation, there's a compiled RelaxNG schema at `mei-customization/examples/hamon.odd.rng`.

Analytical layer (HA-MEI, v0.2.0)

MEI has no native element for tonal regions or non-harmonic tones, so HAMON carries the analytical layer on `<harm>` through the `@type` token instead (a `hamon:` prefix is allowed and ignored). The adapter `hamonpy.adapters.mei.mei_to_hamon(text)` reads these in document order and builds a `HamonSequence` with regions, `HarmonyLabel.layer`, and `ToneSemantic`.

@type is a token set (MEI NMTOKENS). Alongside the analytical/region tokens below, a `<harm>` may also carry a **harmony-layer provenance token** — `omr` (OMR-detected) or `score` (written). The two coexist happily (e.g. `type="omr function"`): a reader takes the tokens it understands and ignores the rest. `hamonpy` keeps the analytical/region token and ignores `omr/score`.

<code><harm type="..."></code>	Maps to
key / region / modulation	a <code>TonalRegion</code> (content = a key: C, A:minor, D:dorian; bare lowercase a ⇒ A minor). The first key region is key, later ones modulation.
tonicization	a nested <code>TonalRegion</code> (content = a degree like V, bVI; local tonic computed from the current key, parent set).
function / degree / bass / chord	a label on that analytical layer.

<code><harm type="..."></code>	Maps to
tone / melodic	a ToneSemantic on the melodic layer (content = a pitch + class, e.g. F[NHT:passing], C[HT]).
(absent / other)	a plain harmony label (chord symbol, Roman numeral, ...).

Time-aligned positions (schema 0.2.1)

mei_to_hamon also folds each `<harm>`'s time anchoring into the group's optional position (grammar/hamon-schema.json ≥ 0.2.1):

- `@startid` ⇒ `position = { ref }`.
- `@tstamp` ⇒ `position = { measure, beat }`, where `measure` is the enclosing `<measure @n>` (tracked in document order) and `beat` is the timestamp.

Region directives (`type="key"/tonicization/...`) produce a `TonalRegion` rather than a group, so they never consume a position — the next harmony keeps its own anchor.

```

<harm type="hamon:key"          tstamp="1">C</harm>
<harm type="degree"            tstamp="1">I</harm>
<harm type="hamon:tonicization" tstamp="2">V</harm>    <!-- → G-major region
↪ -->
<harm type="degree"            tstamp="2">V7/V</harm>
<harm type="function"          tstamp="2">D</harm>    <!-- dominant, not a D
↪ chord -->
<harm type="tone"              tstamp="2.5">F[NHT:passing]</harm>
<harm type="hamon:key"          tstamp="3">A:minor</harm> <!-- modulation -->

```

This reuses the very same surface grammar and region/tone semantics as [analysis.md](#); the only difference is that MEI is the carrier.

The inverse, `hamon_to_mei(seq, wrap=False)`, serializes a `HamonSequence` back into these `<harm>` elements — regions become `hamon:key/hamon:modulation/hamon:tonicization`, labels carry their `@type` layer and their beat as `@tstamp`, and tones come out as `[HT]/[NHT:...]` — and it round-trips with `mei_to_hamon`. With `wrap=True` the elements are nested in one `<measure n="...">` per measure the positions name (a single anonymous `<measure>` when nothing is positioned), and two `<harm>`s on the same `@tstamp` read back as one layered group. Full Verovio rendering is still to come, in Phase 8.1b–8.6.

Figured bass: `<fb>` and `<f>`

MEI has dedicated structured encoding for figured bass: `<fb>` (a child of `<harm>`) wraps individual figure elements, each an `<f>`:

```

<harm tstamp="1">
  <fb>
    <f>6</f>
    <f>5</f>
  </fb>
</harm>

```

An `<f>` can also carry an accidental attribute:

```

<harm tstamp="1">
  <fb>
    <f accid="s">6</f>  <!-- #6 -->
    <f>5</f>
  </fb>
</harm>

```

The HAMON fixture corpus keeps figured bass as plain text in <harm> (e.g. <harm>6-5</harm>), which is simpler and more portable. Save the structured <fb> form for when you need full MEI encoding — for instance, to render figured bass symbols in Verovio.

Complete MEI example

```

<?xml version="1.0" encoding="UTF-8"?>
<mei xmlns="http://www.music-encoding.org/ns/mei" meiversion="5.1">
  <meiHead>
    <fileDesc>
      <titleStmt><title>Example</title></titleStmt>
      <pubStmt/>
    </fileDesc>
  </meiHead>
  <music>
    <body>
      <mdiv>
        <score>
          <scoreDef>
            <staffGrp>
              <staffDef n="1" lines="5" clef.shape="G" clef.line="2"
                meter.count="4" meter.unit="4"/>
            </staffGrp>
          </scoreDef>
          <section>
            <measure n="1">
              <staff n="1">
                <layer n="1">
                  <note xml:id="n1" pname="c" oct="4" dur="4"/>
                  <note xml:id="n2" pname="g" oct="3" dur="4"/>
                </layer>
              </staff>
              <!-- Chord symbol with hamon structured attributes -->
              <harm startid="#n1" type="chordSymbol"
                root.pname="c" chord.kind="major-seventh">CΔ7</harm>
              <!-- Dominant seventh -->
              <harm startid="#n2" type="chordSymbol"
                root.pname="g" chord.kind="dominant">G7</harm>
            </measure>
            <measure n="2">
              <staff n="1">
                <layer n="1">
                  <note xml:id="n3" pname="c" oct="4" dur="1"/>
                </layer>
              </staff>
              <!-- Figured bass, structured form -->

```

```

    <harm startid="#n3">
      <fb><f>6</f><f>4</f></fb>
    </harm>
  </measure>
</section>
</score>
</mdiv>
</body>
</music>
</mei>

```

Mapping: JSON fields → MEI

All mappings are relative to the canonical JSON schema (grammar/hamon-schema.json). MEI encodes harmony at two depths: **text form** (the surface string in <harm> text content) and **structured form** (the HAMON ODD attributes on <harm>). Keep both present at once.

Chord symbols (kind: "chordSymbol")

JSON field	HAMON → MEI	MEI → HAMON	Loss / note
surface	text content of <harm>	text content → parsed surface	always present
root.note	root.pname="c" (structured)	root.pname → uppercase note	lowercase in MEI (c-g)
root.accidental: "sharp"	root.accid="s" (structured)	"s" → sharp	MEI accid token differs from HAMON
root.accidental: "flat"	root.accid="f"	"f" → flat	none
root.accidental: "double-sharp"	root.accid="ss" or "x"	→ double-sharp	two MEI tokens map to same HAMON value
root.accidental: "double-flat"	root.accid="ff"	→ double-flat	none
quality + seventh	chord.kind="major-seventh" (structured)	MusicXML kind vocabulary	same vocabulary as MusicXML
bass.note	bass.pname="e" (structured)	bass.pname → uppercase note	none
bass.accidental	bass.accid="f"	same as root accid	none
rendering.seventhGlyph	surface text only	from surface	not in structured attrs

Figured bass (kind: "figuredBass")

JSON field	HAMON → MEI	MEI → HAMON	Loss / note
surface	<harm>6-5</harm> text	text → parsed surface	text form preferred for portability
number + tail	<fb><f>6</f><f>5</f></fb>	<fb>children → number + tail	structured form for rendering

JSON field	HAMON → MEI	MEI → HAMON	Loss / note
alterations[]	<f accid="s">6</f>	@accid → Alteration	none

Other notation systems

JSON kind	HAMON → MEI	MEI → HAMON	Loss / note
roman	<harm>V7</harm> text	text → parsed as surface	no structured MEI form
nashville	<harm>4m</harm> text	text → parsed	no structured MEI form
functional	<harm>T</harm> text	text → parsed	no structured MEI form
noChord	<harm>N.C.</harm> text	text → noChord	none
text	<harm>...</harm> text	text → TextSemantic	none

Structure and anchoring

JSON field	HAMON → MEI	MEI → HAMON
HarmonyGroup time position	@tstamp (beat) or @startid (note xml:id)	@tstamp / @startid → time position
Display placement	@place="above" / "below"	—
Notation system	type="chordSymbol" (HAMON convention)	@type value → detectedSystem

Losses and ambiguities

- **Notation system ambiguity in plain text:** <harm>V7</harm>, <harm>5</harm>, and <harm>6-5</harm> say nothing about which system they use. HAMON infers roman, nashville, and figuredBass respectively from the grammar; a @type attribute would spell it out. So far the HAMON ODD defines no controlled vocabulary for @type beyond chordSymbol.
- **Structured vs. text form:** A <harm> can hold both root.pname/chord.kind attributes and text content. Given only structured attributes, HAMON reconstructs a surface from them; given only text, the structured fields are simply missing. This is why the HAMON ODD encourages keeping both.
- **<fb> vs. text figures:** Figured bass shows up either as <fb><f>6</f><f>5</f></fb> (structured) or as <harm>6-5</harm> (text). HAMON's MEI extractor reads both.
- **No native Roman numeral structure:** MEI has @deg (scale degree) in MEI.analytical, but nothing dedicated for Roman numerals in MEI.harmony, so Roman numerals go in <harm> as plain text.
- **MEI version:** The HAMON ODD targets MEI 5.x. MEI 4.x files use a slightly different attribute vocabulary for some elements.

How HAMON reads MEI

hamonpy reads MEI <harm> elements directly in Python — there's no external importer. Two entry points cover the two use cases:

- **Analytical adapter** (hamonpy/hamonpy/adapters/mei.py) — `mei_to_hamon(text)` scans <measure> and <harm> elements in document order, maps each <harm>'s @type token and text content to a hamon surface line (regions, layers, tones), and parses the result into a HamonSequence (the analytical layer above).
- **Surface extractor** (hamonpy/hamonpy/adapters/formats.py) — `read_harmony_labels("mei", text)` pulls just the surface strings, scanning for:
 - <harm label="..."> — reads the label attribute.
 - <harm ...>text</harm> — reads the text content directly.

Either way the surface string is handed to HAMON's grammar (`parse_hamon_sequence`), which parses it into the typed AST.

Relationship to Verovio

Verovio draws <harm> elements above the staff:

- **Text content** renders as styled text — this works today, with no C++ changes.
- **<fb> children** render as stacked figured bass symbols via `DrawFb()`.
- **Rich chord symbol rendering** (superscript 7, the Δ glyph, italic m) is planned, but it needs C++ changes in `src/view_control.cpp` : `DrawHarm()`.

See `verovio.md` for the full integration architecture and roadmap.

Examples from the HAMON fixture corpus

Fixture	MEI snippet	HAMON surface	HAMON kind
C_major	<harm type="chordSymbol">C</harm>	C	chordSymbol
CΔ7	<harm type="chordSymbol">CΔ7</harm>	CΔ7	chordSymbol
B_hdim7	<harm type="chordSymbol">Bø7</harm>	Bø7	chordSymbol
rn_V7	<harm>V7</harm>	V7	roman
rn_ii	<harm>ii</harm>	ii	roman
fb_6_5	<harm>6-5</harm>	6-5	figuredBass
ns_2m	<harm>2m</harm>	2m	nashville

Full fixture files: `fixtures/chords/`, `fixtures/roman/`, `fixtures/figuredbass/`, `fixtures/nashville/`. Schema and examples: `mei-customization/`.

Humdrum ****kern** — Harmony Encoding

Overview

Humdrum is David Huron's toolkit for music analysis. Everything in it is built around **spines**: parallel streams of tokens separated by tabs, each named by an **exclusive interpretation** header on the first line (****kern**, ****harm**, and so on). One file can carry several spines at once, each describing a different aspect of the same music.

Harmony goes in its own spines. The spine type fixes two things: the notation system (chord symbols, Roman numerals, figured bass, or functional) and, for chord symbols, which vocabulary standard applies. HAMON maps every Humdrum harmony spine into its unified HamonSequence AST.

File structure

A minimal Humdrum file comes in three parts:

**harm	← exclusive interpretation (spine header)
I	← data record (one per "event")
V7	← data record
*-	← spine terminator

Multi-spine files tab-separate their tokens. Comments start with ! (local) or !! (global). Barlines start with =. Tandem interpretations — tempo, key, meter, and the like — start with * but aren't *-.

```
**kern  **harm
=1  =1
4c 4e 4g      I
4g 4b 4d      V
=2  =2
1c 1e 1g      I
*-  *-
```

HAMON reads harmony only from the spines that carry it; it ignores the ****kern** pitch data when extracting harmony.

Layered import (analytical layer, v0.2.0)

A Humdrum score often carries **several harmony spines at once** — say ****harm** (Roman-numeral analysis), ****function** (Riemannian function), and ****fb** (figured bass) all running alongside ****kern**. The layered adapter `hamonpy.adapters.humdrum.humdrum_to_hamon(text)` keeps that alignment intact:

Spine	Analytical layer
**harm	degree
**function	function
**fb	bass (read as figured bass)
**mxhm / **jazz / **irb / **harte	chord

Every data line with at least one non-null harmony token turns into one `HarmonyGroup`, and the co-temporal spine tokens become its primary labels, each tagged with its `HarmonyLabel.layer`. The ****function** spine un-shadows `D` so it reads as the dominant rather than a `D`-major chord, and ****fb** numbers read as figured bass. When a file has a **single** harmony spine, `HAMON` drops the layer tag — you get a plain single-system reading — and sets the sequence hint to match (****harm**→`rn`, ****function**→`fun`, ****fb**→`fb`, chord spines→`cs`).

```
**kern  **harm  **function  **fb
4c      I      T             .
4g      V7     D             6-5
*_      *_     *_            *_
```

becomes two groups: `[rn:I, fn:T]` and `[rn:V7, fn:D, fb:6-5]`.

If you just need a flat, deduplicated list of surfaces with no layers, the `formats.read_harmony_labels("kern_text")` extractor is still there — the fixture round-trip tests use it.

Time-aligned positions (schema 0.2.1)

`humdrum_to_hamon` follows the numbered barlines (=N) and sets each group's `position = {measure}`. It leaves out the beat within the measure: pinning that down would mean parsing the ****kern** spine's rhythm — the note durations — which the harmony adapter doesn't read. Measure-level alignment is as fine as the barlines alone can reliably get you. Bare = barlines (no number) produce no position.

Harmony spine types

****harm** — generic harmony

This is the catch-all spine for any written harmony label. In practice it holds whatever notation suits the piece: Roman numerals in analytical scores, Nashville numbers in pop and country, figured bass in Baroque works. You have to read a token's meaning from context — the key signature, the labels before it, or a `HAMON @<system>` hint.

```
**harm
I
IV
V7
I
*_
```

```
**harm
6-5
4-3
*_
```

```
**harm
1
4
5
2m
*_
```

HAMON routes ****harm** tokens by pattern: - A Roman numeral pattern (I, bVII, ii65, ...) → roman - A figured bass pattern (6-5, 4-3, 7-6, ...) → figuredBass - A Nashville pattern (1, 4m, b7, ...) → nashville - Anything else → chord symbol or textLabel

****mxhm — MusicXML chord symbols**

Chord symbols in the MusicXML convention: a root note (capital letter), an optional ASCII accidental (# or b), then a quality suffix. This spine type says outright that its vocabulary is “the same as MusicXML <harmony>”.

```
**mxhm
C
Am
G7
Cmaj7
Bø7
*_
```

Accidentals look like Bb, F#, Eb. Quality suffixes follow the MusicXML kind-text conventions: m, maj7, m7, 7, dim, °, aug, +, ø7, sus4, sus2, and so on.

****jazz — jazz chord symbols**

Chord symbols in jazz notation convention. The surface forms overlap heavily with ****mxhm**; the distinction mainly records provenance — a jazz lead sheet versus a MusicXML export — and can influence how enharmonic spellings get chosen. At the grammar level, HAMON treats ****jazz** exactly like ****mxhm**.

```
**jazz
CΔ7
Am7
G7
Bø7
Bb
*_
```

Unicode quality glyphs (Δ, °, ø) turn up often in jazz notation, and HAMON’s chordSymbol rule accepts them.

****irb — IRB chord symbols**

Chord symbols in the International Reference Basis vocabulary — a chord-symbol standardization used in some academic and digital-library settings. The surface forms match jazz and MusicXML chord symbols, and at parse time HAMON treats ****irb** exactly like ****mxhm** and ****jazz**.

****fb — figured bass**

Figured bass notation: a bass pitch lives in ****kern**, and the ****fb** spine gives the interval structure above it. An ****fb** token holds the figures only — never the bass note.

```
**fb
6-5
4-3
7-6
*_
```

Figures are integers. When several sound at once, they stack vertically inside one token, separated by -. HAMON parses these as `figuredBass` labels through the `numberHarmony` rule — the hyphen-digit signal is what tells figured bass apart from Nashville numbers.

****function — functional harmony**

Functional harmony labels in Riemannian notation — T, S, D and their variants. This spine type comes out of Humdrum extensions used in German-language analytical traditions.

```
**function
T
S
D
T
*_
```

HAMON maps ****function** tokens to the `functionalHarmony` rule. It supports T, S, D, PD, SD, and DD. Secondary functions and variants (Tp, Sp, DP, and the like) fall through to `textLabel` until the grammar's `functionalHarmony` rule grows to cover them.

Token conventions shared across spines

Convention	Example	Notes
Null token	.	Spine continues with no new harmony; HAMON skips it
Rest token	r	No harmony; HAMON skips it
Barline	=1, =	Not a harmony token; HAMON skips it
Tandem interpretation	*M4/4, *k[b-]	Not a harmony token; HAMON skips it
Comment	! label	Not a harmony token; HAMON skips it

Mapping: JSON fields → Humdrum

All mappings are relative to the canonical JSON schema (`grammar/hamon-schema.json`). Humdrum encodes every harmony as a plain-text token, and the spine type (****mxhm**, ****harm**, ****fb**, ****function**) picks the notation system.

Chord symbols (kind: "chordSymbol") — spines ****mxhm, **jazz, **irb**

JSON field	HAMON → Humdrum token	Humdrum token → HAMON	Loss / note
root.note	first char(s): C, G, A	first letter → root.note	none
root.accidental: "sharp"	# suffix on root: C#	# → sharp	ASCII only; rendering.accidentalGlyphs may hold original glyph
root.accidental: "flat"	b suffix: Bb	b → flat	none
quality: "major"	no suffix: C	no suffix → major	none
quality: "minor"	m: Cm	m → minor	none
quality: "augmented"	+ or aug: C+	→ augmented	none
quality: "diminished"	° or dim: C°	→ diminished	none
quality: "half-diminished"	ø: Cø	→ half-diminished	none
seventh: "dom7"	7: G7	7 after root → dom7	none
seventh: "maj7"	maj7, Δ7, M7: Cmaj7	→ maj7; glyph → rendering.seventhGlyph	multiple surface variants
seventh: "min7"	m7: Am7	→ min7	none
seventh: "dim7"	°7: B°7	→ dim7	none
seventh: "hdim7"	ø7: Bø7	→ hdim7	none
suspensions: [4]	sus4: Csus4	→ suspensions:[4]	none
suspensions: [2]	sus2: Csus2	→ suspensions:[2]	none
bass.note	/E: C/E	after / → bass.note	none
extensions[] / alterations[]	appended to token: C7(#11)	captured in rendering.rawParts	structured extraction not yet implemented

Roman numerals (kind: "roman") — spine ****harm**

JSON field	HAMON → **harm	**harm → HAMON	Loss / note
prefixAccidentals[]	b / # prefix: bVII	prefix chars → prefixAccidentals	none
degree tail	Roman token: V, ii appended: V65	token → degree remaining chars → tail	case preserved not further parsed
secondary	/V suffix	after / → secondary	none

Figured bass (kind: "figuredBass") — spines ****fb or **harm**

JSON field	HAMON → token	token → HAMON	Loss / note
number	leading integer: 6	first digit → number	none

JSON field	HAMON → token	token → HAMON	Loss / note
tail	-5 suffix: 6-5	- \d signal → figuredBass kind; remainder → tail	none

Nashville (kind: "nashville") — spine **harm

JSON field	HAMON → token	token → HAMON	Loss / note
prefixAccidentals[]	b / # prefix: b7	prefix → prefixAccidentals	none
number	integer: 4	digit → number	disambiguated from figuredBass by absence of - \d
tail	quality suffix: 4m	suffix → tail	none

Functional (kind: "functional") — spine **function

JSON field	HAMON → token	token → HAMON
chain[0]	T, S, D, PD, SD, DD	token → chain[0]
chain[1..n]	not representable in a single token	—

Other

JSON kind	HAMON → Humdrum	Humdrum → HAMON
noChord	. (null token)	. → skipped (no label)
text	token as-is	unmatched token → TextSemantic

Losses and ambiguities

- ****harm ambiguity:** The spine type on its own can't tell Roman numeral from Nashville. HAMON settles it by grammar detection — Roman tokens have letter-based degrees, Nashville tokens are pure integers with an optional quality suffix — and a @<system> hint in the output clears up anything left over.
- **Enharmonics:** Humdrum uses ASCII accidentals (#, b), and HAMON keeps the exact source spelling (surface-first).
- **Extensions beyond the grammar:** Complex chord symbols like C7(#11b9) that outrun the current HAMON grammar rules are kept losslessly as textLabel.
- **Time anchoring:** Humdrum encodes a harmony's time position through where its token sits within the spine. HAMON currently pulls out a flat label list and keeps no beat or onset information.
- **Multiple harmony spines:** If a file has both **harm and **mxhm, HAMON reads the left-most matching column. Combining layers isn't supported yet.

How HAMON reads Humdrum

hamonpy reads Humdrum with a pure-Python, text-level extractor (`hamonpy/hamonpy/adapters/formats.py, _extract_kern`):

1. Scans for the first spine-header line — one carrying an exclusive interpretation like `**kern, **harm, **mxhm, **jazz, **irb, **harte, **fb, or **function`.
2. From that header, records which columns are harmony spines (`**harm, **mxhm, **jazz, **irb, **harte, **fb, **function`).
3. Reads down the data lines, skipping the spine terminator (`*-`), comments (`!`), tandem interpretations (`*...`), and barlines (`=`). For each harmony column it collects every non-null token (anything that isn't `.`).
4. Normalizes a few tokens on the way out — e.g. Humdrum Harte notation `C:maj7` becomes `Cmaj7`.
5. If no spine header is found, it falls back to grabbing any chord-like token (one that starts with a note letter A–G) from the non-comment lines.

Results are deduplicated, order-preserving. For layered, position-aware reading — several harmony spines kept aligned into `HarmonyGroups` — use `hamonpy.adapters.humdrum.humdrum_to_hamon(text)` described above; `_extract_kern` gives you the flat surface list.

The format keys hamonpy recognizes for Humdrum harmony are `kern_harm`, `kern_mxhm`, `kern_jazz`, `kern_irb`, `kern_harte`, `kern_fb`, and `kern_function` (any key containing `kern` or `humdrum` routes here).

How HAMON writes Humdrum

`hamonpy.export.hamon_to_humdrum(seq)` (the `humdrum` target of `hamon export / hamon report`) is the mirror of the layered reader:

- **one spine per analytical layer** — a layered sequence such as `m:25,ts:1,cs:C,rn:I` becomes parallel `**mxhm` and `**harm` spines, one data line per group, with `.` where a layer is silent; an unlayered sequence is a single spine chosen by its system (`**mxhm, **harm, **fb, **function`);
- **the key as a tandem interpretation** — a tonal region opens with `*C: / *b-:` on every spine (upper case major, lower case minor), which the reader turns back into the region; a tonicization (`@key:V`) has no Humdrum spelling and is not written;
- **measures as barlines** — `=25` before the first group of each measure, from the groups' `m:` positions. The beat is not written: a harmony-only file has no `**recip` to place it on, so `ts:` is the one thing this round-trip loses.

Layers Humdrum has no spine for (melodic, key, tone) are omitted and reported as loss by `hamon report`.

```
**mxhm  **harm
*C:  *C:
=25 =25
C    I
=28 =28
Fm7 iv7
Bb7 bVII7
*_  *_
```

Examples from the HAMON fixture corpus

Fixture	Spine	Token	HAMON kind	Notes
C_major	**mxhm	C	chordSymbol	major triad, no suffix
Bb_major	**mxhm	Bb	chordSymbol	flat accidental
C_delta7	**jazz	CΔ7	chordSymbol	Unicode delta glyph
C_M7	**irb	CM7	chordSymbol	capital-M major seventh
B_hdim7	**mxhm	Bø7	chordSymbol	half-diminished glyph
C_sus4	**mxhm	Csus4	chordSymbol	suspended fourth
rn_V7	**harm	V7	roman	dominant seventh Roman
rn_ii	**harm	ii	roman	lowercase = minor
fb_6_5	**fb	6-5	figuredBass	suspension figure
fb_6_5	**harm	6-5	figuredBass	same figure, different spine
ns_2m	**harm	2m	nashville	minor 2

Full fixture files: fixtures/chords/, fixtures/roman/, fixtures/figuredbass/, fixtures/nashville/.

LilyPond — Harmony Encoding

Overview

LilyPond engraves music from a plain-text input language, and its typographic quality has made it a favourite in academic and new-music circles. In a LilyPond source file, harmony labels show up in two places:

1. **\chordmode blocks** — LilyPond’s native chord notation, meant for printing chord symbols above a staff.
2. **% hamon-surface: comments** — a hamon-specific annotation for the label types LilyPond can’t write natively (Roman numerals, figured bass, Nashville, functional).

LilyPond has no structured harmony model to import from, so HAMON reads it with a text-level regex extractor that handles both forms.

Export: real \chordmode since 2026-09-03

The exporter used to emit a placeholder `c1` with every harmony parked in `% hamon-surface: comments`. It produced a valid file that contained **no LilyPond harmony at all** — and because the reader preferred those comments, the round-trip reported LilyPond as near-lossless. It was reading our comments back.

`hamon_to_lilypond` now writes chord tokens:

```
\version "2.24.0"
\chordmode {
  d1:m7 g:7 c:maj7 f bes:maj7 s ees:7
}
```

Two rules it follows, both about not asserting what HAMON does not know:

- **The duration sits on the first token only**, and the rest inherit it. Writing `1` on every chord would state a rhythm the label stream has no opinion about.
- **A chord it cannot spell exactly becomes a skip (s)**, not an approximation. The `s` above is an `F#m7b5`: LilyPond can say it (`fis:m7.5-`) but this writer’s mapping does not yet, and emitting `fis:m7` would be a *different chord*. The loss is reported rather than hidden — see `documentation/processing.md` on the xencoding report.

Extending the mapping to LilyPond’s step-alteration syntax (`a:7.9-`, `g:7.9-.13-`) would remove most of those skips; it needs matching reader support and is on the roadmap.

The reader accepts durations on chord tokens (`d1:m7`), which it previously rejected — so it can read LilyPond written by LilyPond, not only files this exporter produced.

\chordmode syntax

\chordmode takes a run of pitch tokens, each one optionally followed by a :modifier suffix. Pitches use lowercase Dutch note names, and the modifier after : sets the chord quality.

Pitch names

HAMON root	LilyPond token
C	c
D	d
E	e
F	f
G	g
A	a
B	b
C# / Db	cis / des
Bb	bes (not bes = B $\flat$, not bb)
Eb / D#	ees or es / dis
F# / Gb	fis / ges
Ab / G#	aes or as / gis

Quality modifiers (:suffix)

LilyPond modifier	HAMON surface	Semantic
(none)	C	major triad
:m	Cm	minor triad
:aug	C+	augmented triad
:dim	C $^\circ$	diminished triad
:7	G7	dominant seventh
:7+	Cmaj7	major seventh (display depends on majorSevenSymbol)
:m7	Am7	minor seventh
:dim7	B $^\circ$ 7	diminished seventh
:m7.5-	B $\flat$ 7	half-diminished seventh
:sus2	Csus2	suspended second
:sus4	Csus4	suspended fourth
:maj7	Cmaj7	major seventh (alternative spelling)

majorSevenSymbol — display override

Internally, LilyPond encodes every major-seventh chord as :7+, no matter which glyph it prints. What you actually see is controlled by \set majorSevenSymbol:

```
\chordmode {  
  \set majorSevenSymbol = \markup { "\Delta7" }  
  c:7+ % renders as C\Delta7  
}  
  
\chordmode {  
  \set majorSevenSymbol = \markup { "M7" }  
}
```

```
c:7+    % renders as CM7
}
```

With no `majorSevenSymbol` override, LilyPond renders `:7+` as `maj7`. HAMON's LilyPond extractor reads the `majorSevenSymbol` markup string and applies it as the surface for every `:7+` token in that block.

Slash bass

A bass note is appended with `/+`:

```
\chordmode { c/e }    % C/E (first inversion)
```

Full example

```
\chordmode {
  c      % C
  a:m    % Am
  g:7    % G7
  \set majorSevenSymbol = \markup { "Δ7" }
  c:7+   % CΔ7
  bes    % Bb
  b:m7.5- % Bø7
}
```

% hamon-surface: comment annotation

LilyPond has no native syntax for Roman numerals, Nashville numbers, figured bass, or functional labels. So for fixture files — and any LilyPond score that carries these label types — HAMON puts a comment annotation on the line before the relevant note (or in place of it):

```
% hamon-surface: V7
% hamon-surface: ii
% hamon-surface: 6-5
% hamon-surface: 2m
```

HAMON's extractor scans for lines matching `% hamon-surface: <label>` and collects those label strings directly, skipping `\chordmode` altogether. When a file has both a `% hamon-surface:` annotation and a `\chordmode` block, the annotation wins.

Mapping: JSON fields → LilyPond

Every mapping below is stated against the canonical JSON schema (`grammar/hamon-schema.json`). LilyPond has two encoding paths: `\chordmode` for chord symbols, and `% hamon-surface:` comments for everything else.

Chord symbols (`kind: "chordSymbol"`) — `\chordmode`

JSON field	HAMON → LilyPond	LilyPond → HAMON	Loss / note
root.note	lowercase Dutch name: c, g, a	Dutch name → uppercase note	none
root.accidental: "sharp"	is suffix: cis, fis	is → sharp	rendering.accidentalGlyphs stores original
root.accidental: "flat"	es/ees suffix: bes, des	es/ees → flat	bes not bflat
quality: "major"	no :modifier	no modifier → major	none
quality: "minor"	:m	:m → minor	none
quality: "augmented"	:aug	:aug → augmented	none
quality: "diminished"	:dim	:dim → diminished	none
quality: "half-diminished"	:m7.5-	:m7.5- → half-diminished + seventh: "hdim7"	LilyPond bundles quality+seventh
seventh: "dom7"	:7	:7 → dom7	none
seventh: "maj7"	:7+ + \set majorSevenSymbol = \markup{"Δ7"}	:7+ → maj7; markup string → rendering.seventhGlyph	glyph depends on majorSevenSymbol; defaults to maj7 if absent
seventh: "min7"	:m7	:m7 → min7	none
seventh: "dim7"	:dim7	:dim7 → dim7	none
suspensions: [4]	:sus4	:sus4 → suspensions:[4]	none
suspensions: [2]	:sus2	:sus2 → suspensions:[2]	none
bass.note	/+<pitch> suffix	after /+ → bass	none

All other notation systems — % hamon-surface: comment

LilyPond has no native syntax for kind: "roman", "nashville", "figuredBass", "functional", "noChord", or "text". HAMON writes the HarmonyLabel.surface string verbatim into a comment annotation:

JSON field	HAMON → LilyPond	LilyPond → HAMON
surface (any non-chordSymbol kind)	% hamon-surface: V7	comment text → surface; system detected by grammar

Losses and ambiguities

- **Major-seventh display:** LilyPond stores only :7+; the rendered symbol comes from a separate \set directive. HAMON reads that directive when it is there and falls back to maj7 when it isn't. Going the other way (HAMON → LilyPond) means writing the correct majorSevenSymbol markup.
- **No native Roman/Nashville/figured-bass syntax:** LilyPond simply has no mechanism for these systems. The % hamon-surface: comment convention is HAMON's own, and LilyPond renderers ignore it.
- **Pitch spelling:** LilyPond's Dutch names spell B_b as bes (not bb) and E_b as ees/es. HAMON's extractor normalizes these back to ASCII accidentals (b, #).

How HAMON reads LilyPond

The adapter (hamonpy/hamonpy/adapters/formats.py, _extract_lilypond):

1. Scans for % hamon-surface: <label> lines. If found, returns those labels directly.
 2. Otherwise, finds all \chordmode { ... } blocks.
 3. For each token inside a block: strips { / }, skips LilyPond commands (\...), parses <pitch>[:<modifier>], converts Dutch pitch name + accidental suffix to ASCII, maps the modifier to a HAMON surface suffix.
 4. Reads \set majorSevenSymbol = \markup { "..." } and applies the display string to subsequent :7+ tokens.
-

Examples from the HAMON fixture corpus

Fixture	LilyPond token	HAMON surface	HAMON kind
C_major	c	C	chordSymbol
Bb_major	bes	Bb	chordSymbol
C_minor	c:m	Cm	chordSymbol
G_dom7	g:7	G7	chordSymbol
C_maj7	c:7+ (no symbol override)	Cmaj7	chordSymbol
C_delta7	c:7+ + "Δ7"	CΔ7	chordSymbol
C_M7	c:7+ + "M7"	CM7	chordSymbol
B_hdim7	b:m7.5-	Bø7	chordSymbol
rn_V7	% hamon-surface: V7	V7	roman
fb_6_5	% hamon-surface: 6-5	6-5	figuredBass
ns_2m	% hamon-surface: 2m	2m	nashville

Full fixture files: fixtures/chords/, fixtures/roman/, fixtures/figuredbass/, fixtures/nashville/.

ABC Notation — Harmony Encoding

Overview

[ABC notation](#) is a plain-text notation widely used for folk, traditional, and session music. It embeds harmony labels as **quoted strings**, placed right before the note they apply to.

ABC keeps no separate semantic structure for harmony — the quoted string is always a plain surface label. HAMON parses whatever is inside the quotes with the same grammar it uses for every other format, pulling the quoted strings out with a text-level regex extractor.

Chord symbol syntax

A chord symbol is a quoted string that sits directly before a note, with no space between them:

```
"Am7"A2 "G7"G4
```

The ABC standard recommends the format `<note><accidental><type>/<bass>`:

- `<note>` — A–G
- `<accidental>` — b (flat) or # (sharp)
- `<type>` — quality and extension suffix: m, min, maj, dim, aug, +, sus, 7, 9, etc.
- `/<bass>` — optional slash bass note

```
"C"C          % C major
"Am"A         % A minor
"G7"G         % G dominant seventh
"F#min7"F     % F# minor seventh
"C/E"E       % C over E bass
"Bb"B        % B-flat major
```

ABC implementations are liberal about chord spellings; the standard enforces no strict grammar beyond the recommended format. That looseness suits HAMON's surface-first approach.

Alternate chords

ABC allows alternate chords for display (playback may ignore them):

```
"G(Em)"G      % G, with Em as an alternate voicing label
```

HAMON grabs the whole quoted string and parses it. If the content doesn't match the `chordSymbol` rule, it falls through to `textLabel`.

Annotations vs. chord symbols

A quoted string that starts with ^, _, <, >, or @ is an **annotation** — a text-placement marker, not a chord symbol. HAMON's ABC extractor skips any string beginning with one of these characters.

Non-chord-symbol labels

ABC has no native syntax for Roman numerals, figured bass, Nashville numbers, or functional labels. In practice people just write them as quoted strings, and HAMON's grammar works out the system from the content:

```
"V7"C          % Roman numeral – hamon detects kind: "roman"
"6-5"C         % Figured bass – hamon detects kind: "figuredBass"
"4m"C          % Nashville – hamon detects kind: "nashville"
"N.C."z        % No chord – hamon detects kind: "noChord"
```

Mapping: JSON fields → ABC

Every mapping below is stated against the canonical JSON schema (grammar/hamon-schema.json). ABC stores all harmony as a plain quoted surface string — there is no per-field structure.

JSON field	HAMON → ABC	ABC → HAMON	Loss / note
surface (any kind)	"CΔ7" (quoted surface string)	quoted string content → surface	none — surface stored verbatim
semantic (any kind)	not encoded separately	derived by parsing surface	no structured fields in ABC
rendering.*	reflected in surface	recovered by parsing surface	glyph variants preserved via surface

All kind values map the same way:

JSON kind	ABC quoted string example
chordSymbol	"CΔ7", "Am7", "Bb", "Bø7"
roman	"V7", "ii", "bVII"
figuredBass	"6-5", "4-3"
nashville	"2m", "b7"
functional	"T", "D"
noChord	"N.C."
text	"Ger65", "It6"

Losses and ambiguities

- **No system declaration:** at the encoding level, ABC can't tell a Roman numeral from a chord symbol. HAMON's grammar infers the system from the content, and a `@<system>` hint in the `.hamon` output settles the edge cases.
 - **Annotation prefix collision:** strings starting with `^`, `_`, `<`, `>`, `@` are ABC annotations, not chords, so HAMON skips them.
 - **Alternate chord notation** ("G(Em)"): HAMON keeps the full string, but whether the parenthesized part is an alternative or an extension is ambiguous — it lands in `rendering.rawParts` or `textLabel`.
 - **Time anchoring:** ABC ties chord symbols to notes by their position in the text stream. HAMON pulls out a flat label list and does not keep beat position.
-

How HAMON reads ABC

The adapter (`hamonpy/hamonpy/adapters/formats.py`, `_extract_abc`):

1. Scans the source for all `"..."` quoted strings.
 2. Skips strings starting with annotation characters (`^`, `_`, `<`, `>`, `@`).
 3. Returns the content of each remaining quoted string as a surface label.
-

Examples from the HAMON fixture corpus

Fixture	ABC snippet	HAMON surface	HAMON kind
C_major	"C"C	C	chordSymbol
Bb_major	"Bb"B	Bb	chordSymbol
C_delta7	"CΔ7"C	CΔ7	chordSymbol
B_hdim7	"Bø7"B	Bø7	chordSymbol
rn_V7	"V7"C	V7	roman
fb_6_5	"6-5"C	6-5	figuredBass
ns_2m	"2m"C	2m	nashville

Full fixture files: `fixtures/chords/`, `fixtures/roman/`, `fixtures/figuredbass/`, `fixtures/nashville/`.

MuseScore — Harmony Encoding

Overview

[MuseScore](#) is an open-source music notation editor. Its native format, MSCZ, is a ZIP archive wrapping an XML file with a .mscx extension. MuseScore can also export standard MusicXML, and the two representations encode harmony differently.

HAMON reads MuseScore's native MSCX with a text-level extractor.

MSCX harmony element

In MSCX, a chord symbol is a <Harmony> element (capital H), reached through a <Measure> → <voice> → <Harmony> path. The surface label lives in it as plain text:

```
<Harmony>
  <text>Cmaj7</text>
</Harmony>
```

Older MuseScore versions (v3 and below) may use <name> instead of <text>:

```
<Harmony>
  <name>Cmaj7</name>
</Harmony>
```

HAMON's extractor looks for both <text> and <name>, and when both are present it takes <text>.

What <text> contains

MuseScore stores the chord symbol **exactly as typed**, Unicode glyphs and all. There is no separate semantic encoding: CΔ7, CM7, and Cmaj7 are each kept verbatim and rendered as typed. That makes MSCX the most faithful of the supported formats for chord-symbol surface fidelity — HAMON just parses whatever sits in <text>.

Extended example

```
<Measure>
  <voice>
    <Harmony>
      <text>Bø7</text>
    </Harmony>
    <Chord>
      <Note><pitch>71</pitch></Note>
    </Chord>
```

```

<Harmony>
  <text>V7</text>
</Harmony>
<Chord>
  <Note><pitch>67</pitch></Note>
</Chord>
</voice>
</Measure>

```

Roman numerals, Nashville, figured bass

MuseScore 4 added a dedicated Roman-numeral analysis input mode. The XML it produces still stores the label as plain text in `<Harmony><text>`, so HAMON parses V7, ii, 6-5, and the rest exactly as it would from any other source.

Mapping: JSON fields → MuseScore MSCX

Every mapping below is stated against the canonical JSON schema (`grammar/hamon-schema.json`). MSCX has no semantic structure for harmony: every label is a plain surface string in `<Harmony><text>`. There is no per-field encoding — HAMON writes the whole `HarmonyLabel.surface` and reads it straight back.

JSON field	HAMON → MSCX	MSCX → HAMON	Loss / note
surface (any kind)	<code><Harmony><text>CΔ7</text></Harmony></code>	<code><text><Harmony></code> surface	none — surface is stored verbatim
semantic (any kind)	not encoded	derived by parsing surface	no structured fields in MSCX
rendering.*	reflected in surface text	recovered by parsing surface	glyph variants preserved via surface

All kind values map the same way:

JSON kind	<text> content example
chordSymbol	CΔ7, Am7, Bb, Bø7
roman	V7, ii, bVII
figuredBass	6-5, 4-3
nashville	2m, b7
functional	T, D->T
noChord	N.C.
text	anything else

MSCX never declares the notation system, so HAMON's grammar infers it from the surface value.

Losses and ambiguities

- **No semantic layer:** MSCX keeps no semantic encoding next to the display text. A non-standard abbreviation or a typo therefore comes through as `textLabel` instead of the type you meant.
- **Formatting markup:** MuseScore 4 encodes styled text (superscripts, bold, and so on) as embedded XML fragments inside `<text>`, a subset of HTML. HAMON strips those tags and reads only the plain text.
- **<name> vs <text>:** tools that write MSCX programmatically may emit `<name>` (the MuseScore 3 convention) or `<text>` (MuseScore 4). HAMON's extractor handles both.
- **Time anchoring:** `<Harmony>` elements are children of `<voice>`, siblings of the `<Chord>` elements. HAMON reads labels in document order but does not yet keep beat position.

How HAMON reads MuseScore MSCX

The adapter (`hamonpy/hamonpy/adapters/formats.py`, `_extract_musescore`):

1. Scans for `<Harmony>` blocks with a regex matching the full element.
2. Inside each block, extracts the content of `<text>` first, then `<name>` as fallback.
3. Strips any embedded HTML/XML markup tags from the extracted string.
4. Returns deduplicated, non-empty strings.

Comparison: MSCX vs MusicXML export

Aspect	MSCX (<code><Harmony><text></code>)	MusicXML (<code><harmony><kind></code>)
Surface fidelity	Exact — <code>CΔ7</code> , <code>CM7</code> , <code>Cmaj7</code> preserved as typed	Partial — display in text attr; semantic in inner text
Semantic encoding	None	Yes (major-seventh, minor, ...)
Roman numerals	Plain text	Not standard
Extension/alteration details	Plain text	<code><degree></code> children

Examples from the HAMON fixture corpus

Fixture	<code><text></code> content	HAMON surface	HAMON kind
<code>C_major</code>	<code>C</code>	<code>C</code>	<code>chordSymbol</code>
<code>Bb_major</code>	<code>Bb</code>	<code>Bb</code>	<code>chordSymbol</code>
<code>C_delta7</code>	<code>CΔ7</code>	<code>CΔ7</code>	<code>chordSymbol</code>
<code>B_hdim7</code>	<code>Bø7</code>	<code>Bø7</code>	<code>chordSymbol</code>
<code>C_dim</code>	<code>C°</code>	<code>C°</code>	<code>chordSymbol</code>
<code>rn_V7</code>	<code>V7</code>	<code>V7</code>	<code>roman</code>
<code>rn_ii</code>	<code>ii</code>	<code>ii</code>	<code>roman</code>
<code>fb_6_5</code>	<code>6-5</code>	<code>6-5</code>	<code>figuredBass</code>
<code>ns_2m</code>	<code>2m</code>	<code>2m</code>	<code>nashville</code>

Full fixture files: fixtures/chords/, fixtures/roman/, fixtures/figuredbass/, fixtures/nashville/.

MNX — Harmony Encoding

Overview

[MNX](#) (Music Notation XML) is a W3C Community Group draft aimed at eventually succeeding MusicXML. Despite the name, its primary serialization is JSON, not XML. As of 2025 the spec is still under active development, and no production software implements it in full.

We document MNX here for completeness — it is a format HAMON may need to support down the line. There are no HAMON fixtures or adapters for it yet.

Harmony in MNX

The MNX draft puts a harmony object inside measure content. It defines a structured semantic model much like MusicXML's, but it also adds provisions for Roman numeral analysis and figured bass that MusicXML has no native support for.

Chord symbol (draft schema)

```
{
  "type": "harmony",
  "value": {
    "root": { "step": "C", "alter": -1 },
    "kind": { "value": "minor-seventh" },
    "display": "Cm7"
  }
}
```

Key fields: - `root.step` — pitch class (A-G) - `root.alter` — semitone alteration (-1 = flat, 1 = sharp) - `kind.value` — semantic keyword (reuses MusicXML's vocabulary) - `display` — the string to render (equivalent to MusicXML's `text` attribute) - `bass` — optional slash bass, same structure as `root`

Roman numeral analysis (draft)

The MNX draft has a separate `roman-numeral` harmony type:

```
{
  "type": "harmony",
  "value": {
    "roman-numeral": {
      "degree": "V",
      "alter": 0,
      "mode": "major",

```

```

    "quality": "dominant-seventh"
  }
}

```

This is richer than anything in the formats HAMON supports today, and it maps to HAMON's roman kind more cleanly than MusicXML does.

Mapping: HAMON grammar → MNX (draft)

HAMON concept	MNX field
Root note	root.step
Accidental	root.alter
Quality (semantic)	kind.value
Display surface	display
Slash bass	bass.step + bass.alter
Roman numeral	roman-numeral.degree + roman-numeral.quality
Figured bass	not yet defined in draft
Nashville	not yet defined in draft
Functional	not yet defined in draft

Status in HAMON

- **No adapter:** hamonpy/ has no MNX support yet.
- **No fixtures:** there are no .mnx snippets in fixtures/.

Once the MNX spec stabilizes and software starts producing MNX files, HAMON will add an adapter. The `display` field lines up directly with HAMON's surface-first model, so chord symbols should slot in easily, and the structured `roman-numeral` type will make Roman numeral support straightforward.

References

- MNX specification draft: <https://w3c.github.io/mnx/>
- MNX discussion and examples: <https://github.com/w3c/mnx>

DCML Annotation Format

The DCML (Digital and Cognitive Musicology Lab) annotation format is a TSV-based Roman numeral scheme that Johannes Hentschel and colleagues built at EPFL. It underpins several large, hand-annotated corpora of tonal music — ABC, Beethoven, Brahms, Chopin, Debussy, and more.

Reference

Hentschel, J., Neuwirth, M., & Rohrmeier, M. (2021). The Annotated Beethoven Corpus (ABC): A dataset of harmonic analyses of all Beethoven string quartets. *Frontiers in Digital Humanities*, 8. <https://doi.org/10.3389/fdigh.2021.718509>

TSV Structure

Each file is tab-separated, one row per annotated beat position. The columns that matter for harmony are:

Column	Type	Example	Description
chord	string	V7/IV	Full chord symbol — the surface label (Roman numeral + form + figbass + optional slash secondary)
numeral	string	V	Roman numeral degree (uppercase = major, lowercase = minor context)
form	string	M, m, o, +, %	Quality modifier: M or empty = major, m = minor, o = diminished, + = augmented, % = half-diminished
figbass	string	7, 65, 43, 2	Figured bass inversion/quality: 7=seventh, 6=first-inv triad, 65/43/2=inverted sevenths
changes	string	(b7), (#5)	Chromatic or added tones in parentheses; may be absent
relativeroot	string	IV, V	Secondary target: the denominator after /, e.g. V/IV → relativeroot="IV"
localkey	string	I, vi, IV	Current local key as a Roman numeral relative to the global key
globalkey	string	C, a	Global key; uppercase = major, lowercase = minor

Column	Type	Example	Description
mc	int	4	Measure count (not always present)
onset	fraction	0	Beat onset within the measure

Form vocabulary

form value	Meaning
(empty)	Major triad (or as otherwise implied by numeral case)
M	Explicit major (often for major seventh: VM7)
m	Minor
o	Diminished
+	Augmented
%	Half-diminished

Figured bass vocabulary

figbass	Inversion / chord type
(empty)	Root position triad
6	First inversion triad
64	Second inversion triad
7	Root position seventh
65	First inversion seventh
43	Second inversion seventh
2	Third inversion seventh

Mapping to HAMON

DCML Roman numerals map cleanly onto HAMON's RomanSemantic:

DCML field	HAMON field	Notes
numeral	degree	Uppercase/lowercase preserved
form (M/m/o/+/%)	part of tail	Normalised: m7, °7, ø7, etc.
figbass	appended to tail	7→7, 65→65, etc.
changes	appended to tail (parenthesised)	Round-trip as a tail suffix
relativeroot	secondary + HarmonyAttributes.applied	The denominator after /; also drives a tonicization region (below)
globalkey + localkey	HamonSequence.regions (TonalRegion)	The analytical layer — see below

Key columns → tonal regions (v0.2.0)

Since v0.2.0 the importer maps DCML's key columns into the analytical layer ([analysis.md](#)):

- **globalkey** is a key name whose case gives the mode (C = C major, a = A minor, F#, eb). It's the reference point for resolving the Roman key columns to real pitches, and it never becomes a region of its own.
- **localkey** is a Roman numeral *relative to the global key* (case = mode). HAMON computes its absolute tonic in the global key's scale — mode-aware, so minor and modal globals spell correctly. The first localkey becomes a TonalRegion of kind key; each change after that becomes a modulation.
- **relativeroot** is a Roman numeral *relative to the local key*. A run of rows that share a non-emptyativeroot becomes a **nested** TonalRegion of kind tonicization (degree = theativeroot, parent = the local-key region), and every chord in that run also carries HarmonyAttributes.applied.target.

For example, global C, local I, chord V7/V withativeroot=V gives you a C-major key region plus a nested G-major tonicization region (degree: "V"), and the chord keeps both secondary: "V" and applied.target: "V".

Reconstruction of surface string

chord = numeral + form + figbass [+ changes] [+ "/" +ativeroot]

Example: numeral=V, form="", figbass=7,ativeroot=IV → chord=V7/IV.

Known limitations

- changes (chromatic alterations) round-trip as a raw tail suffix; HAMON doesn't model individual altered tones as first-class fields.
- Region **import** and **export** both work. On export, hamon_to_dcml_tsv adds localkey/globalkey columns whenever the sequence has regions: it uses the **home (first) key region as the globalkey** and expresses every region's localkey relative to it (tonicizations becomeativeroot). That keeps the absolute keys intact on a DCML→HAMON→DCML round-trip; the globalkey string matches the original only when the piece's home key is I.
- The *plain* dcml.py adapter ignores mc / onset (the time positions). For time-aligned tables, use the **expanded** adapter below, which fills HarmonyGroup.position from quarterbeats (or mc/mn_onset).

Expanded (time-aligned) tables — the Hentschel annotation standard & ms3

The DCML harmony annotation standard (Hentschel, Neuwirth & Rohrmeier) ships its corpora as *expanded* harmony tables — see [The Annotated Mozart Sonatas: Score, Harmony, and Cadence](#) and the *Annotated Corpus of Tonal Piano Music from the Long 19th Century* — and the ms3 MuseScore parser ([ms3.md](#)) emits the same shape. These tables carry extra columns that the plain adapter ignores:

Column	Example	Maps to
quarterbeats	129/2	HarmonyGroup.position.time (absolute quarter offset, as a fraction)
mc / mn + mn_onset	12 + 1/2	position.measure + position.beat (fallback when quarterbeats is absent)

Column	Example	Maps to
cadence	PAC, HC, IAC, DC, EC	<code>extract_cadences()</code> (side-channel — no AST node yet)
phraseend	{, }, }{	<code>extract_phrase_ends()</code> (side-channel)

hamonpy.adapters.dcml_expanded:

```
from hamonpy.adapters.dcml_expanded import (
    expanded_tsv_to_hamon, expanded_tsv_text_to_hamon,
    extract_cadences, extract_phrase_ends,
)

seq = expanded_tsv_to_hamon("sonata.harmonies.tsv") # chords + regions +
    ↪ positions
for g in seq.groups:
    print(g.primary[0].surface, g.position.time)      # time-aligned

cadences = extract_cadences(open("sonata.harmonies.tsv").read()) #
    ↪ [Cadence(type="PAC", position=...), ...]
phrases = extract_phrase_ends(open("sonata.harmonies.tsv").read())
```

The chord-surface logic and the localkey/relativeroot → region logic are **shared** with the plain `dcml.py` adapter; all the expanded adapter adds on top is positions and the cadence/phrase side-channels. HAMON's AST has no cadence or phrase node yet, so those come back alongside the sequence instead of being folded into groups — future work.

The CLI auto-detects expanded tables — a `.tsv` with a `quarterbeats` column — as `dcml_expanded`; to force it, run `hamon convert file.tsv -f dcml_expanded`.

Pitch arrays — the DiLeMMA training tables

[dilemmadata](#) (Hentschel) is the data infrastructure behind the AnalysisGNN multitask analysis models. It ships **pitch arrays**: TSV tables where *one row is one note* and the annotations are repeated on every note they cover. Harmony is not a list of labels here — it is a column, and its rhythm is a flag on the note grid.

Two dialects live in `pitch_arrays/`, and `hamonpy.adapters.dilemma` reads both:

Dialect	Files	Grouping column	Chord column	Keys
DLC (Distant Listening Corpus)	<code>pitch_arrays/DLC/</code>	<code>folded_phrase</code>	as found in hamonpy (plain DCML)	localkey / globalkey, as DCML degrees
AN (AugmentedNet)	<code>pitch_arrays/AN/</code>	<code>training_interval</code>	as found in hamonpy (plain DCML)	localkey / a_tonitizedKey, absolute and music21-spelled (B- = B flat)

The adapter **collapses** the note grid back to one row per harmony and hands the result to `dcml_expanded`, so chord surfaces, tonal regions and positions come out of exactly the same

code as for the corpora these arrays were derived from. For the AN dialect the absolute keys are converted to DCML degrees first (the first `a_localKey` is taken as the piece's `globalKey`), and AugmentedNet's Cad — the cadential six-four — is written as DCML's `V(64)`, which HAMON parses.

```
from hamonpy.adapters.dilemma import pitch_array_to_hamon, extract_spans
```

```
seq =  
    ↪ pitch_array_to_hamon("pitch_arrays/DLC/chopin_mazurkas/BI115-1op33-1.tsv")  
len(seq.groups)           # 90 harmonies, not 509 notes  
spans = extract_spans(open(path).read()) # index-aligned with seq.groups
```

What the pitch arrays add over the expanded tables is **harmonic rhythm**: AN states each harmony's duration outright (`a_duration`), and in DLC it follows from consecutive onsets. HAMON's `Position` is onset-only, so `extract_spans()` returns them on the side — a `HarmonySpan(position, duration, source)` per group, where `source` is "explicit" (the corpus said so) or "derived" (computed from the next onset), and `duration` is `None` for the last DLC harmony, whose end the array never states. Same treatment as cadences and phrase ends: reported honestly, not folded into the AST.

The CLI auto-detects pitch arrays — a `.tsv` with an `unfolded_harmony_index` or `a_annotationNumber` column — as `dilemma`, ahead of the `quarterbeats` sniff (DLC arrays have a `quarterbeats_playthrough` column). The `corpora/` directory of that repo holds git submodules with SSH URLs; the pitch arrays do not need them. The companion `*_slices.tsv` files carry no harmony.

Corpus availability

The public DCML-annotated corpora live on GitHub under the DCMLab organisation: - [DCMLab/ABC](#) — Annotated Beethoven Corpus - [DCMLab/mozart_piano_sonatas](#) - [DCMLab/chopin_mazurkas](#)

Harte Notation

Harte notation writes chord symbols as compact text, and it was built for automatic chord recognition (ACR) research. Christopher Harte and colleagues at Queen Mary, University of London proposed it, and it stuck: today it is the de-facto standard for MIREX chord evaluation and the Million Song Dataset derivatives.

Reference

Harte, C., Sandler, M., Abdallah, S., & Gómez, E. (2005). Symbolic representation of musical chords: A proposed syntax for text annotations. *Proceedings of the 6th International Society for Music Information Retrieval (ISMIR) Conference*, 66–71.

Syntax

```
chord = root ":" shorthand ["/" bass_interval]
      | "N"           (no chord)
      | "X"           (unknown / don't care)

root = note_name [accidental]
note_name = "A" | "B" | "C" | "D" | "E" | "F" | "G"
accidental = "b" | "#" | "bb" | "##"

shorthand = "maj" | "min" | "dim" | "aug"
           | "maj7" | "min7" | "7" | "dim7" | "hdim7" | "minmaj7"
           | "maj6" | "min6" | "9" | "maj9" | "min9"
           | "sus2" | "sus4"
           | "1"           (root only)
           | "5"           (power chord)
           | (*7)          (extended notation with degree list)

bass_interval = ["b" | "#" | "bb" | "##"] degree_number
```

Examples

Harte token	Meaning
C:maj	C major triad
C:min	C minor triad
G:7	G dominant seventh
Ab:maj7	A-flat major seventh
D:min7	D minor seventh
B:hdim7	B half-diminished seventh

Harte token	Meaning
E:dim7	E diminished seventh
F:sus4	F suspended fourth
C:maj/3	C major, first inversion (E bass)
N	No chord

Shorthand vocabulary

Harte shorthand	Quality	Seventh	HAMON equivalent
maj	major	—	(no suffix)
min	minor	—	m
dim	diminished	—	°
aug	augmented	—	+
maj7	major	maj7	maj7
min7	minor	min7	m7
7	major	dom7	7
dim7	diminished	dim7	°7
hdim7	half-dim	hdim7	ø7
minmaj7	minor	maj7	mMaj7
sus4	major+sus4	—	sus4
sus2	major+sus2	—	sus2
maj6	major	—	6
min6	minor	—	m6
9	major	dom7+9	9
maj9	major	maj7+9	maj9
min9	minor	min7+9	m9
1	root only	—	(omit)
5	power chord	—	5

Mapping to HAMON

Harte tokens map to HAMON ChordSymbolSemantic:

Harte field	HAMON field	Notes
root note name	root.note	Uppercase
root accidental	root.accidental	# or b
shorthand	quality + seventh	See table above
/bass_interval	bass	Harte names the bass as an altered degree above the root (/b7), HAMON as a pitch class. The two convert both ways — the degree fixes the letter, the accidental fixes the pitch — so this is a mapping, not a guess.

On import, HAMON drops the `:` separator and rewrites the shorthands: `min` becomes `m`, `dim` becomes `°`, and so on.

Known limitations

- Extended degree lists like `(*3,b7)` have no first-class fields in HAMON; they land in `tail`.
- A **compound** bass degree returns as its simple equivalent (`/9` → `/2`): the two name the same pitch, and HAMON's bass is a pitch class, which does not record the octave that told them apart. A re-spelling, not a loss — `report.py` finds nothing, because the semantics are identical.
- `X` (unknown/don't care) has no HAMON equivalent, so HAMON skips it on import.
- `minmaj7` maps to a `mMaj7` tail in HAMON, and that round-trips losslessly. (Until 2026-09-01 it did not: the letter-led spellings `mmaj7`/`mM7` normalized to a **dominant** seventh, because the chord-tail tokenizer swallowed `mmaj` as one word and left a bare `7`. Every spelling is now pinned in `conformance/corpus.txt`.)

File format

Harte tokens sit one per line in plain-text `.txt` or `.lab` files, usually with timestamps:

```
0.000    0.500    N
0.500    2.000    C:maj
2.000    4.000    G:7
4.000    6.000    F:maj
```

The adapter reads both layouts: bare tokens (one chord per line) and the tab-separated timestamp form.

Since v0.5 the timestamps are **not discarded**. Both columns are read, in the clock the file states them in:

- the start seconds become the group's `Position.seconds`, written `s:0.5` in the surface;
- end - start becomes the label's extent, written `[dur:1.5s]` — the `s` suffix is what says it is seconds and not quarter notes.

So the example above imports as `s:0,N[dur:0.5s] / s:0.5,C[dur:1.5s] / s:2,G7[dur:2s] / s:4,F[dur:2s]`. A bare token line states no time, so it gets no position and no extent — nothing is inferred from line order, and the seconds are never converted into a metric `m:/ts`: position or into quarters, which would need a tempo map the file does not carry. See [positions.md](#).

The **writer** emits the same two forms, and the choice is all-or-nothing for the file: it writes `start end label` when **every** harmony states both an `s`: onset and a `[dur:...s]` extent, and bare tokens otherwise. So a `.lab` imported into HAMON comes back out byte for byte, which is what the physical clock was for — this round-trip used to lose 100% of the file's positional content.

Two things it will not do:

- **Invent an end.** A harmony with an onset but no extent leaves a column this writer has no honest value for; deriving it from the next onset is precisely what the Dezzrann writer was cured of, because the invented value comes back on re-import looking like source data.
- **Mix the two forms.** A `.lab` is a positional-column format, so a file with some timed lines and some bare ones is not one. A single untimed harmony drops the whole file to bare tokens, and the seconds the other groups had are lost — a real loss, which `report.py` reports as such rather than papering over.

iReal Pro Format

iReal Pro is a commercial app that keeps chord charts in a proprietary, URL-encoded format. You share a chart as an `irealb://` URI, and decoding it yields chord symbols in a compact ASCII notation.

Reference

- iReal Pro app: <https://irealpro.com>
- Community decoder: <https://github.com/pianosnake/ireal-reader>
- Format documentation: unofficial, reverse-engineered

URI Structure

`irealb://<title>=<composer>=<style>=<key>=<time-sig>=<encoded-chart>=<bpm>`

Multiple songs are separated by `===`.

The `<encoded-chart>` is first obfuscated with a character-rotation cipher (applied to 50-character substrings), then percent-encoded.

Chord notation inside a decoded chart

Once decoded, each chord event is a combination of:

- **Root note:** A-G, optionally followed by # or b
- **Quality suffix:** standard short notation (see table below)
- **Separator space** between successive chords

Quality suffixes

iReal suffix	Meaning	HAMON equivalent
(none)	Major triad	(no suffix)
- or m	Minor triad	m
^7	Major seventh	maj7
7	Dominant seventh	7
-7 or m7	Minor seventh	m7
^	Major (alt. glyph)	(no suffix)
dim or o	Diminished	°
dim7 or o7	Diminished seventh	°7
h7 or ø7	Half-diminished seventh	ø7
aug or +	Augmented	+
sus or sus4	Suspended fourth	sus4

iReal suffix	Meaning	HAMON equivalent
2 or sus2	Suspended second	sus2
-^7 or m^7	Minor-major seventh	mmaj7
add9	Add ninth	add9
6	Major sixth	6
-6 or m6	Minor sixth	m6
69	Six-nine	69
9	Dominant ninth	9
-9 or m9	Minor ninth	m9
^9	Major ninth	maj9
13	Dominant thirteenth	13
alt	Altered dominant	alt (tail)

Special symbols

iReal symbol	Meaning
n or N	No chord
w	Whole rest
p	Repeat previous bar
x or r	Repeat last two bars
f	Fermata (ignore)
XyQ	Start/end repeat markers

Slash bass notation

G/B means G major over a B bass. The slash and bass note come after the quality suffix.

Mapping to HAMON

iReal chord symbols map to HAMON ChordSymbolSemantic. HAMON pulls out only the harmony content; it does not model repeat signs, fermatas, or bar-structure markers.

iReal field	HAMON field
Root note + accidental	root
Quality suffix	quality + seventh + suspensions
Slash bass	bass
n/N	NoChordSemantic

Known limitations

- The obfuscation cipher runs per 50-character substring; the adapter implements the standard rotation scheme.
- HAMON does not model bar structure (repeat signs, section markers).
- The alt (altered dominant) suffix goes into a tail string; HAMON populates no first-class alteration fields for it.
- iReal sometimes writes ^7 (the hat glyph) for a major seventh; the adapter normalizes it to maj7.

Dezrann (‘.dez’) Integration

[Dezrann](#) (Algomus; Foscarin, Giraud, Rigaux et al.) is an open web platform for browsing annotated music corpora on a single timeline — it lines up scores, audio, and analytical labels so you can move through them together. For the details, see the TISMIR paper: *Dezrann: A Web Platform for Unified Access to Annotated Music Corpora* (<https://doi.org/10.5334/tismir.212>).

Dezrann is **not** a harmony standard. By design it stays *agnostic* about the analytical ontology: a label is just a position and a free tag. HAMON adds the normalisation Dezrann leaves out — it parses the harmony tags into the typed AST and keeps Dezrann’s quarter-note anchor as a `HarmonyGroup.position`.

The .dez format

A .dez file is JSON: a required `labels` array plus an optional meta block. Every label is anchored on **musical time, measured in quarter notes from the start of the piece**:

Field	Required	Description
<code>start</code>	☐	Onset in quarter notes (the only mandatory field)
<code>duration</code>		Inter-onset interval, in quarters
<code>type</code>		Annotation family (Harmony, Chord, Cadence, Tonality, ...)
<code>tag</code>		Free detail — for harmony labels, the chord/Roman surface
<code>staff, comment, layers</code>		Staff, free commentary, annotation sources

```
{
  "labels": [
    { "type": "Harmony", "start": 0, "duration": 4, "tag": "Cmaj7" },
    { "type": "Harmony", "start": 4, "tag": "G7" },
    { "type": "Cadence", "start": 12, "tag": "C:PAC" }
  ],
  "meta": { "producer": "hamonpy" }
}
```

Mapping to HAMON

Dezrann	HAMON
label tag (for harmony types)	parsed HarmonyLabel (chord symbol, roman, ...)
label start (quarters)	HarmonyGroup.position.time (absolute fraction)
type $\in$ {Harmony, Chord, Chords}	included by default (configurable)
non-harmony types (Cadence, ...)	skipped on import
unparseable tag	TextSemantic fallback (lossless)

API (`hamonpy.adapters.dezrann`)

```
from hamonpy.adapters.dezrann import dez_to_hamon, hamon_to_dez
```

```
# Dezrann → hamon (system="auto" detects cs/rn/...; pass "rn"/"cs" to force one)
seq = dez_to_hamon(open("piece.dez").read(), system="auto")
```

```
# hamon → Dezrann
dez_json = hamon_to_dez(seq, label_type="Harmony")
```

`dez_to_hamon` takes a parsed dict, JSON text, or a .dez file path (`dez_file_to_hamon`). It emits labels in start order. On export, duration is the gap to the next group's start, whenever both positions are known.

CLI

`hamon convert piece.dez` recognises the .dez extension and writes canonical HAMON JSON. See [cli.md](#).

Limitations

- Only harmony-bearing label types come across; cadence, phrase, and structure labels are out of scope, since HAMON has no AST node for them yet.
- HAMON does not redistribute Dezrann corpora. Get them from the platform or their authors (see [datasets.md](#)).

JAMS

JAMS (JSON Annotated Music Specification) is a JSON container for time-aligned MIR annotations — chords, keys, beats, segments, melody, tags. Each annotation carries rich meta-data: who annotated it, which corpus and data source it came from, whether it was validated, and a confidence score. It is a common way to distribute audio-derived annotations, chord-recognition corpora in particular.

Reference

Humphrey, E. J., Salamon, J., Nieto, O., Forsyth, J., Bittner, R. M., & Bello, J. P. (2014). JAMS: A JSON Annotated Music Specification for reproducible MIR research. *Proceedings of the 15th ISMIR Conference*, 591–596. Docs: https://jams.readthedocs.io/en/stable/jams_structure.html

Structure

```
JAMS
├─ file_metadata      { title, artist, release, duration (s), identifiers,
  ↳ jams_version }
├─ annotations[]
│   ├── namespace      e.g. "chord", "chord_harte", "key_mode", "beat",
  ↳ "segment_open"
│   ├── data[]          { time (s), duration (s), value, confidence }
│   ├── annotation_metadata { annotator, corpus, data_source, curator,
  ↳ validation, ... }
│   └─ sandbox          arbitrary payload
└─ sandbox
```

For **harmony**, the relevant namespaces are:

- **chord / chord_harte** — the value is a **Harte** chord label ("C:maj7", "D:min7/b3", "N").
- **key_mode** — the value is "TONIC:mode" ("C:major", "Bb:minor").

Observation time/duration are in **audio seconds** — HAMON's s: clock since v0.5.

Mapping to/from HAMON (hamonpy/hamonpy/adapters/jams.py)

JAMS	HAMON
chord / chord_harte observation value	a chord-symbol HarmonyLabel (parsed via the Harte adapter)
observation order (by time)	group order
key_mode value	a tonal region (@key:)

JAMS	HAMON
observation time (audio seconds)	Position.seconds — the s: clock (v0.5); written back on export
observation duration (audio seconds)	attributes.durationSeconds, surface [dur:1.5s] — the extent in the same clock (positions.md); written back too
annotation_metadata, confidence	not modelled yet — see the planned provenance layer

- `jams_to_hamon(text_or_dict)` — reads the first chord/chord_harte annotation into ordered groups and the first key_mode into a region.
- `hamon_to_jams(seq)` — emits a chord annotation (Harte values) and, when the sequence has regions, a key_mode annotation; one observation per group. A group's stated s: seconds become its time and a stated [dur:...s] its duration. **What the source did not state is written as null, never as a number.** This writer used to fall back to the group's index for time and to 1 for duration — which a JAMS consumer reads as real audio timestamps, so a sequence positioned in bars came out claiming its harmonies land one second apart. That is the invention the Dezzrann writer was cured of, and `report.py` was duly flagging every one of them as added.

The cost, taken deliberately: a HAMON sequence with **no** audio clock now yields a document that is *not* schema-valid, since JAMS requires a numeric time. The format has no way to say “unknown”, and saying nothing beats saying something false.

The adapter is **dependency-free**. It reads and writes the JAMS JSON directly, so you do not need the jams Python package.

What JAMS carries natively (the loss picture)

Standard JAMS harmony is just chord symbols plus key. So, like Harte, it natively **loses** Roman numerals, figured bass, functional (T-S-D), Nashville, chord-scales, applied-chain detail, and **metric positions** — its times are audio seconds. Those seconds are no longer a loss in the other direction: since v0.5 they are HAMON's s: clock, and JAMS is one of the two targets that carry it natively. What sets it apart from Harte is the first-class **sandbox**: the full HAMON reading can ride along there **out-of-band**, with no loss. That is exactly the distinction the ICCCM'26 loss viewer draws for the JAMS target (native vs. native + HAMON).

Provenance

JAMS's annotation_metadata (annotator, corpus, data source, validation) and per-observation confidence line up closely with HAMON's planned **provenance / traceability layer**. Once that layer lands, JAMS becomes a natural carrier for import/export provenance in both directions.

See also: [Harte](#) (the chord syntax JAMS uses), [positions](#) (metric vs audio time), [Dezzrann](#) (another time-aligned label JSON).

music21 Integration

hamonpy integrates with [music21](#), MIT's Python toolkit for computer-aided musicology. The integration has two layers:

1. **Adapter functions** (`music21_adapter.py`) — convert directly between a `HamonSequence` and a `music21 Stream` of native `ChordSymbol` / `RomanNumeral` / `NoChord` objects.
2. **Native I/O format** (`music21_converter.py`) — register `.hamon` as a `music21` format, so `converter.parse(..., format="hamon")` and `stream.write("hamon")` behave like any built-in format.

Installation

`music21` is an optional dependency:

```
pip install hamonpy[music21]
```

Or install manually:

```
pip install "music21>=9"
```

Quick start

music21 → HAMON

```
import music21.corpus
from hamonpy.adapters.music21_adapter import music21_stream_to_hamon

# Load a score from the music21 corpus
score = music21.corpus.parse("bach/bwv66.6")

# Extract harmony labels
seq = music21_stream_to_hamon(score)
for group in seq.groups:
    label = group.primary[0]
    print(label.surface, "→", label.semantic)
```

HAMON → music21

```
from hamonpy.parse import parse_hamon_sequence
from hamonpy.adapters.music21_adapter import hamon_to_music21_stream

seq = parse_hamon_sequence("@cs\nC\nAm\nF\nG7")
part = hamon_to_music21_stream(seq)
part.show() # or part.write("musicxml", fp="output.xml")
```

.hamon as a native music21 format

Register it once, then use music21's own converter:

```
from hamonpy.adapters.music21_converter import register_hamon_format
register_hamon_format() # idempotent

from music21 import converter
s = converter.parse("@cs\nC\nF\nG7", format="hamon") # text → Stream
s = converter.parse("song.hamon") # by file extension
s.write("hamon", fp="out.hamon") # Stream → .hamon
```

Parsing goes through `hamon_to_music21_stream` (Roman numerals resolve against the sequence's `@key: regions`); writing goes through `music21_stream_to_hamon + sequence_to_hamon_text`. To override the emitted `@<system>` hint, pass `sequenceSystemHint="rn"` (or similar) as a keyword to write.

Key context for roman numerals

Roman numerals resolve to concrete pitches against the prevailing key, which HAMON derives from the sequence's analytical `@key: regions` (or a label's `local_key`):

```
seq = parse_hamon_sequence("@rn\n@key:G\nV7/V\nV")
part = hamon_to_music21_stream(seq)
# V7/V in G → A7 (A C# E G); V → D major (D F# A)
```

Tonicizations live in the roman secondary (e.g. V/V), not in the prevailing key, so they map onto music21's secondaryRomanNumeral figure form.

API

music21_stream_to_hamon(stream, sequence_system_hint="cs") → HamonSequence

Converts a `music21.stream.Stream` to a `HamonSequence`.

It walks `stream.flatten().getElementsByClass(['ChordSymbol', 'RomanNumeral', 'NoChord'])` and converts each element:

music21 class	HAMON semantic
harmony.ChordSymbol	ChordSymbolSemantic
roman.RomanNumeral	RomanSemantic
harmony.NoChord	NoChordSemantic

Parameters: - `stream`: Any `music21.stream.Stream` (Score, Part, flat stream, etc.) - `sequence_system_hint`: Override the HAMON system hint ("cs", "rn", etc.)

Returns: `HamonSequence` with one `HarmonyGroup` per harmony event.

hamon_to_music21_stream(seq) → music21.stream.Part

Converts a `HamonSequence` to a `music21.stream.Part`.

HAMON semantic	music21 class
ChordSymbolSemantic	harmony.ChordSymbol
RomanSemantic	roman.RomanNumeral

HAMON semantic	music21 class
NoChordSemantic	harmony.NoChord
NashvilleSemantic, FiguredBassSemantic, etc.	skipped

Time placement. When the groups carry `HarmonyGroup.position` (schema 0.2.1):

- if every group has a measure, elements are inserted into `music21.stream.Measure` objects at offset `beat - 1`, so `measureNumber` and `beat` round-trip;
- else if every group has an absolute time, elements are inserted at `time × 4` quarter-note offsets;
- otherwise elements fall back to sequential offsets (1 quarter note apart).

Going the other way, `music21_stream_to_hamon` reads each element's measure context back into `position` — measure from `measureNumber`, beat from the meter or the measure-relative offset. So a `MusicXML → HAMON → music21 → HAMON` trip preserves `measure+beat`.

Chord symbols export through a `music21` **chord figure** built from the semantic fields, which preserves natively: slash bass (`G7/B`), suspensions (`Csus4`), added tones (`Cadd9`), and alterations (`Dm7b5` → lowered fifth, `C7b9`). Roman numerals keep the secondary target (`V/IV`), front alterations (`bVI`), and inversion figures from tail (`V65`, `ii43`), and they resolve against the `region/local_key` key when one is available.

Native I/O format API (`music21_converter.py`)

Symbol	Description
<code>register_hamon_format()</code>	Registers the <code>HamonConverter</code> <code>SubConverter</code> with <code>music21</code> 's global converter (idempotent). Returns the converter class.
<code>HamonConverter</code>	<code>music21 SubConverter</code> subclass: <code>registerFormats = ("hamon",)</code> , input/output extension <code>.hamon</code> . <code>parseData</code> → <code>Stream</code> ; <code>write</code> → <code>.hamon text</code> .

Mapping table

ChordSymbol quality mapping

HAMON quality + seventh	music21 chordKind
major (no seventh)	"major"
minor (no seventh)	"minor"
augmented	"augmented"
diminished	"diminished"
major + maj7	"major-seventh"
minor + min7	"minor-seventh"
major + dom7	"dominant"
diminished + dim7	"diminished-seventh"
half-diminished + hdim7	"half-diminished"
minor + maj7	"minor-major-seventh"

Accidental mapping

HAMON accidental	music21 pitch modifier
flat	-
sharp	#
doubleFlat	--
doubleSharp	##

Known limitations

- **Time positions:** now carried via `HarmonyGroup.position` (schema 0.2.1) — see *Time placement* above. Groups without a position fall back to sequential offsets. The measure→beat mapping assumes 1 quarter = 1 beat (it infers no time signature), so compound and irregular meters may not land precisely.
- **omits:** the exported chord figure does not yet reflect the `ChordSymbolSemantic.omits` field (it does carry extensions, alterations, adds, bass, and suspensions).
- **Nashville / functional:** these HAMON semantics have no direct music21 counterpart, so export skips them.
- **Figured bass:** music21 has its own figured-bass notation, but this adapter does not bridge to it, so `FiguredBassSemantic` labels are skipped on export.

ms3 Integration

ms3 (Johannes Hentschel, DCML/EPFL) is a Python library that parses MuseScore 3/4 files (.mscx / .mscz) without a lossy round-trip. It reads harmony labels written in the **DCML annotation standard** and hands them back as *expanded* harmony tables — pandas DataFrames.

Those tables are already in the expanded DCML shape, so the HAMON ms3 adapter just delegates to the **DCML expanded adapter**. Time alignment (quarterbeats), tonal regions, and cadence/phrase extraction all come along for free.

Installation

ms3 is an optional dependency:

```
pip install hamonpy[ms3]      # or: pip install ms3
```

API (hamonpy.adapters.ms3_adapter)

```
from hamonpy.adapters.ms3_adapter import (
    ms3_score_to_hamon, ms3_expanded_to_hamon,
    extract_cadences, extract_phrase_ends,
)
```

```
# 1. Straight from a MuseScore file (requires ms3 + MuseScore)
seq = ms3_score_to_hamon("sonata.mscx")
```

```
# 2. From an already-extracted expanded table (DataFrame, .to_csv object,
#      or a list of row dicts – handy for testing without ms3 installed)
```

```
import ms3
expanded = ms3.Score("sonata.mscx").mscx.expanded
seq = ms3_expanded_to_hamon(expanded)
```

```
for g in seq.groups:
    print(g.primary[0].surface, g.position.time)  # time-aligned
```

ms3_expanded_to_hamon accepts: - a **pandas DataFrame** (or anything exposing to_csv(sep=..., index=...)), - a **list of row dicts** (column order = keys of the first row).

Under the hood it renders the table to DCML TSV text and feeds that to expanded_tsv_text_to_hamon, so chords, regions, positions, cadences, and phrase ends are all available.

Mapping

The column → AST mapping is the same one used for the **DCML expanded tables**: chord/numeral/form/figbass/changes/relativeroot become a typed label, localkey/globalkey

become tonal regions, quarterbeats becomes `position.time`, and cadence/phraseend go to a side channel via `extract_cadences` / `extract_phrase_ends`.

Limitations

- `ms3_score_to_hamon` needs `ms3` (and `MuseScore` for `.mscz` conversion). Without `ms3` it raises `ImportError` with install guidance. The `DataFrame/dict` path needs neither.
- Only the expanded **harmonies** table is consumed. Note and measure tables are out of scope, because HAMON is a harmony-label layer.

partitura Integration

partitura (CP-JKU) is a Python package for symbolic music processing. Its score model has a `Harmony` base class with `RomanNumeral` and `ChordSymbol` subclasses. Each is a `TimedObject`, and you can recover its onset in quarters via `part.quarter_map(obj.start.t)`.

`hamonpy` bridges `partitura`'s harmony objects to and from the typed AST, carrying onsets through as time-aligned `HarmonyGroup.position` values.

Installation

`partitura` is an optional dependency:

```
pip install hamonpy[partitura]    # or: pip install partitura
```

API (`hamonpy.adapters.partitura_adapter`)

```
import partitura as pt
from hamonpy.adapters.partitura_adapter import (
    partitura_part_to_hamon, hamon_to_partitura_part,
)

# partitura → hamon
score = pt.load_score("piece.musicxml")
part = score.parts[0]
seq = partitura_part_to_hamon(part, system="auto")    # "rn"/"cs" forces a
↳ system
for g in seq.groups:
    print(g.primary[0].surface, g.position.time)

# hamon → partitura
part = hamon_to_partitura_part(seq)
```

Mapping

partitura	HAMON
<code>Harmony.text</code> (or <code>ChordSymbol.root/kind/bass</code>)	<code>parsed HarmonyLabel</code> surface
<code>obj.start.t</code> → <code>part.quarter_map(...)</code>	<code>HarmonyGroup.position.time</code> (quarters)
<code>RomanNumeral</code>	<code>RomanSemantic</code> (system <code>rn</code>)
<code>ChordSymbol</code>	<code>ChordSymbolSemantic</code> (system <code>cs</code>)
unparseable text	<code>TextSemantic</code> fallback

On export (`hamon_to_partitura_part`), Roman-numeral labels become `RomanNumeral` objects and everything else becomes a plain `Harmony` — the surface text round-trips either way. Onsets use `divs_per_quarter` (default 4).

Limitations

- Only harmony objects are read and written. `partitura`'s full note/score model is out of scope, since HAMON is a harmony-label layer.
- `ChordSymbol` export uses the base `Harmony(text=...)` instead of reconstructing `partitura`'s `root/kind` fields. The surface survives; the structured fields do not.

FlexOHR

FlexOHR — *Flexible and Extensible Object for Harmony Representation* — is DCMLab’s harmony object model (`pip install flexohr`). It represents, stores, and queries harmony as an immutable recursive object (OHR) with a pitch-space paradigm and a DCML codec. HAMON bridges to it so a HAMON label can be handed to FlexOHR’s model — and back — with **HAMON as the wire format** between the encodings and whatever analysis runs on the FlexOHR side.

HAMON is not an analysis library; FlexOHR is the representation model the analysis side uses. The adapter keeps the two interoperating without either adopting the other’s model.

Install

```
pip install -e "./hamonpy[flexohr]"
```

Usage

```
from hamonpy.parse import parse_hamon_sequence
from hamonpy.adapters.flexohr import (
    hamon_to_flexohr,          # HamonSequence -> list[flexohr.OHR]
    flexohr_to_hamon,          # list[flexohr.OHR] -> HamonSequence
    hamon_semantic_to_ohr,      # one chordSymbol/roman semantic -> OHR
    ohr_to_hamon_semantic,      # one OHR -> semantic
)

seq = parse_hamon_sequence("@cs\nCmaj7\nAm7\nDm7\nG7")
ohrs = hamon_to_flexohr(seq)          # -> [OHR, OHR, OHR, OHR]
back = flexohr_to_hamon(ohrs)         # -> HamonSequence

# Roman numerals need a tonal context (default C major, overridable):
from hamonpy.parse import parse_hamon_sequence
v7 = parse_hamon_sequence("@rn\nV7").groups[0].primary[0].semantic
ohr = hamon_semantic_to_ohr(v7, key="C") # OHR rooted on the C-major scale
```

Mapping

HAMON	FlexOHR
ChordSymbolSemantic.root (PitchClass)	SPC (specific pitch class, fifths-based)
quality + seventh	ChordQuality (e.g. major+maj7 → major_seventh, half-diminished+hdim7 → half_diminished_seventh)

HAMON	FlexOHR
suspensions [4] / [2]	ChordQuality.suspended_fourth / suspended_second
bass (slash bass)	resolved to Inversion via OHR.with_(bass=...)
RomanSemantic.degree (+ prefixAccidentals)	SD (scale degree) in a key Scale, via the DCML codec
roman tail (figbass + quality symbol)	ChordQuality + Inversion
key context	Scale.from_collection_type(major/natural_minor, tonic)

Chord symbols round-trip **root, quality + seventh, suspensions, and slash bass** losslessly. Roman numerals map **degree (+ accidentals), triad/seventh quality, and the figured-bass inversion** in a key context.

Limitations (first cut)

- Chord-symbol **extensions / alterations / adds / omits** are not yet carried into FlexOHR (root + quality + seventh + suspensions + bass are).
- Roman **applied/secondary chains** (V/V), **pedal**, and parenthesised **changes** (V(4)) are out of scope for now — see the our DCML audit, which tracks the same gaps in the Roman-numeral AST.
- Only chord symbols and Roman numerals are supported; Nashville, figured bass, and functional labels are not mapped (FlexOHR has no dedicated vocabulary for them).
- FlexOHR emits DCML strings via `ohr.to_format("dcml")`; when building an OHR for a round-trippable string, pass `inversion=` explicitly (the adapter always does).

HAMON — Analytical Layer (v0.2.0)

Up to v0.1.0, HAMON only described **harmony labels**: a chord symbol, a Roman numeral, a Nashville number, a figured bass, a functional token, a no-chord, or raw text. That tells you *what the chords are*, but it doesn't capture a *harmonic analysis* of a piece.

The **analytical layer** (v0.2.0) fills that gap. It adds the structures a harmonic-analysis project needs without breaking anything: every field introduced here is **optional**, so a 0.1.0 document is still a valid 0.2.0 document. The canonical model lives in [grammar/hamon-schema.json](#); the optional surface syntax is defined in [grammar/hamon.ebnf](#).

What follows is the normative description of the layer, organized by analytical capability.

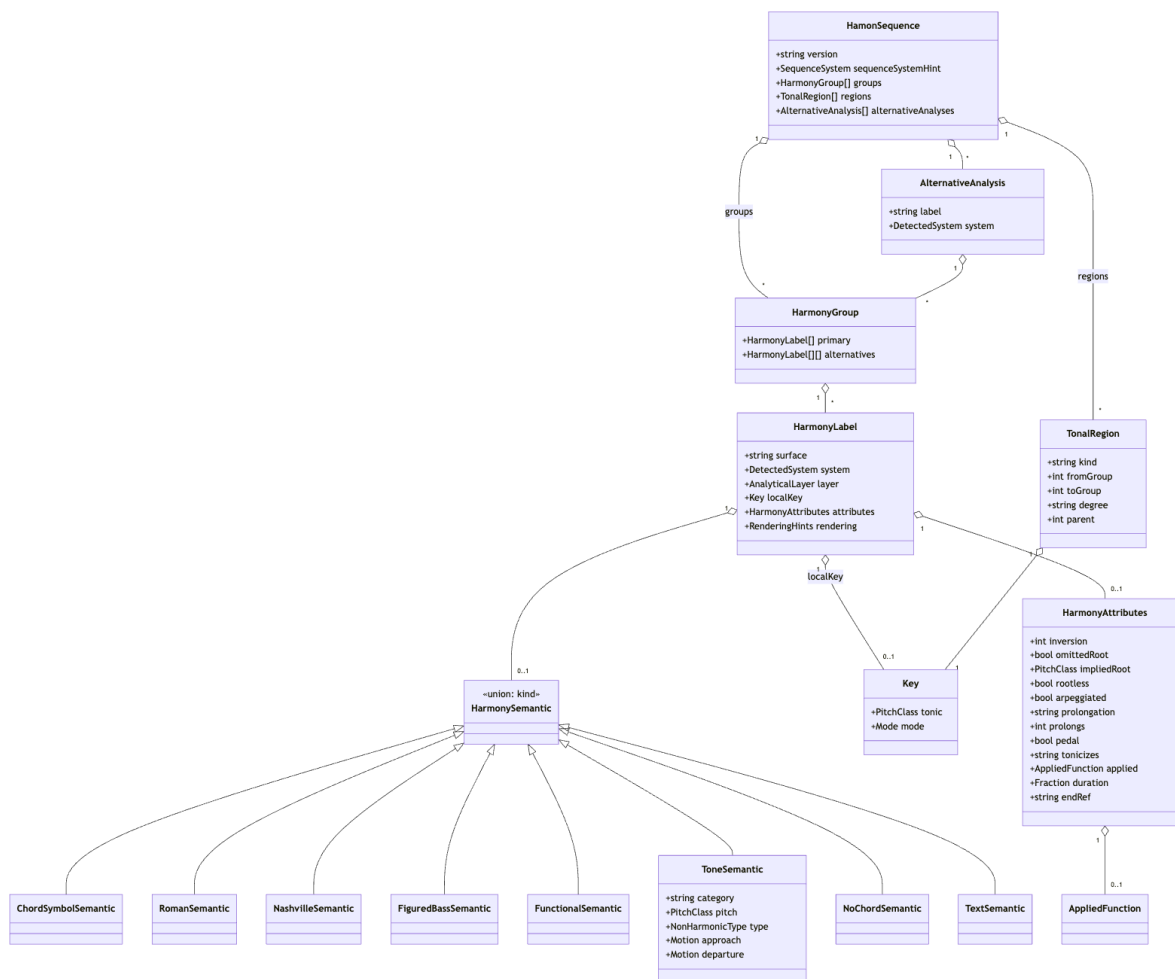
0. Design principles

1. **The JSON model is the hub.** Some analyses don't fit on a single one-line surface string — a full multi-voice non-harmonic-tone reading, or two analysts' competing progressions. For those, the JSON structure is canonical; the surface syntax just covers the common inline cases.
2. **Additive and optional.** Analytical fields never change what an existing label means. A reader that ignores them still recovers the 0.1.0 reading.
3. **Layered, not flattened.** One time point can carry several *co-temporal* readings of the *same* sonority — a degree, a function, a bass figure, a key — each tagged with the analytical layer it expresses. Don't confuse this with *alternatives* (rival readings) or *simultaneity* (two chords at once).
4. **Spans are explicit.** Tonal regions, tonicizations, and alternative progressions all reference group indices, so an analysis scopes exactly onto the stretch of sequence it annotates.

Concept	Same sonority?	Same time point?	Construct
Layers	yes	yes	HarmonyLabel.layer within one group's primary
Simultaneity	no	yes	several labels in one group's primary
Alternatives (point)	—	yes	HarmonyGroup.alternatives
Alternatives (progression)	—	a span	HamonSequence.alternativeAnaly
Regions	—	a span	HamonSequence.regions

Model overview (UML)

Here is the canonical JSON model (grammar/hamon-schema.json) with the v0.2.0 analytical additions. HarmonySemantic is a discriminated union keyed on the kind field; the analytical types hang off HamonSequence and HarmonyLabel.



1. Alternative harmonies — individual chords *and* whole progressions

1.1 Point alternatives (already in v0.1.0)

Two readings of a single time point go in the group-level alternatives. On the surface you separate them with `|`; in JSON, each alternative is a list of labels:

```
@rn
V7 | bVII7      # primary reading V7, alternative reading bVII7 at the same
  ↪ beat

{ "primary":      [ { "surface": "V7",    ... } ],
  "alternatives": [ [ { "surface": "bVII7", ... } ] ] }
```

1.2 Progression alternatives (new in v0.2.0)

A rival reading of a *whole span* — two analysts disagreeing, or a chord-symbol reading set against a functional one — is a `HamonSequence.alternativeAnalyses[]` entry. Each one carries an optional label, the system it's written in, the scope it re-analyses (a group index range; leave it out to mean the whole sequence), and its own groups.

```
{
  "groups": [ /* primary reading: Dm7 | G7 | Cmaj7 */ ],
  "alternativeAnalyses": [
    {
      "label": "functional",
      "system": "fun",
      "scope": { "fromGroup": 0, "toGroup": 2 },
      "groups": [ /* S | D | T */ ]
    }
  ]
}
```

2. Tonal regions (*regiones tonales*) and 3. tonicizations

A **tonal region** is a span of groups governed by one local key. You model it with `TonalRegion { key, kind, fromGroup, toGroup?, degree?, parent?, label? }` on `HamonSequence.regions`.

`kind` tells the three cases apart:

- key / region — a stable tonal area;
- tonicization — a brief emphasis of a degree as a *local tonic* (a passing *región tonal*), usually set up by a secondary dominant;
- modulation — a confirmed change of key.

Regions can **nest**. A tonicization inside a region points back to its enclosing region through `parent` and names the tonicized scale degree in `degree`.

Surface

A `@key:` directive opens a region. It runs over the groups that follow until the next `@key:`, or until the sequence ends:

```
@rn
@key:C
I      vi      ii      V7      I
@key:G      # modulation to the dominant
ii      V7      I
```

A region defined relative to the prevailing key — a tonicization — takes a degree instead of an absolute pitch:

```
@rn
@key:C
I
@key:V      # tonicize the dominant: a passing G-major region
V7/V      V      # i.e. D7 → G, heard locally as V7 → I
@key:C
I
```

```

{
  "regions": [
    { "key": { "tonic": { "note": "C" }, "mode": "major" }, "kind": "key",
      ↪ "fromGroup": 0, "toGroup": 0 },
    { "key": { "tonic": { "note": "G" }, "mode": "major" }, "kind":
      ↪ "tonicization",
      "degree": "V", "parent": 0, "fromGroup": 1, "toGroup": 2 },
    { "key": { "tonic": { "note": "C" }, "mode": "major" }, "kind": "key",
      ↪ "fromGroup": 3 }
  ]
}

```

A label inside a tonicization can also carry its own localKey, so it describes itself without a reader having to consult the region table.

4. Secondary dominants and 5. dominant of the dominant (V/V)

You can write these three ways, depending on the system you're in; all three interoperate through the model.

1. **Roman numeral** — the applied notation that v0.1.0 already supported: V7/V, viio7/ii. RomanSemantic.secondary holds the target degree.
2. **Functional** — the DD token (Riemann's *Doppeldominante*) is the dominant of the dominant; a chain spells out the resolution: DD->D->T.
3. **Analytical attribute** — any label can declare an applied function through HarmonyAttributes.applied { target, chain? } and name the degree it tonicizes in HarmonyAttributes.tonicizes. Stacked secondaries (V/V/V) go in the chain, outermost first: ["V", "V"].

Surface

```

@rn
V7/V    V7    I           # secondary dominant of the dominant, then V, then I

```

```

@fun
DD -> D -> T           # Doppeldominante resolving through the dominant

```

The bracketed-attribute form attaches an applied reading to any label:

```

@cs
D7[of:V]  G7    C           # D7 functions as V/V in C
{ "surface": "D7", "semantic": { "kind": "chordSymbol", "root": { "note": "D"
  ↪ }, "quality": "major", "seventh": "dom7" },
  "attributes": { "applied": { "target": "V" }, "tonicizes": "V" } }

```

Think of the secondary dominant as the surface event and the **tonicization region** (§2/3) as its analytical span. The two work together: applied and tonicizes annotate the chord, while a TonalRegion of kind:"tonicization" annotates the passage.

Works on every chord root (fixed 2026-06-15). [of:...]
— and any bracketed attribute — now attaches to chord symbols whose root is followed by a letter, meaning flat roots and quality letters: Bb7[of:V], Db7[of:I], Dm7[of:V], Cmaj7[of:I], Em7b5[of:ii]. These used to lex as a single word, so the bracket got swallowed into the text run. The fix lets you express **tritone substitutions** as a flat-root dominant plus a target — e.g. Db7[of:I] = subV7/I. Parenthesised tension lists (A7(b9,b13))

also stay a single chord now. See the [berklee-jazz](#) and [jazzmus-berklee-dezrann](#) use cases and [berklee-jazz.md](#).

6. Functional analysis with layers

A figured-bass-style functional analysis stacks several aligned readings under each sonority. HAMON models this with **layered labels**: inside one `HarmonyGroup.primary`, every label carries a distinct `HarmonyLabel.layer` $\in$ {chord, degree, function, bass, melodic, key, tone}.

A typical four-layer column under one chord:

layer	content	example
key	the tonality	C major
function	tonal function	D (dominant)
degree	scale degree	V7
bass	bass / figured-bass	6_5 (first inversion)
melodic	melodic / non-harmonic tone	see §7

Surface

Prefix each reading with a layer tag (`<layer>:`) and join the readings with `,` inside one group — they're co-temporal, not rival:

@auto

fn:D, rn:V7, fb:6-5

```
{
  "primary": [
    { "surface": "D", "layer": "function", "semantic": { "kind": "functional",
      ↪ "chain": ["D"] } },
    { "surface": "V7", "layer": "degree", "semantic": { "kind": "roman",
      ↪ "degree": "V", "tail": "7" } },
    { "surface": "6-5", "layer": "bass", "semantic": { "kind":
      ↪ "figuredBass", "number": 6, "tail": "-5" } }
  ]
}
```

An `fb:` layer reads a number as a figured-bass figure rather than a Nashville degree; an `fn:` layer reads a bare token as a tonal function, so `fn:D` is the dominant even though `D` on its own would otherwise be a chord root.

The melodic/bass layer ties into **figured bass** (already a first-class `FiguredBassSemantic`) and into the **note-level analysis** in §7.

7. Melodic note analysis — harmonic (HT) and non-harmonic (NHT) tones

A note-by-note melodic analysis sorts each tone into **harmonic** (a chord tone, HT) or **non-harmonic** (a non-chord tone, NHT), and names the type of every non-harmonic one. That's what the new `ToneSemantic` (`kind:"tone"`) captures, on a melodic layer.

```
ToneSemantic { category, pitch?, type?, metric?, approach?, departure?, voice?
}
```

type (NonHarmonicType) covers the standard catalogue:

type	meaning
chord-tone	harmonic tone (HT)
passing	passing tone (use metric for accented/unaccented)
neighbor / upper-neighbor / lower-neighbor	neighbor (auxiliary) tone
incomplete-neighbor	incomplete neighbor
suspension	suspension
retardation	upward-resolving suspension
appoggiatura	appoggiatura (accented, approached by leap)
escape	escape tone (<i>échappée</i>)
anticipation	anticipation
pedal	pedal / organ point
cambiata	Fuxian <i>nota cambiata</i> / changing-note figure
changing-tone	changing tones (double neighbor)

approach and departure (Motion) record how the melody moves in and out (step-up, leap-down, resolve, ...). That motion is what tells, say, an appoggiatura (leap in, step out) apart from a passing tone (step in, step out).

Surface

On a melodic layer, you write a tone as a pitch followed by a bracketed class: [HT] for a harmonic tone, [NHT:<type>] for a non-harmonic one.

```
@auto
```

```
melodic:E[HT] , melodic:F[NHT:passing] , melodic:G[HT]
```

```
[
  { "surface": "E", "layer": "melodic", "semantic": { "kind": "tone",
    ↪ "category": "harmonic", "pitch": { "note": "E" }, "type": "chord-tone"
    ↪ } },
  { "surface": "F", "layer": "melodic", "semantic": { "kind": "tone",
    ↪ "category": "nonharmonic", "pitch": { "note": "F" }, "type": "passing",
    ↪ "metric": "unaccented", "approach": "step-up", "departure": "step-up" }
    ↪ },
  { "surface": "G", "layer": "melodic", "semantic": { "kind": "tone",
    ↪ "category": "harmonic", "pitch": { "note": "G" }, "type": "chord-tone"
    ↪ } }
]
```

This is the structure interactive melodic-analysis tools target (for example the Illescas analyzer, Phase 6.4): each analyzed note becomes a ToneSemantic, which you can align with the chord layer that decides what counts as harmonic.

8. Modal harmony

Modal analysis needs two things: a key mode beyond major/minor, and the freedom to spell the degrees that mode implies — bVII in mixolydian, bII in phrygian, a reinterpreted major IV, the raised vi of dorian.

Mode adds the seven diatonic church modes alongside major/minor: ionian, dorian, phrygian, lydian, mixolydian, aeolian, locrian. major≡ionian and minor≡aeolian, and both spellings are allowed so a source can keep its own vocabulary. A modal context is nothing more than a Key/TonalRegion with that mode; the degrees are ordinary RomanSemantic labels measured against it.

Surface

```
@rn
@key:D:dorian
i      IV      bVII  i      # dorian: major IV and subtonic bVII
{ "regions": [ { "key": { "tonic": { "note": "D" }, "mode": "dorian" }, "kind":
  ↪  "region", "fromGroup": 0 } ] }
```

9. Omitted fundamentals (*fundamentales omitidas*)

There are two distinct claims here, and two attributes on HarmonyAttributes to match:

- rootless: true — the **voicing** drops the root by convention (a jazz rootless voicing). A performance fact.
- omittedRoot: true (optionally with impliedRoot) — the **analysis** asserts an understood fundamental that never sounds (the classic incomplete dominant seventh, or a diminished seventh read as a rootless dominant minor ninth). An analytical claim.

The general “this degree is absent” case (omit3, no5) stays on ChordSymbolSemantic.omits. omittedRoot/impliedRoot are reserved for the *fundamental* specifically, because that’s the one that carries analytical weight.

Surface

[no1] marks the omitted root; [rootless] the voicing convention:

```
@rn
viiø7[no1]      # diminished seventh read as a rootless V7(b9): root omitted
{ "surface": "viiø7", "semantic": { "kind": "roman", "degree": "vii", "tail":
  ↪  "ø7" },
  "attributes": { "omittedRoot": true, "impliedRoot": { "note": "G" } } }
```

10. Arpeggiation / chord unfolding (*despliegue de los acordes*)

When a single harmony *unfolds* across time — an arpeggio, an Alberti bass, a broken chord, or a broader prolongation — it’s one analytical event spread over several groups.

- HarmonyAttributes.arpeggiated: true — the chord is realized as an arpeggio.
- HarmonyAttributes.prolongation ∈ {arpeggiation, pedal, neighbor, passing, prolongation} classifies *how* a harmony elaborates a structural one (Schenkerian-style).

- `HarmonyAttributes.prolongs`: `<groupIndex>` points the elaborating harmony at the structural harmony it depends on.

Surface

```
@cs
C[arp]          # C major realized as an arpeggio (despliegue)

@rn
I    V[ped]    I          # the V is a neighbor/pedal prolongation of I
{ "surface": "C", "semantic": { "kind": "chordSymbol", "root": { "note": "C" },
  ↪  "quality": "major" },
  "attributes": { "arpeggiated": true } }
```

A passing or neighbor chord that prolongs an earlier structural harmony points to it by group index:

```
{ "attributes": { "prolongation": "passing", "prolongs": 0 } }
```

10-bis. Extent (v0.4.1) — how long a reading lasts

A `Position` says where a harmony *starts*. `[dur:...]` and `[endref:...]` say where it **ends**, and they sit on the **label**, not on the group.

That placement is the whole point. A group is one time point, shared by every reading in it; the extent belongs to the reading. Two alternatives can hear the same passage differently — one a V spanning two bars, the other V then V7 — and only a per-label extent can hold both:

```
@rn
@meter:4/4
m:1,ts:1,V[dur:8]|V[dur:4]      # two readings, same onset, different
  ↪  segmentation
```

Surface	Field	Meaning
<code>[dur:3/4]</code>	<code>HarmonyAttributes.duration</code>	extent in quarter notes — the same unit as <code>Position.time</code> . A fraction (3/4), an integer (2) or a decimal (1.5).
<code>[dur:1.85s]</code>	<code>HarmonyAttributes.duration</code>	Extent in seconds (v0.5) — the same clock as <code>Position.seconds</code> . One key, an optional suffix: the extent inherits the clock the source stated.
<code>[endref:note-12]</code>	<code>HarmonyAttributes.endRef</code>	the score object it ends on (MEI <code>@endid</code>), the structural counterpart of <code>ref</code> :

```
{ "surface": "Cmaj7", "semantic": { "kind": "chordSymbol", "root": { "note":
  ↪  "C" }, "seventh": "maj7" },
  "attributes": { "duration": { "numerator": 4, "denominator": 1 }, "endRef":
  ↪  "note-12" } }
```

Only what a source stated. Absent does not mean zero, it means *unknown*. The rule that a harmony runs until the next group is a **computation**, and it is never written into the data — a derived extent stored as if it were annotated is indistinguishable from one an editor asserted, and that is how a lossy conversion comes to look lossless. An extent that cannot be read ([dur:soon]) is ignored rather than guessed; the surface still round-trips.

Where it comes from today, in quarter notes: Dezzrann duration, DCML expanded duration_qb, the AugmentedNet dialect of the DiLeMMA pitch arrays (a_duration), and MEI @endid. **Harte and JAMS state theirs in seconds** (Harte's end, JAMS's duration), which since v0.5 is read into durationSeconds rather than dropped. The two are never converted into each other — crossing the clocks needs a tempo map the label stream does not have. See [positions.md](#).

11. Chord-scales (v0.3.0) — jazz/Berklee

This records the recommended scale or scales for a chord or a region (chord-scale theory). There are two places to anchor it:

- **Per chord** — one or more [scale:NAME] attributes → HarmonyAttributes.scales (a list of ScaleSpec). Repeat the bracket to give alternatives.
- **Per region** — a @scale:NAME[,NAME...] directive that binds to the region opened by the preceding @key: → TonalRegion.scales.

NAME is an open vocabulary: the church modes plus altered, lydian-dominant, locrian-natural-2, whole-tone, diminished-hw / diminished-wh, harmonic-minor, melodic-minor, bebop-dominant, blues, and so on. A ScaleSpec can also carry an explicit tonic (when it differs from the chord root or region key) and explicit pitches for non-standard scales.

Surface

```
@key:C
@scale:major,lydian
Dm7[scale:dorian]
G7[scale:mixolydian][scale:altered]
Cmaj7[scale:lydian]
{ "surface": "G7[scale:mixolydian][scale:altered]",
  "semantic": { "kind": "chordSymbol", "root": { "note": "G" }, "quality":
    ↪ "major", "seventh": "dom7" },
  "attributes": { "scales": [ { "name": "mixolydian" }, { "name": "altered" } ]
    ↪ } }
```

A per-chord scale takes the chord root as its tonic unless tonic says otherwise; a per-region scale takes the region key. See [berklee-jazz.md](#) for the jazz/Berklee workflow.

Mapping summary (schema \$defs)

Capability	Schema \$def / field
Point alternatives	HarmonyGroup.alternatives
Progression alternatives	HamonSequence.alternativeAnalyses → AlternativeAnalysis
Tonal regions / modulation	HamonSequence.regions → TonalRegion (kind: key\ region\ modulation)

Capability	Schema \$def / field
Tonicizations	TonalRegion (kind: tonicization, degree, parent) + HarmonyLabel.localKey
Secondary dominants / V/V	RomanSemantic.secondary, FunctionalSemantic.chain (DD), HarmonyAttributes.applied/tonicizes
Layered functional analysis	HarmonyLabel.layer (AnalyticalLayer)
Melodic HT/NHT analysis	ToneSemantic + NonHarmonicType + Motion
Modal harmony	Mode (church modes) on Key / TonalRegion
Omitted fundamentals	HarmonyAttributes.omittedRoot / impliedRoot / rootless
Arpeggiation / unfolding	HarmonyAttributes.arpeggiated / prolongation / prolongs

Surface summary (EBNF rules)

Surface	EBNF rule
@key:C, @key:A:minor, @key:D:dorian, @key:V	keyDecl, keyTarget, modeSuffix
fn:D, rn:V7, fb:65, melodic:E [no1], [arp], [ped], [inv:1], [of:V], [HT], [NHT:passing]	layerTag, layerName analysisAttr
V7/V, viio7/ii	romanNumeral / rnSecondary (v0.1.0)
DD->D->T	functionalHarmony (v0.1.0)

Implementation status

The analytical layer is **wired end-to-end in the reference implementation** — every surface form in this document parses from .hamon text into the AST.

- **Canonical model** (grammar/hamon-schema.json, v0.2.0) — complete.
- **Typed AST** (hamonpy/hamonpy/ast.py) — complete.
- **Grammar + reference parser** (grammar/hamon.ebnf, antlr/*.g4, the generated Python parser, visitor and normalizer) — complete. The keyDecl region directive, layerTag, bracketed analysisAttr, and HT/NHT tones all parse into regions, layer, attributes, and ToneSemantic. Tonicization tonics are computed from the parent key (@key:V in $C \rightarrow G$).
- **Tests** — a surface-parse suite hamonpy/tests/test_analysis_parse.py (plus the conformance/ golden corpus, which includes analytical labels); a model suite hamonpy/tests/test_analysis.py.
- **Example fixtures** — fixtures/analysis/ (canonical-model JSON).

Berklee-style jazz analysis & the MuRET-hamon pipeline

This page covers two things: how HAMON encodes a **Berklee-method jazz harmonic analysis**, and how it serves as the **interchange layer** in the *MuRET-hamon* workflow (produce → store → render visually) without tying itself to any one renderer.

Berklee constructs in HAMON

A Berklee reading is a *functional* analysis of chord changes. HAMON represents it as a chord-symbol (@cs) sequence inside a key region, carrying applied functions and tensions:

Berklee concept	HAMON surface	Captured as
Key / tonal center	@key:C	TonalRegion (home key)
Diatonic seventh chords	Cmaj7, Dm7, Em7, Fmaj7, G7, Am7, Bø7	ChordSymbolSemantic
Tensions	Cmaj7#11, Dm9, G13, G7b9b13	extensions / alterations
Secondary dominant (V7/x)	A7[of:ii], D7[of:V]	HarmonyAttributes.applied.target
Related ii of a V7/x	Eø7[of:ii]	applied.target
Tritone substitution (subV7/x)	Db7[of:I], Ab7[of:V]	flat-root dominant + applied.target
Chord-scale (per chord)	G7[scale:altered], Dm7[scale:dorian]	HarmonyAttributes.scales
Chord-scales (per region)	@scale:dorian,mixolydian	TonalRegion.scales
Time position (for sync/render)	t:12/1	HarmonyGroup.position.time (quarters)

A grammar fix on 2026-06-15 enabled the flat-root / quality-letter [of:...] cases (Db7[of:I], Dm7[of:V], Em7b5[of:ii]) and parenthesised tension lists (A7(b9,b13)) — see [analysis.md](#) (§4) and the CHANGELOG. Since v0.3.0 the parenthesised tensions are also **lifted into the semantic** (alterations / extensions), so G7(b9,b13) and the stacked G7b9b13 normalize to the same thing. **Chord-scales** (per chord and per region) arrived in v0.3.0 — see [analysis.md §11](#). **Still missing:** a first-class subV marker; for now subV is just the flat-root dominant plus [of:target].

The MuRET-hamon pipeline — HAMON as the communication medium

JAZZMUS lead sheet (MusicXML / Humdrum ****kern** + recognised chords)

| hamonpy.adapters.humdrum (the ****jazz** chord spine) · or
 ↪ hamonpy.adapters.formats extractors



HAMON – raw changes (typed chord symbols)

| analyst adds the Berklee layer (@key, [of:...], subV, t: anchors)



HAMON – the canonical, exchangeable analysis (.hamon / .hamon.json)

|→ MuRET-engraving – renders the analysed score (the visual layer)

|→ Dezrann (.dez) – time-aligned labels on the score/audio

↪ timeline

|→ ms3 / DCML ecosystem – exchange with Roman-numeral corpora & tooling

- **MuRET-hamon** (app, OMR-PRAIG-UA-ES/MuRET-hamon) drives the pipeline: it runs the Berklee analysis on the HAMON model, renders with **MuRET-engraving**, and saves .hamon.json. HAMON stays *render-agnostic* — the analysis artifact is the contract, and MuRET-engraving and Dezrann are interchangeable visual consumers of it.
- The two runnable recipes in this repo are the reference point: [use-cases/berklee-jazz/](#) (the encoding/concept demo, with a “what HAMON captures vs. needs” table) and [use-cases/jazzmus-berklee-dezrann/](#) (the full JAZZMUS → HAMON → Dezrann pipeline).
- Related interchange targets: [dezrann.md](#), [ms3.md](#), [humdrum.md](#), [music21.md](#).

Where things live

Artifact	Path
Concept use case (encoding + gap table)	use-cases/berklee-jazz/
Pipeline use case (JAZZMUS→HAMON→Dezrann)	use-cases/jazzmus-berklee-dezrann/
Applied-function / [of:...] spec + the fix note	documentation/analysis.md
Dezrann .dez adapter	hamonpy/hamonpy/adapters/dezrann.py · documentation/dezrann.md
Grammar (source of truth)	grammar/hamon.ebnf · antlr/hamonParser.g4
Conformance corpus (Python golden)	conformance/

Time-aligned positions (measures & beats)

HAMON is a **harmony-label** standard. It tells you *what* the harmony is and *where* it starts — not the note durations or rhythm, which stay in the source score. A harmony's *extent* is implicit: it runs until the next label. Each harmony's **onset position** is encoded optionally, at the **group** level. Since v0.4, HAMON also declares the **meter** (@meter:), which is what lets a beat position (ts) be interpreted and range-checked.

Surface: the position items (written first)

Since v0.4 the position is written **first** in the group, comma-separated with the labels, using short key:value items. There are five forms, and you can combine them freely:

Surface	Meaning	JSON (group.position)
m:1,ts:3	measure : beat	{ "measure": 1, "beat": 3.0 }
m:4	measure only	{ "measure": 4 }
t:5/4	absolute musical position, as a fraction of quarter notes	{ "time": { "numerator": 5, "denominator": 4 } }
s:12.34	absolute physical position, in seconds (v0.5)	{ "seconds": 12.34 }
ref:note-9	reference to a score object id (MEI @startid)	{ "ref": "note-9" }

ts: is a **timestamp/beat** following MEI [data.BEAT](#): a **1-based decimal**, with fractions allowed (ts:2.5).

t: and s: are both absolute and are **not the same clock**: t: counts quarter notes (musical time), s: counts seconds (physical time — audio, or one performance). Neither is ever computed from the other; see Hold, don't convert.

@version:0.4.0

@meter:4/4

@key:C

m:25,ts:1,cs:C,rn:I

m:27,ts:1,cs:F,rn:iv

m:28,ts:3,cs:Fm7,rn:ii

The position items come **before** the label(s), so an analysed chord and its position live together in one comma-separated group (m:3,ts:3,cs:A7[of:ii][scale:altered]).

Meter (time signature) — @meter:N/D

A ts beat only means something relative to a **time signature**. Since v0.4 a @meter:N/D directive declares the meter in effect. Write it **at the start and at every metric change** (just like @key:). The **numerator N is the beat count**, and that fixes the valid range of ts: following MEI data.BEAT, the downbeat is ts:1 and the next barline is ts:(N+1), so **ts spans [1, N+1)**.

@meter	beats	valid ts
4/4	4	1 ... < 5 (e.g. 4.5 ✓, 5 ✗)
2/2	2	1 ... < 3
3/4	3	1 ... < 4
6/8	6	1 ... < 7

```
@meter:4/4
m:1,ts:1,cs:C
m:2,ts:4,cs:G7      # ts:4 ✓ (last beat of 4/4)
@meter:2/2          # metric change — from here ts spans [1,3)
m:3,ts:1,cs:C
```

- **Additive and optional.** A sequence with no @meter is still valid. Without one, the beat range is simply unknown, so ts goes unchecked.
- **Non-blocking validation.** A ts outside the current meter's range (say ts:5 in 4/4) is a **warning**, never a parse error — HAMON has to round-trip imperfect sources. (hamonpy emits a HamonWarning; the pure check is hamonpy.validate.validate_positions.)
- **JSON.** Meters serialize to an optional top-level meters array. Each entry is { "numerator", "denominator", "fromGroup" }, where fromGroup is the index of the first group it governs.
- HAMON declares the meter **only to interpret ts**. It still does not model note durations or rhythm.

JSON

Positions serialize to an optional position object on each HarmonyGroup (grammar/hamon-schema.json → HarmonyGroup.position). It holds five fields, all optional and freely combinable:

```
{ "groups": [
  { "primary": [ { "surface": "C", "layer": "chord", "semantic": { "kind":
    ↪ "chordSymbol", ... } } ],
    "position": { "measure": 25, "beat": 1.0 } }
] }
```

- measure (int) + beat (float, 1-based) — mirror an MEI @tstamp.
- time (fraction) — an **absolute** offset in **quarter notes**, independent of barlines. This one holds up when there is no bar structure to lean on. Quarters because that is what every source uses: DCML quarterbeats, Dezrann start, the DiLeMMA pitch arrays, music21 offsets. t:4 is the fifth quarter — in 4/4, the downbeat of bar 2.
- seconds (float, v0.5) — an **absolute** offset in **physical** time. This is what an audio annotation states, and until v0.5 it was the one thing HAMON could not hold.
- ref (string) — a score-object id (MEI @startid), for exact anchoring.

Design rationale (scope)

- **Meter only bounds ts; no durations.** @meter: declares the time signature so a beat can be interpreted and range-checked. It stops there — HAMON does not duplicate the score's rhythmic model. It anchors *when a harmony begins*, and the span runs until the next label.
- **Optional and sparse.** Most exchange scenarios (lead sheets, label lists) need no positions at all. You add them only when time-alignment matters.
- **Four anchoring strategies coexist:** metric (m:/ts:), absolute musical (t:), absolute physical (s:), and structural (ref:). A converter stores whichever the source provides — all of them, if it provides all of them.

Position loss (the ICCCM'26 matrix)

The ICCCM'26 loss matrix counts, inside each cell, the onsets the target can't carry natively (position in the cell's itemised `lost_aspects`) — and since v0.5 that question is asked **per clock**, because a target that holds one clock still loses the other. Most score and annotation formats carry `measure:beat` (`POSITION_NATIVE` in `hamonpy/capability.py`); the audio-time pair, **Harte** and **JAMS**, carry seconds instead (`SECONDS_NATIVE`). So a metric onset is lost by **plain DCML** (its `chord/localkey/globalkey` writer has no `mc/mn/quarterbeats` columns — the *expanded* tables do), **Harte** and **JAMS**; a `s:` onset is lost by everything except those last two. A position counts as dropped only when the target can carry **none** of the clocks it states.

Mapping to/from formats

Format	Carries position as
MEI	<harm @tstamp> (<code>measure:beat</code>) or @startid (→ ref)
DCML expanded (ms3)	<code>mc/mn</code> → <code>measure</code> , <code>mn_onset</code> → <code>beat</code> , <code>quarterbeats</code> → <code>time</code>
DiLeMMa pitch arrays	one row per note ; the harmony's onset is the first note carrying it (<code>quarterbeats_playthrough / j_offset</code> → <code>time</code>) — see dcml.md
Humdrum Harte .lab	spine row alignment (implicit <code>measure:beat</code>) <code>start end label</code> ; the <code>start seconds</code> → <code>s:</code> (v0.5) — a different clock, not a metric position
JAMS plain DCML , RomanText , iReal , bare Harte labels	<code>observation time in seconds</code> → <code>s:</code> (v0.5) <code>measure numbers only</code> , or <code>none</code>

Extent — how long a harmony lasts (v0.4.1, seconds since v0.5)

A position is an **onset**. The extent is `[dur:...]` and `[endref:...]`, and it lives on the **label**, not the group — full reference in [analysis.md](#). Absent means *unknown*: the rule that a harmony runs to the next group is a computation, never stored.

That rule is right most of the time and wrong in four cases, which is why the field exists — the last harmony of a piece (there is no next group), a harmony that ends before the next one starts (a rest, an N.C., a fermata close), an anacrusis or repeat where the next label is not the next in time, and two alternatives that segment the same passage differently.

Where the extent comes from, and where it still does not:

Format	States extent as	HAMON
Dezrann .dez	start + duration (quarters)	□ read and written; the writer no longer fills the gap
DCML expanded	quarterbeats + duration_qb	□ read
DiLeMMa (AugmentedNet)	a_duration	□ read
MEI <harm>	@tstamp2, @endid	□ @endid → endRef; @tstamp2 needs the meter, not read yet
Harte .lab	start + end, in seconds	□ read as s: + [dur:...s] (v0.5) — the physical clock
JAMS	time + duration, in seconds	□ same, and written back

Before this existed, a Dezrann round-trip was not lossless but *looked* lossless: the writer recomputed duration from the gap, the re-parsed copy matched, and `report.py` — which diffs the canonical dicts — could not object, because an aspect the AST does not model can never be reported as dropped. It now is one.

The two clocks: s: physical time (v0.5)

Until v0.5 HAMON had metric (m:/ts:), musical absolute (t:) and structural (ref:) and **nothing in seconds**. So Harte and JAMS, whose *only* positional data is audio seconds, came in with no position whatsoever — an interlingua that drops what a source stated is not converting, it is discarding. s:12.34 closes that: physical time is a **fourth clock**, alongside the others rather than instead of them.

```
@version:0.5.0
```

```
@cs
```

```
s:0,C
```

```
s:2.5,F
```

```
s:4.25,G7
```

For a multimodal source — audio plus a transcription of it — both clocks are stated and both are kept, in one position:

```
m:25,ts:1,s:48.5,G7
```

That line is the whole point. In the recording the **seconds are the certain datum** and the bar is an estimate; in the score it is the other way round. Only holding both says which is which.

Hold, don't convert

Hold, don't convert. Every clock a source states is stored as stated; a clock it did not state stays absent; conversions are computed on demand where a map exists. Never overwrite a stated clock with a derived one — that launders an estimate into a fact.

This is why there is no tempo field and no s: ⇌ t: conversion in the AST. Going from one clock to the other needs a tempo map that the label stream does not have; a converter that owns one can compute the crossing itself, and what it computes is its own, not the source's. hamonpy therefore never fills seconds from time or the reverse — Harte's importer sets only seconds, MEI's sets only measure/beat.

The extent inherits the clock

An onset in seconds implies an extent in seconds, so [dur:...] takes an optional s suffix: **[dur:3/4] is quarter notes, [dur:1.85s] is seconds**. One key, two clocks, and the suffix is the whole difference — a bare value is never read as seconds. They are separate fields (attributes.duration and attributes.durationSeconds) for the same reason t: and s: are: a target that speaks quarters must see a seconds extent as *absent*, not as a number it can use.

So the audio-time formats now keep both ends of their annotation:

Format	States	HAMON
Harte .lab	start end label, in seconds	□ start → s:, end - start → [dur:...s]
JAMS	observation time + duration, in seconds	□ both, and both written back

A bare Harte label list, with no columns, still gets no position and no extent: absent means unknown. Harte's **writer** puts the columns back when every harmony states both, so a .lab round-trips byte for byte; one untimed harmony and the whole file falls back to bare tokens, because a half-timed .lab is not a .lab (see [harte.md](#)).

Measureless music. Unmetered and **mensural** repertoire has no barlines and no time signature, so @meter: and m:/ts: don't apply — they assume a measure, exactly as data.BEAT does. There, the absolute t: and ref: items are your anchors. See the “measureless harmony” open item.

See also: [analysis.md](#) (the analytical layer), [dcml.md](#) (expanded time-aligned tables), [systems.md](#) (system hint vs analytical layer).

Systems, the ‘@’ hint, and the analytical layer

Here’s a question that comes up a lot: *if a label carries [of:II] (a Roman-numeral degree) and [scale:lydian-dominant], does it still make sense to declare @cs (chord symbol)?* It does — because three things are **orthogonal**:

1. **Base harmony system** — how the *label itself* is spelled: chord symbol (cs), Roman numeral (rn), Nashville number (ns), figured bass (fb), functional (fun).
2. **Analytical layer** — optional attributes you can layer onto **any** label: [of:...] (applied/secondary target), [scale:...] (chord-scale), [NHT:...] (non-harmonic tone), [inv:...], @key:... regions, and so on. See [analysis.md](#).
3. **Per-label detected_system** — what each label actually is, computed independently of the sequence hint.

D7[of:II][scale:lydian-dominant] is a **chord symbol** (D7) with two analytical attributes. The II inside [of:...] is Roman-numeral *notation for the applied target* degree, and [scale:...] is a Berklee chord-scale. **Neither one touches the base system**, so @cs is exactly right for a jazz lead sheet where every label is a chord symbol.

The @<system> sequence hint

The first non-version directive can declare the sequence’s system:

Hint	Effect
@cs @rn @ns @fb @fun	declares the default/expected base system for the sequence
@auto (or none)	each label’s system is detected independently

The hint does two jobs. It sets the sequence-level system, and it **disambiguates** tokens that could parse under more than one system — a bare 6 as figured bass vs. Nashville, or bVI as a Roman numeral vs. a flat chord root.

@cs vs @auto — when to use which

The hint sets the label’s system field, while detected_system always reports what the label really is:

Input	detected_system	system (with @cs)	system (with @auto)
D7[of:II][scale:...]	cs	cs	cs
V7/V	rn	cs ← forced	rn
6-5	fb	cs ← forced	fb

- **Homogeneous** sequence (a chord-symbol lead sheet with analysis, like the ICCCM'26 flagship) → **declare the system** (@cs). It's honest, and it disambiguates.
- **Genuinely mixed** systems in one sequence (some chord symbols, some Roman, some figured bass) → **@auto**, so each label keeps its own system and a V7/V isn't stamped system=cs. This is why the ICCCM'26 *pangram* uses @auto.

Either way, the parsed kind and detected_system come out identical — only the sequence-level system label differs. Both are valid encodings of the same content.

See also: [analysis.md](#) (the analytical layer in full), [positions.md](#) (time-aligned positions).

Datasets & the harmony hub

HAMON is a **hub of harmony**: it converts real corpora into one canonical model (grammar/hamon-schema.json) so they can interoperate. What it deliberately **does not** do is redistribute anyone else's dataset. Instead it ships a *registry* of the original sources plus a small downloader. You fetch each corpus from its authors, under its own license, and convert it with a format adapter.

- Registry: [datasets/manifest.json](#)
- Policy + quickstart: [datasets/README.md](#)
- Worked recipes: [use-cases/](#)
- API: `hamonpy.datasets`

The registry (datasets/manifest.json)

Each entry records the name, title, description, authors, homepage, license, and citation, along with the on-disk format, the hamon_adapter that reads it, and a download block (method = http | git | builtin | manual, plus a url when there is a single archive to grab).

Dataset	Format	Adapter	Download
when-in-rome	RomanText	<code>hamonpy.adapters.romantext</code>	git (MarkGotham)
distant-listening-corpus	DCML TSV	<code>hamonpy.adapters.dcmllib</code>	git (DCMLab)
dcml-corpora	DCML TSV	<code>hamonpy.adapters.dcmllib</code>	git (DCMLab)
jazz-harmony-treebank	treebank.json	<code>hamonpy.adapters.treelib</code>	git (DCMLab)
key-modulation	Humdrum **kern	<code>hamonpy.adapters.humdrum</code>	git (DDMAL)
dilemmadata	pitch-array TSV (note-level)	<code>hamonpy.adapters.dilemma</code>	git (Zenodo)
jazzmus / jazzmus-code	MusicXML + **kern / code	<code>hamonpy.adapters.humdrum</code>	gatingface (gated) / git
interactive-melodic-analysis	HT/NHT (HA-MEI)	<code>hamonpy.adapters.mei</code>	manual (contact authors)
isophonics	Harte .lab	<code>hamonpy.adapters.harte</code>	manual (isophonics.net)
music21-corpus	music21 stream	<code>hamonpy.adapters.music21</code>	builtin adapter (music21.corpus)
irealpro	irealb:// URI	<code>hamonpy.adapters.irealb</code>	manual (export your own)

The first nine are the datasets from the HAMON use-cases paper; there is a worked recipe for each under [use-cases/](#).

Using the companion downloader

You can fetch those datasets unattended with the paper's `download_music_datasets.py`, which writes its own manifest. HAMON understands that manifest's integration contract, so you can point it at data you have already downloaded:

```
import os
from hamonpy import datasets
from hamonpy.adapters.romantext import romantext_file_to_hamon

os.environ["MUSIC_DATASETS_MANIFEST"] =
    ↪ "/data/music-corpora/music_datasets_manifest.json"
root = datasets.resolve_local_path("when-in-rome")      # maps name → downloader
    ↪ id → local path
seq = romantext_file_to_hamon(str(next((root /
    ↪ "Corpus").rglob("analysis.txt"))))

resolve_local_path(name, manifest_path=None) respects the contract's status_ok_values
(cloned/updated/exists/downloaded), and it raises for gated datasets (jazzmus) or unavail-
able ones (interactive-melodic-analysis).
```

API

```
from hamonpy import datasets

datasets.list_datasets()          # names
datasets.get_dataset("dcml-corpora") # the manifest entry (license, citation,
    ↪ ...)
datasets.default_cache_dir()      # ~/.cache/hamon/datasets (override:
    ↪ HAMON_CACHE_DIR)
datasets.download_dataset(name, dest_dir=None, url=None)

download_dataset pulls only http(s):// and file:// sources into the cache; it never bundles,
scrapes, or mirrors anything. For git / builtin / manual datasets it raises and points you at
the homepage and license — you obtain those yourself. Pass url= to use a mirror or a local
copy.
```

To override where the manifest lives, set `HAMON_DATASETS_MANIFEST`.

Responsibilities

HAMON provides only the conversion layer. **The data and its terms belong to the original authors** — so honour each dataset's license and cite its citation in anything you publish. Downloaded payloads stay in the git-ignored cache and are never committed.

Converting a corpus

```
from hamonpy import datasets
from hamonpy.adapters.dcml import dcml_tsv_to_hamon

info = datasets.get_dataset("dcml-corpora")
print("Cite:", info["citation"], "| License:", info["license"])
# obtain the corpus per info["homepage"], then:
seq = dcml_tsv_to_hamon("path/to/annotations.tsv")
```

The source adapters map key/region columns — DCML localkey/relativeroot, the Humdrum ****harm/**function** layering — into the analytical layer (TonalRegion, HarmonyLabel.layer). See [../STATUS.md](#).

Command-line interface ('hamon' / 'python -m hamonpy')

The hub, as a command-line tool: convert any supported harmony file into the canonical HAMON JSON (grammar/hamon-schema.json), and inspect the dataset registry.

Once you've run `pip install -e ./hamonpy`, the `hamon` command is on your path; `python -m hamonpy` does the same thing.

convert

```
hamon convert <file> [--format F] [-o out.json] [--indent N]
```

This auto-detects the input format and writes the canonical JSON to stdout (or to -o):

```
hamon convert harmonies.tsv           # DCML TSV → JSON with tonal regions
hamon convert analysis.txt            # RomanText (When in Rome)
hamon convert snippet.krn             # Humdrum (incl. key_modulation)
↳ **text/=>)
hamon convert treebank.json           # Jazz Harmony Treebank
hamon convert analysis.mei            # MEI / HA-MEI (regions + tones)
hamon convert chart.txt --format ireal # force a format
```

Detection goes by extension (.hamon, .tsv, .rntxt, .krn, .json, .mei, .xml, .lab, .abc, .ly, .mscx, .dez, .jams, .hrm, .har, .xlsx), falling back to a content sniff. To override it, pass --format — the 22 input formats are hamon, dcml, dcml_expanded, dilemma, romantext, treebank, humdrum, key_modulation, mei, harte, ireal, dezrann, jams, musicxml, abc, lilypond, musescore, bps_fh, harm, rock_corpus, kp, choro.

Two extensions carry more than one format, so the sniff decides: for .krn, the DDMAL **text/KEY=>: encoding is key_modulation and standard **harm/**function/**fb spines are humdrum; for .tsv, a quarterbeats column means dcml_expanded, a note-level pitch array means dilemma, and anything else dcml.

hamonpy.serialize.sequence_to_json produces the JSON. It has the same canonical, camel-Case shape as grammar/hamon-schema.json (and the TS AST).

Ready-to-run inputs for every command below live in [examples/](#) (examples/README.md walks through each one).

export — any format → any format

```
hamon export <file> --to TARGET [--format F] [-o out] [--save-hamon PATH]
↳ [--report]
```

Reads harmony from **any** supported input (same auto-detection as `convert`) and writes **any** target format. There is no fixed pairing: the input is parsed into the canonical HAMON model and the target is generated from it, so **X → HAMON → Y** works for every readable X and writable Y. Writes to stdout, or to `-o`.

```
hamon export song.mei --to harte           # MEI <harm> → Harte chord labels
hamon export changes.tsv --to romantext    # DCML TSV → RomanText
hamon export chart.txt --format ireal --to mei # iReal → MEI (force the input
↪ format)
```

`--to` accepts: `abc`, `dcml`, `dezrann`, `hamon`, `harte`, `humdrum`, `ireal`, `jams`, `lilypond`, `mei`, `mus-escore`, `musicxml`, `romantext`.

The HAMON JSON is always the intermediate. Two options capture it and the loss in the same run — because no two formats carry the same information, a conversion can drop what the target can't represent:

- `--save-hamon PATH` — also write the intermediate canonical HAMON JSON.
- `--report` — also print, to stderr, **what the target could not carry** (the same semantic-loss summary as the `report` command below).

```
hamon export song.mei --to harte --save-hamon mid.json --report
```

If the target can't represent the source's harmony system at all (e.g. Roman numerals → Harte), the output is empty and a note on stderr explains why.

report

```
hamon report <file> --to TARGET [--format F] [--write-output out] [--json]
```

Runs the same export, then **diffs the result back against the source** and reports every semantic difference — what was dropped, what changed, and the merely notational re-spellings (glyphs that differ across formats but mean the same thing). It writes a human-readable summary by default; `--json` emits the structured report, and `--write-output` also saves the exported text.

```
hamon report song.mei --to harte
```

Source: `song.mei` (format=`mei`)

Target format: `harte`

source groups: 5

△ semantic loss: 7 field(s)

[dropped] groups[0].position: {measure=1, beat=1.0}

...

[dropped] groups[4].primary[0].semantic.alterations[0]: `b5`

· notational re-spelling: 16 field(s) (expected – glyphs/surface differ

↪ across formats)

Harte can't carry beat positions and flattens `Am7b5` to `A:min7`, so the report flags the dropped positions and the lost `b5`.

A round-trip can also **gain** something the source never said — a default the writer supplies, a version the reader stamps (`mei` does), a group the format pads out. Those are reported apart, under `+` invented by the target, because nothing was lost:

Target format: `mei`

source groups: 3

✓ no semantic loss on round-trip

+ invented by the target: 1 field(s) (in the output, never said by the source)

[added] version: `0.2.0`

So lossless still means **nothing was lost** and is unaffected by an addition; faithful is the stricter question — lossless *and* nothing invented. The `--json` form has both booleans, `semanticLoss[]` and `invented[]` (each entry with path / kind / summary / source / output), plus target, sourceGroups, and error.

validate

```
hamon validate <file>... [--format F] [--strict] [--json]
```

Parses one or more files and reports two classes of corpus problem: **opaque labels** (a surface the parser couldn't read as harmony, so its semantic kind stayed text — e.g. a ?... fallback) and **position warnings** (a group's ts: beat that falls outside the meter declared with @meter). It never blocks parsing on its own; with `--strict` it **exits 1** when anything is flagged, so it works as a CI gate over a harmony corpus.

```
hamon validate corpus.hamon --strict
```

```
Source: corpus.hamon (format=hamon)
```

```
Groups: 6, labels: 6
```

```
Kinds: chordSymbol 5, text 1
```

```
  opaque (text) label at group 5: '?tacet'
```

```
  position: group 4 (m:3,ts:5.0) is outside meter 4/4 — ts must be in [1, 5)
```

```
1 opaque label(s), 1 position warning(s).
```

Pass several files at once (`hamon validate corpus/*.hamon --strict`) to QA a whole corpus. `--json` emits one report per file with groups, labels, kinds, `opaqueLabels[]`, and `positionWarnings[]`.

datasets

```
hamon datasets list                # registered dataset names
hamon datasets info when-in-rome  # authors / homepage / license / citation /
  ⇨ adapter
hamon datasets path when-in-rome  # local path of an already-downloaded
  ⇨ dataset
```

`datasets path` reads the companion downloader's manifest through `MUSIC_DATASETS_MANIFEST / MUSIC_DATASETS_ROOT` (see [datasets.md](#)) and prints the on-disk path. For example:

```
export
```

```
  ⇨ MUSIC_DATASETS_MANIFEST=/data/music-corpora/music_datasets_manifest.json
```

```
hamon convert "${hamon datasets path when-in-rome}/Corpus/.../analysis.txt"
```

Library equivalent

```
from hamonpy.cli import convert_file
```

```
from hamonpy.serialize import sequence_to_json
```

```
seq = convert_file("harmonies.tsv")    # auto-detect
```

```
print(sequence_to_json(seq))
```

Regex-based extractors

Three notation formats — **LilyPond**, **ABC notation**, and **MuseScore MSCX** — carry harmony as free-form text with no structured harmony model to read from, so hamonpy pulls harmony labels out of them with lightweight regular-expression extractors.

This page walks through the extraction strategy for each one. For the full format specs, follow the individual format pages linked below.

LilyPond (.ly)

See [lilypond.md](#) for the full format spec.

Extraction strategy (two-pass):

1. **% hamon-surface: comments** — explicit HAMON labels that the author writes as comment annotations in the LilyPond source. Each line matching `% hamon-surface: <label>` yields one label. This is the preferred, lossless path.
2. **\chordmode { ... } bodies** — when there are no explicit comments, the extractor parses the chord tokens inside `\chordmode` blocks. Dutch pitch names (`cis`, `es`, `b`) become English accidentals (`C#`, `Eb`, `B`), and LilyPond quality modifiers (`:m`, `:7`, `:maj7`) map to HAMON suffixes.

Regex (hamonpy_extract_lilypond):

```
r"^|s%|s*hamon-surface\s*:\s*(.+)$"      # explicit comment
r"\chordmode\s*\{([|s|S]*?)\}"          # chordmode body
```

ABC notation (.abc)

See [abc.md](#) for the full format spec.

Extraction strategy: ABC stores chord symbols as inline annotations, wrapped in double quotes right in the melody line:

```
"C"C2 "G7"GGG | "Am"A2 "F"F2 |
```

The extractor collects every `"..."` quoted string in the file. It does no structural parsing — anything in quotes is treated as a potential chord label and handed to the HAMON parser.

Regex (hamonpy_extract_abc):

```
r'"'([^\"]+)"'
```

Note: ABC files can also carry lyrics in quotes. Those usually fail to parse as chord symbols and end up classified as TextSemantic. That's by design: the extractor returns everything and lets the parser decide what is actually a chord.

MuseScore MSCX (.mscx)

See [musescore.md](#) for the full format spec.

Extraction strategy: MuseScore's uncompressed XML keeps harmony labels inside <Harmony> elements. The label text lives in a <text> child (MuseScore 3/4) or a <name> element (older versions):

```
<Harmony>
  <root>14</root>
  <text>Cmaj7</text>
</Harmony>
```

The extractor matches both forms.

Regex (hamonpy _extract_musescore):

```
r"<Harmony\b[^\>]*>[\s\S]*?<text>\s*([^\<]+?)\s*</text>[\s\S]*?</Harmony>"
r"<Harmony\b[^\>]*>[\s\S]*?<name>\s*([^\<]+?)\s*</name>[\s\S]*?</Harmony>"
```

Comparison

Format	Extraction basis	Lossless?	Structured?
LilyPond (comment)	% hamon-surface: annotations	☐ yes	☐ explicit
LilyPond (\chordmode)	Dutch pitch name parsing	mostly	partial
ABC	Quoted strings	☐ yes	no (text only)
MuseScore MSCX	<Harmony><text> XML	☐ yes	partial

All three extractors hand back a list of surface strings. The HAMON ANTLR parser and normalizer take it from there, so any label that survives extraction ends up fully normalized into a typed HarmonySemantic.

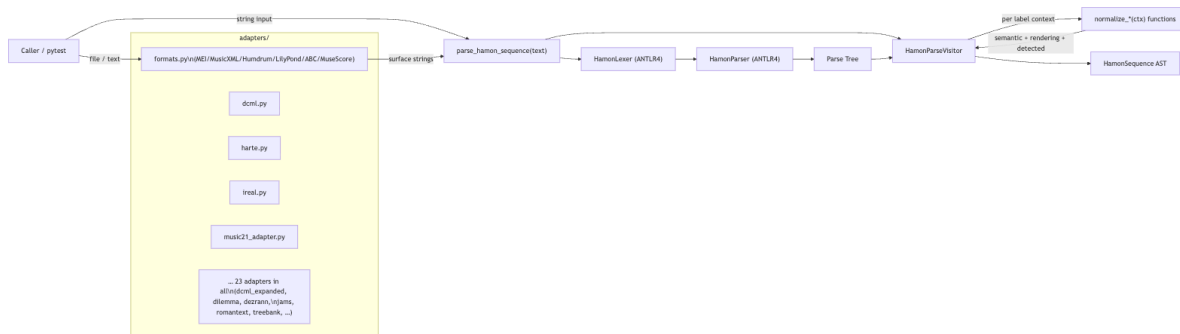
HAMON Architecture (Mermaid UML)

Mermaid diagrams of HAMON's architecture, as implemented in **hamonpy** (the Python reference implementation).

Conventions

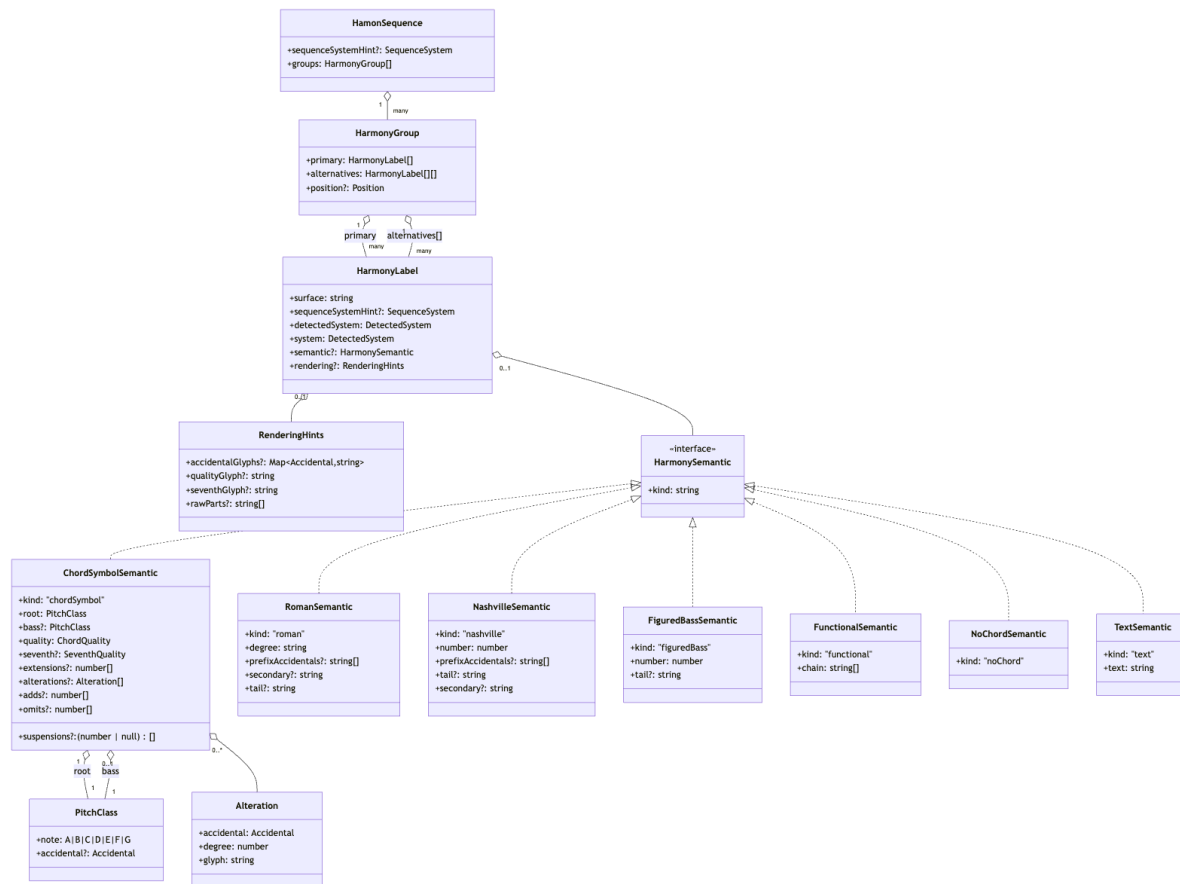
- **Surface-first**: every label keeps the exact substring as written.
- `sequenceSystemHint` comes from the optional `@cs/@rn/@auto/...` prefix line.
- `detectedSystem` is what the grammar and heuristics matched.
- `system` is the final system, once the hint has been applied.

1) Package / component overview



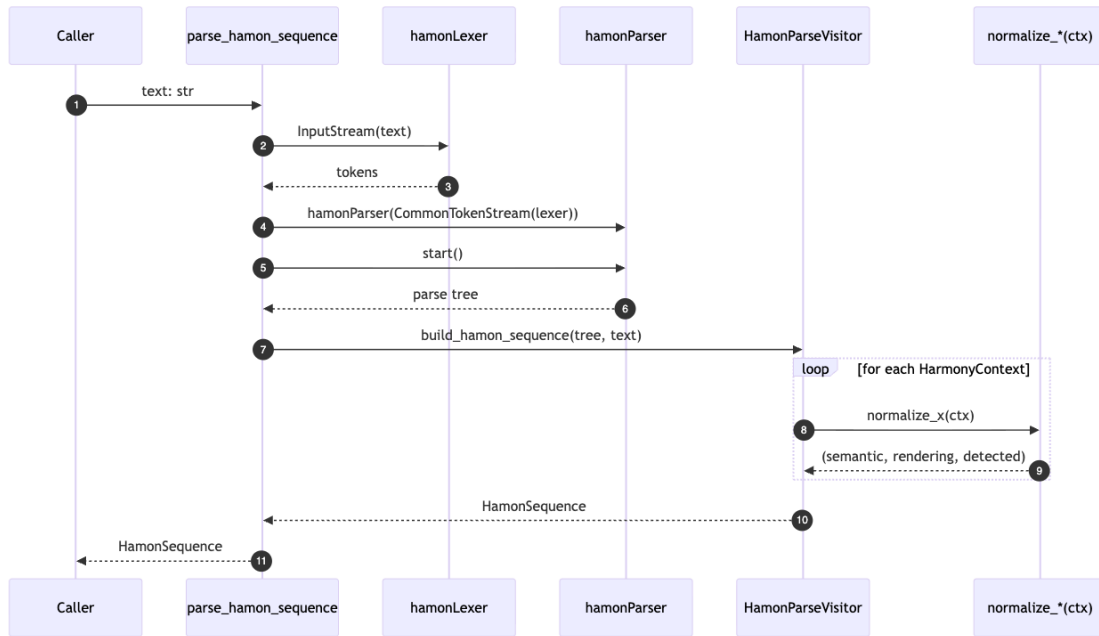
2) Core data model (AST) — class diagram

The core data model (`hamonpy/hamonpy/ast.py`).



3) Parsing pipeline — sequence diagram

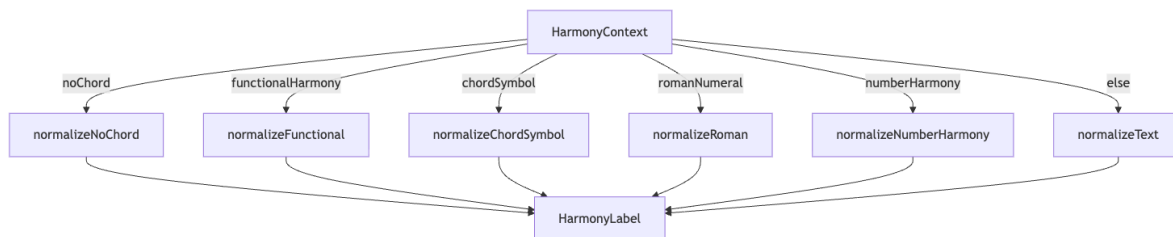
Entry point: `parse_hamon_sequence(text) → HamonSequence`.



4) Visitor decision logic (label family dispatch)

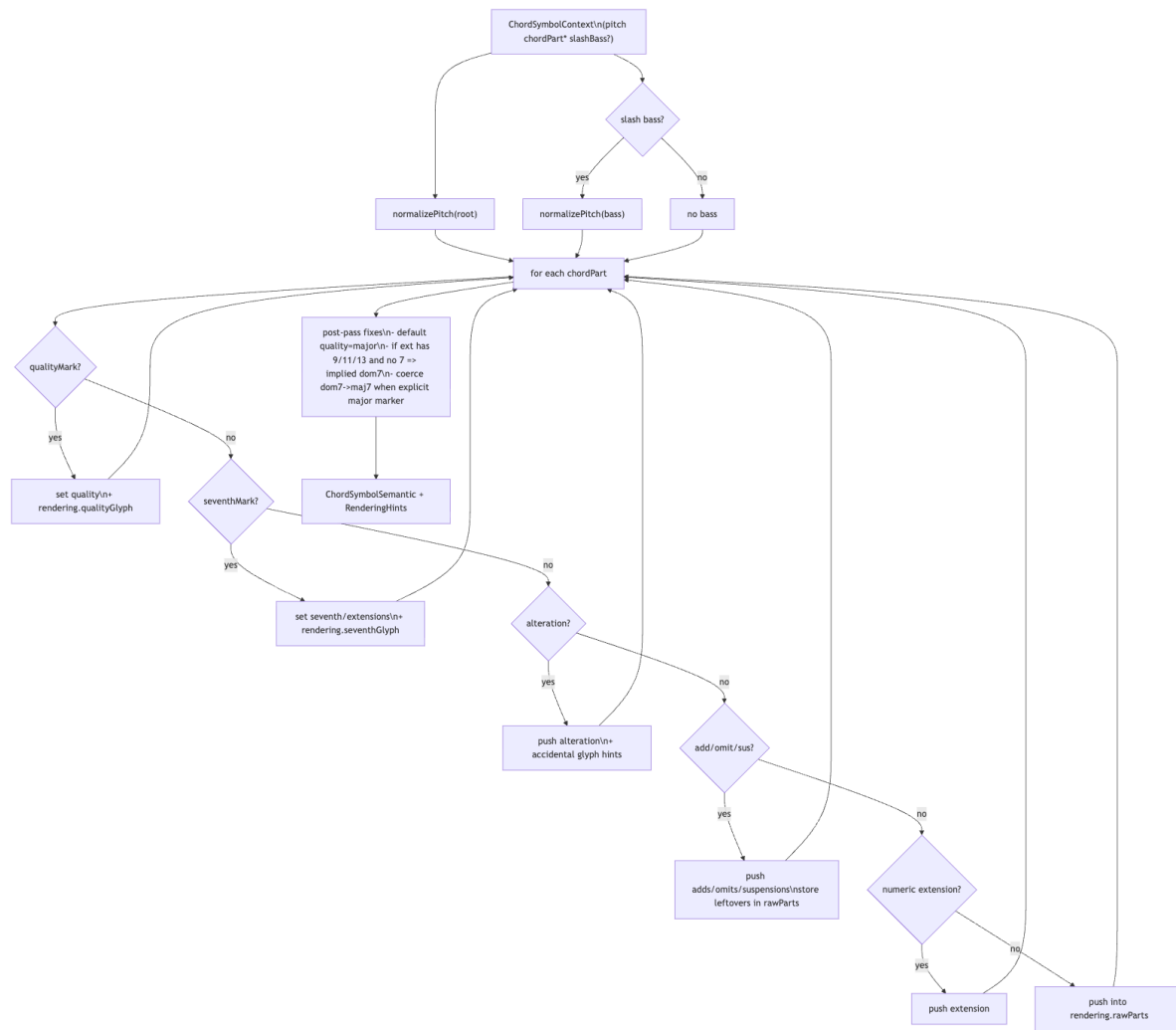
The **parser** first recognizes one of these label families: noChord | functionalHarmony | chordSymbol | romanNumeral | numberHarmony | textLabel.

The **visitor** mirrors that dispatch and calls the matching normalizer.

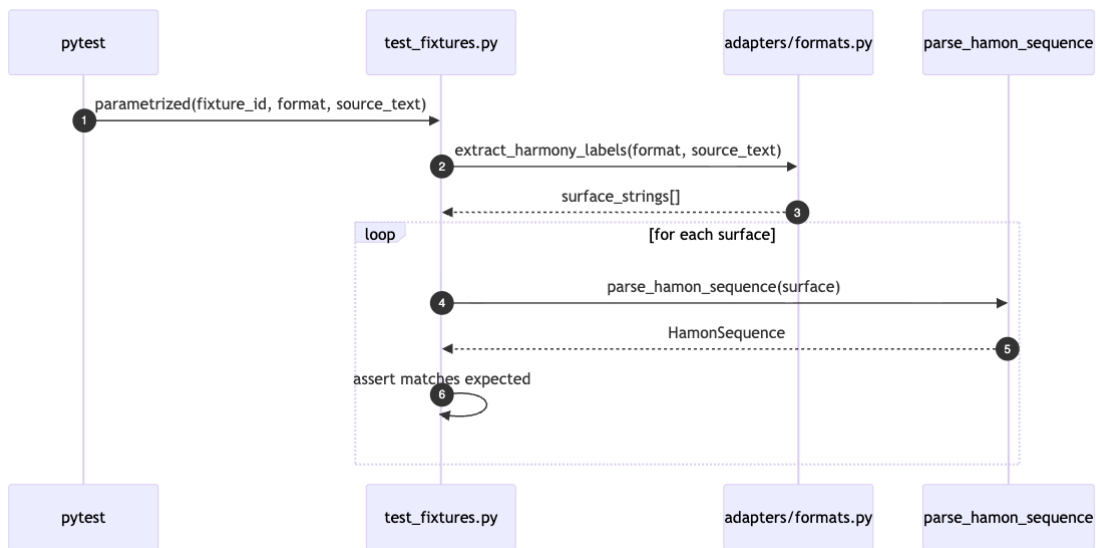


5) Chord symbol normalization — activity diagram

The chord symbol normalizer is deliberately permissive. It parses the pieces it recognizes and stashes whatever's left in rendering.rawParts.



6) Fixture test pipeline



7) Extension notes

Roman / number harmony normalization

- `normalizeRoman()` currently keeps tail as a surface string. Split it into inversion/figures/secondary relationships when you're ready.
- `normalizeNumberHarmony()` uses a **hyphen-digit** heuristic (6-5) to decide figured bass. Swap in a richer rule set when you need one.

Round-trip rendering

- `RenderingHints` records the explicit "how it was printed" choices (b vs. *b*, maj vs. Δ).
- surface is always preserved as-is. Semantics and rendering stay in separate layers.

HAMON Grammar Internals (Mermaid)

The **HAMON surface-first grammar**, as implemented in:

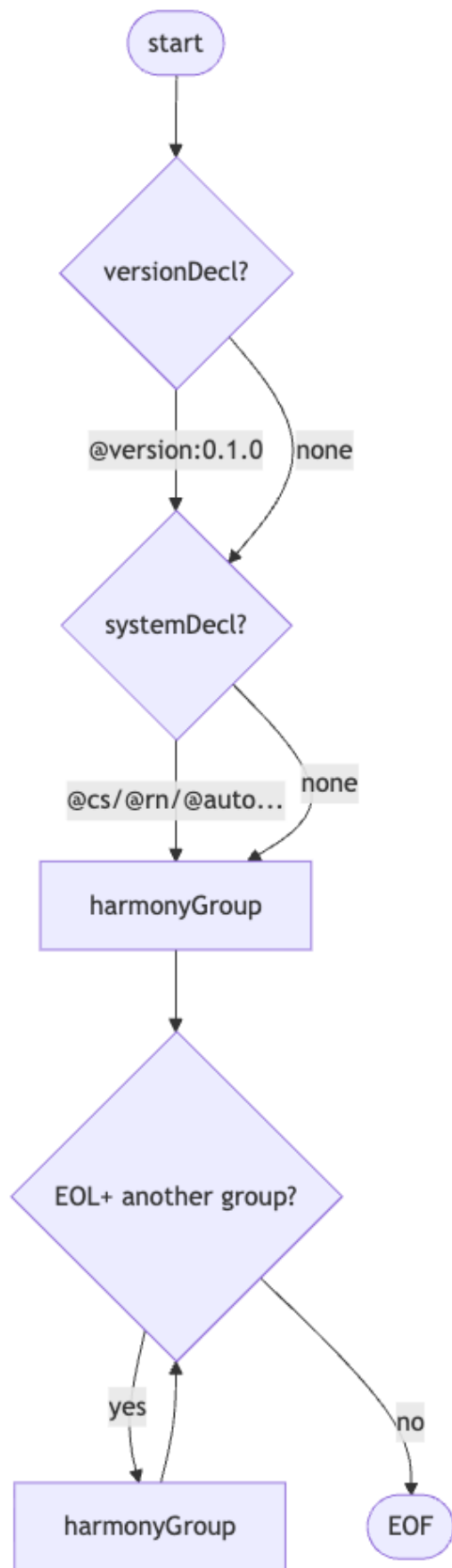
- antlr/hamonLexer.g4
- antlr/hamonParser.g4
- grammar/hamon.ebnf (RR-friendly EBNF mirror; railroad diagrams at documentation/railroad/index.html)

The goal is to parse **real-world written harmony labels**, keep the exact glyphs, and leave any ambiguity for the normalizers and heuristics to resolve later.

1) Top-level structure

A *sequence* may open with an optional version declaration and/or system hint, then carries one or more lines. Each line is a **harmonyGroup**.

- , separates labels inside a list.
- | separates alternatives.
- Newlines separate groups.



Grammar sketch

- `start` : `versionDecl? (EOL)* systemDecl? (EOL)* harmonyGroup ( (EOL)+ harmonyGroup )* (EOL)* EOF`
 - `versionDecl` : `VERSION_DECL` — single token matching `@version:M.m.p`
 - `harmonyGroup` : `harmonyList ( '|' harmonyList )*`
 - `harmonyList` : `harmony ( ',' harmony )*`
-

2) Version declaration and system hint

Version (@version:<semver>)

`versionDecl` : `VERSION_DECL`

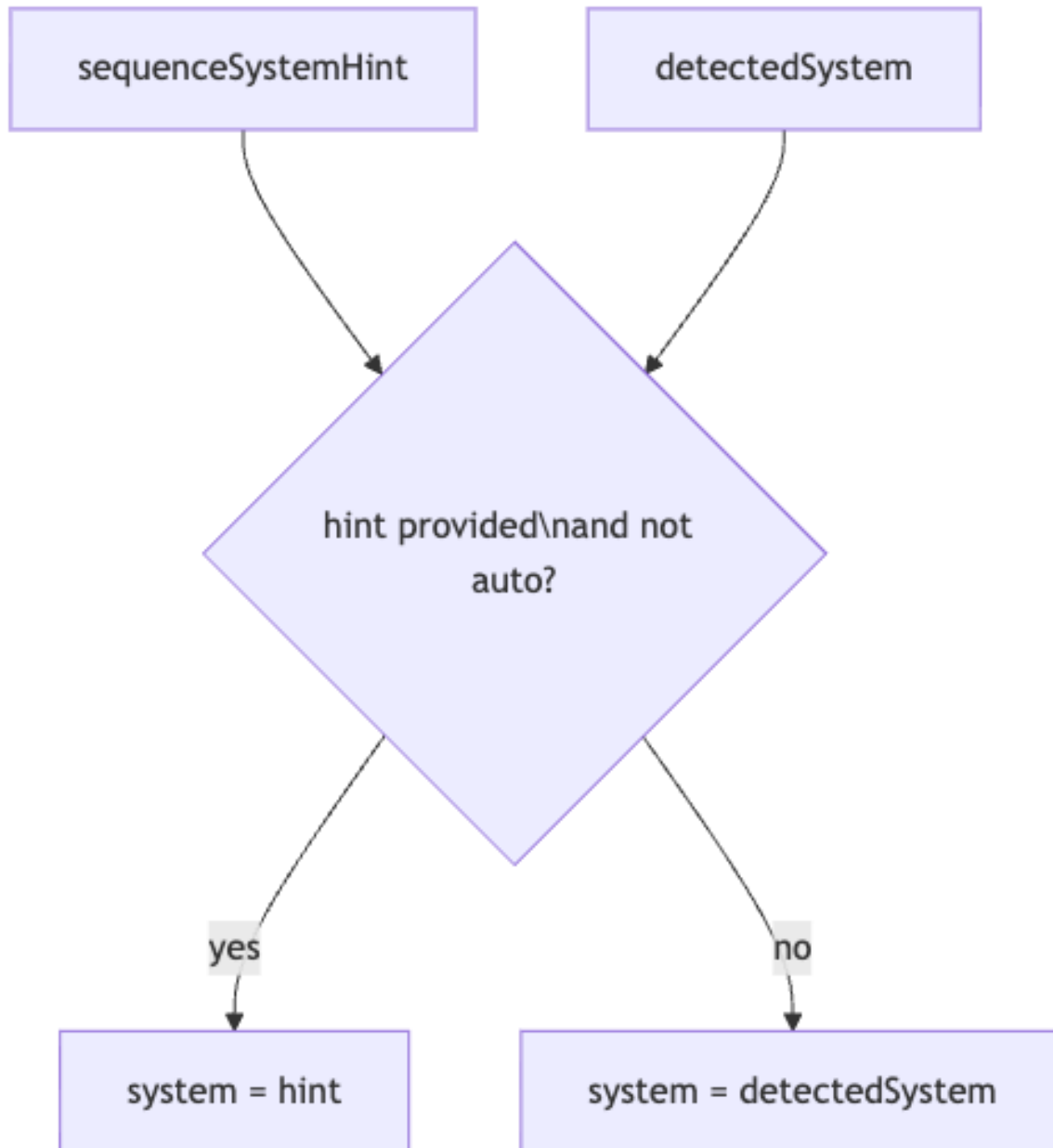
`VERSION_DECL` is a single lexer token matching `'@version:' [0-9]+ ('.' [0-9]+)*`. Longest-match makes it win over `AT ('@')`. It maps to `HamonSequence.version` (a string). Example: `@version:0.1.0`.

System hint (@cs/@rn/...) and system resolution

`systemDecl` : `'@' SYSTEM_ID`

Supported system IDs: `auto`, `cs`, `rn`, `ns`, `fb`, `fun`, `text`

Semantics: - The parser still detects a label family on its own. - The visitor then resolves system:
- `system` = `detectedSystem` when there's no hint or the hint is `@auto` - otherwise `system` = `hint` (and `detectedSystem` is kept alongside it)

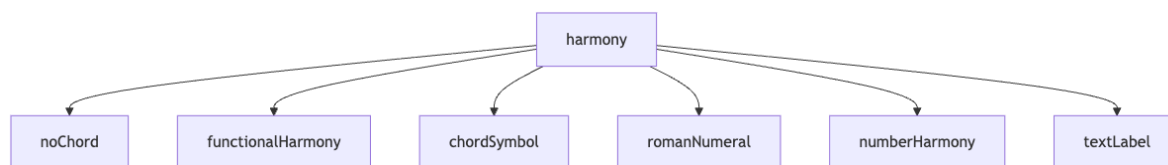


3) Label families (the harmony rule)

harmony is an ordered choice:

1. noChord
2. functionalHarmony
3. chordSymbol
4. romanNumeral
5. numberHarmony
6. textLabel

The order matters whenever a label could match more than one family — the first match wins.



4) Lexer overview (tokenization)

Key tokens (selected):

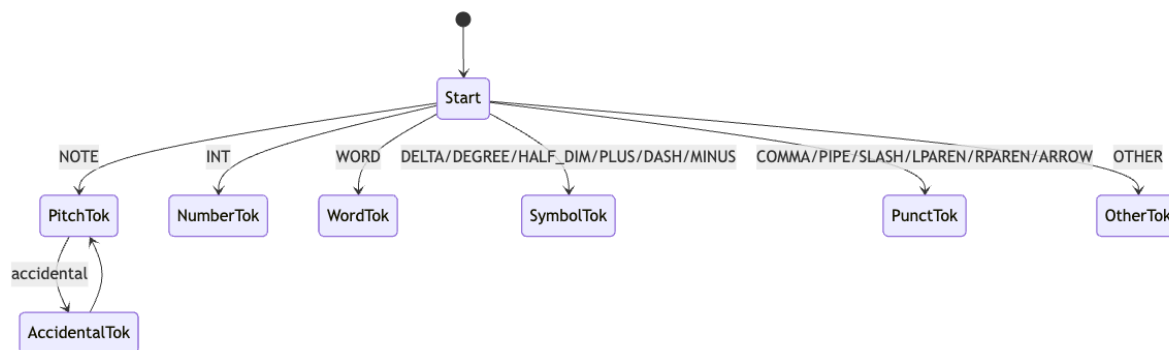
- Separators: COMMA ' , ', PIPE ' | ', EOL, WS (skipped)
- Directives: VERSION_DECL '@version:M.m.p' (whole version string as one token), AT '@', SYSTEM_ID (auto|cs|rn|...)
- Punctuation: LPAREN ' (', RPAREN ') ', SLASH ' / ', DASH ' - ', ARROW ' -> '
- Pitch: NOTE [A-G] and accidentals (SHARP, FLAT, DBLSHARP, DBLFLAT, NATURAL)
- Quality-ish words: MAJ, MIN, DIM, AUG, ADD, OMIT, NO, SUS
- Symbols: DELTA ' Δ ', DEGREE ' ° ', HALF_DIM ' ø ', PLUS ' + ', MINUS ' - '
- Roman numeral token: ROMAN (common roman strings)
- Numbers: INT [0-9]+
- Fallbacks: WORD [A-Za-z]+, OTHER .

DASH ' - ' vs MINUS ' − '

- ASCII - is DASH, used structurally (as in figured-bass 6-5).
- Unicode minus − tokenizes separately as MINUS and counts as a quality marker (minor).

Maximal munch and Roman-with-quality

Under ANTLR's maximal-munch rule, WORD matches iim (3 chars) before ROMAN can match ii (2 chars). To recover, the normalizer has a fallback regex, `_ROMAN_WITH_QUALITY_RE`, that catches patterns like iim7 or Vm7 once the textLabel path has been taken.



5) Chord symbols

Rule:

- chordSymbol : pitch chordPart* slashBass?
- pitch : NOTE accidental?
- slashBass : '/' pitch

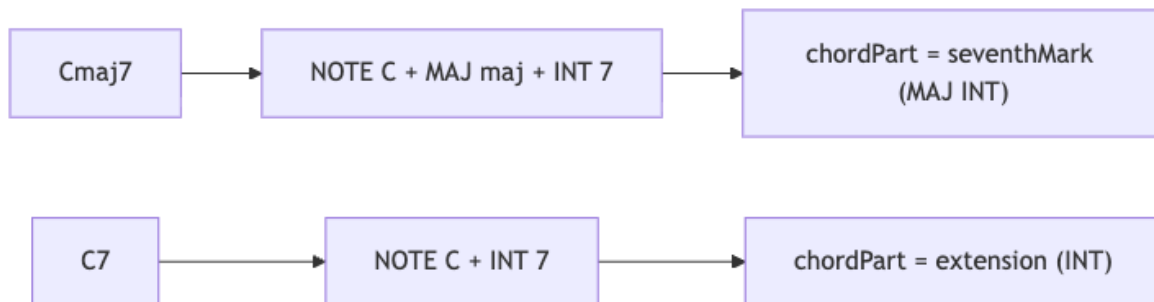
chordPart and losslessness

chordPart is permissive. Anything it doesn't recognize is still kept as WORD/OTHER, and later lands in rendering.rawParts.

Disambiguation: maj7 vs maj + 7 To keep these apart, chordPart tries seventhMark **before** qualityMark:

- seventhMark : DELTA INT? | MAJ INT
- qualityMark : MAJ | MIN | DIM | AUG | DEGREE | HALF_DIM | PLUS | DASH | MINUS

The result: - Cmaj7 tokenizes as NOTE MAJ INT, and MAJ INT parses as seventhMark. - C7 parses as extension.



Parentheses

parenGroup : '(' parenItem* ')'

parenItem covers alteration, seventhMark, INT, WORD, qualityMark, SLASH, DASH, OTHER.

That's what lets surface forms like these through: - C7(#11) - C(maj7) - C7(b9 #11)

6) Roman numerals

Rule:

- romanNumeral : rnAccidental* ROMAN rnTail* rnSecondary?
- rnSecondary : '/' rnAccidental* ROMAN

Roman numerals are *surface-first*: the grammar accepts tails (figures, quality-ish markers, or other characters) without fully interpreting them yet.

Examples: - bII65/V - #iv°7

7) Number harmony (Nashville vs figured bass)

Rule:

- numberHarmony : rnAccidental* INT numberTail* numberSecondary?
- numberTail contains a **structural figured-bass signal**: (DASH INT)

So 6-5 gets parsed structurally, and the normalizer later reuses that same signal (hyphen-digit) to decide fb vs. ns.

Examples: - Nashville-ish: b7, 1m, 5/7 - Figured-bass-ish: 6-5, 4-3, 7-6

8) Text labels

Rule:

- `textLabel` : `QUOTED_TEXT` | `rawRun`
- `rawRun` : `rawAtom+`

This is the catch-all that keeps a label **lossless** when it matches none of the known families. The normalizer then runs a few more heuristics (including `_ROMAN_WITH_QUALITY_RE`) before it settles on `TextSemantic`.

9) Practical notes for evolution

- Adding new systems (more RN flavors, say)? Do it in two steps:
 - 1) capture a permissive surface parse in ANTLR,
 - 2) push the semantic tightening into the normalizers.
- Keep tokens ordered so nothing surprises you:
 - specific tokens (MAJ, ROMAN) must come before the generic ones (WORD, OTHER).
- After editing `antlr/*.g4` or `grammar/hamon.ebnf`, run:
 - `./hamonpy/antlr.sh` to regenerate the Python parsers
 - `./tools/scripts/railroad.sh` to regenerate railroad diagrams
 - (The git pre-commit hook automates the railroad step.)

HAMON ↔ Verovio integration

Overview

Verovio (/Users/drizo/githubs/verovio/gabc/verovio/) is a C++ library that renders MEI music scores to SVG. The custom build in `verovio-hamon/compile_verovio.sh` produces a CLI binary from the local Verovio source — currently the `develop-gabc` branch, which adds GABC support.

HAMON and Verovio meet at MEI's `<harm>` element. HAMON labels ride inside `<harm>` elements in MEI files, and Verovio renders those elements above the staff.

How Verovio currently renders `<harm>`

`src/view_control.cpp` : `View::DrawHarm()` does the rendering: - If the `<harm>` has a `<fb>` child, it renders figured bass symbols. - Otherwise, it renders the text children as-is (the chord symbol surface string).

`src/harm.cpp` : `Harm::GetRootPitch()` parses that text to pull out a root note for transposition. It understands ASCII `#/b` accidentals and the Unicode sharp/flat glyphs that HAMON uses.

Conclusion: HAMON surface strings (`CΔ7`, `ii65`, `6-5`, `N.C.`) written as text inside `<harm>` elements render correctly today, with no C++ changes — Verovio simply treats them as styled text.

MEI customization (`mei-customization/`)

Two ODD files define the schema for MEI files that carry HAMON harmony:

File	Purpose
<code>hamon.odd</code>	Full HAMON MEI customization (all MEI modules including <code>MEI.harmony</code>)
<code>mei-verovio-hamon.xml.odd</code>	Verovio-specific subset (<code>mei-verovio</code> schema) — uses only modules Verovio supports

The Verovio-specific ODD (`mei-verovio-hamon.xml.odd`) starts from Verovio's own ODD and extends it with `MEI.harmony`. It controls which `<harm>` attributes Verovio accepts without raising validation errors.

Generate the RelaxNG schema from the ODD like this:

```
# Requires oXygen XML Editor or TEI Stylesheets
java -jar teigarage.jar --profile=relaxng mei-verovio-hamon.xml.odd
```

Integration pipeline

hamon label (surface string)

↓

hamon→MEI serializer (hamonpy MEI adapter, hamon_to_mei)
produces: <harm tstamp="1" place="above">CΔ7</harm>

↓

MEI file (validates against mei-verovio-hamon RelaxNG schema)

↓

Verovio CLI: verovio --from mei --to svg input.mei
produces: SVG with chord symbols rendered above the staff

The reverse direction — reading <harm> back out of an MEI file — runs entirely in hamonpy:

MEI file with <harm> elements

↓

hamonpy MEI adapter: mei_to_hamon / read_harmony_labels("mei", ...)

↓

surface strings

↓

parse_hamon_sequence → HamonSequence AST

Build the custom Verovio binary

```
cd verovio-hamon
sh compile_verovio.sh          # builds the CLI tool from the local source
# binary ends up at verovio-hamon/verovio/tools/verovio (or build-cli/verovio)
```

The script builds the standard CMake target. Whatever hamon-specific rendering the GABC branch adds is compiled in here.

What is missing / roadmap

1. HAMON → MEI serializer (highest priority)

hamonpy's MEI adapter (hamon_to_mei) already serializes the analytical layer back to <harm> elements. For Verovio rendering it should also: - Create <harm> elements with @tstamp or @startid anchors - Write the surface string as text content (works with Verovio today) - Optionally write <fb> children for figured bass labels

2. RelaxNG schema generation

Automate the ODD → RelaxNG step in tools/scripts/. We need the schema to validate MEI files before handing them to Verovio.

3. Optional: rich chord rendering in Verovio

For properly typeset chord symbols (superscript 7, the Δ glyph, italic m), Verovio's DrawHarm would need to: - Parse the HAMON surface string (or read structured child elements) - Apply font, size, and glyph rules per token (root, quality, seventh, bass)

That means C++ changes in src/view_control.cpp, and possibly new MEI child elements inside <harm>. Define any new attributes in mei-verovio-hamon.xml.odd.

4. Figured bass column alignment

Verovio's `adjustharmgrpsspacingfunctor.cpp` handles horizontal spacing for stacked `<harm>` groups. HAMON figured bass notation (6-5, 4-3, 7-6) uses a single `<harm>` with `<fb>` children, and these already render through the existing `DrawFb` path — no changes needed.

5. MEI export round-trip test

Once HAMON labels have been parsed from a Humdrum `**harm` spine into a `HamonSequence`, write them back to MEI with `hamon_to_mei` and check that the `<harm>` elements land in the right positions (it round-trips with `mei_to_hamon`).

Relevant Verovio source files

File	Role
<code>src/harm.cpp</code> / <code>include/vrv/harm.h</code>	<code><harm></code> element model, <code>GetRootPitch</code> for transposition
<code>src/view_control.cpp</code> : <code>DrawHarm()</code>	SVG rendering of <code><harm></code>
<code>src/adjustharmgrpsspacingfunctor.cpp</code>	Horizontal spacing between chord groups
<code>libmei/</code>	LibMEI schema / attribute accessors
<code>cmake/CMakeLists.txt</code>	Build options (e.g., <code>NO_MUSICXML_SUPPORT</code>)

Relevant HAMON source files

File	Role
<code>mei-customization/mei-verovio-hamon.xml.odd</code>	Verovio MEI schema with harmony
<code>mei-customization/hamon.odd</code>	Full HAMON MEI customization
<code>verovio-hamon/compile_verovio.sh</code>	Build script for custom Verovio
<code>hamonpy/hamonpy/adapters/mei.py</code>	Reads/writes <code><harm></code> (<code>mei_to_hamon</code> / <code>hamon_to_mei</code>)
<code>documentation/mei.md</code>	Notes on MEI harmony elements used by HAMON